

Лекции по C++

2 семестр

Дмитрий Величко

Март 2026

Содержание

1	Исключения в C++	9
1.1	Ключевое слово <code>throw</code>	9
1.1.1	Базовый синтаксис	9
1.1.2	Что можно бросать?	9
1.1.3	Что происходит при <code>throw</code> ?	9
1.1.4	<code>throw</code> как выражение	10
1.2	Раскрутка стека (Stack Unwinding)	10
1.2.1	Определение	10
1.2.2	Пошаговая механика	10
1.2.3	Визуализация стека	11
1.2.4	Что уничтожается, а что нет	11
1.2.5	Частично сконструированные объекты	12
1.2.6	Раскрутка стека и массивы	13
1.2.7	Когда раскрутка НЕ происходит	13
1.3	<code>try / catch</code>	13
1.3.1	Базовый синтаксис	13
1.3.2	Множественные блоки <code>catch</code>	14
1.3.3	Ловля по значению vs по ссылке	14
1.3.4	Вложенные <code>try/catch</code>	15
1.4	Приведение типов в <code>catch</code>	15
1.4.1	Наследование в <code>catch</code>	16
1.4.2	Правильный порядок	16
1.5	Где встречаются исключения в стандартной библиотеке	17
1.6	Runtime-ошибки vs Исключения C++	17
1.6.1	Три вида runtime-ошибок на уровне ОС	17
1.6.2	Пути к <code>std::terminate()</code>	18
1.7	Копирование при броске исключения	18
1.7.1	Где хранится объект исключения?	19
1.7.2	Оптимизация: <code>throw</code> временного объекта	19
1.7.3	Повторный проброс	20
1.8	Спецификатор <code>noexcept</code>	20
1.8.1	Что такое <code>noexcept</code> ?	20
1.8.2	Зачем это нужно?	21

1.8.3	Условный поехсепт	21
1.8.4	Что будет, если поехсепт-функция бросит?	22
1.8.5	Что пометить поехсепт?	22
1.9	Исключения в деструкторах	22
1.9.1	Почему нельзя бросать из деструктора?	22
1.9.2	Как обрабатывать ошибки в деструкторе?	23
1.9.3	Функция <code>std::uncaught_exceptions()</code>	23
1.10	Function-try-block	24
1.11	Гарантии безопасности исключений (Exception Safety)	25
1.11.1	Четыре уровня гарантий	25
1.11.2	Пример: строгая гарантия в <code>push_back</code>	25
1.11.3	Copy-and-swap idiom — строгая гарантия для <code>operator=</code>	26
1.12	<code>std::exception_ptr</code> — исключения между потоками	27
1.12.1	Основные функции	27
1.12.2	Пример: передача исключения между потоками	27
1.12.3	Пример: сбор нескольких исключений	28
1.13	<code>std::nested_exception</code> — вложенные исключения	28
1.13.1	Распаковка вложенных исключений	29
1.14	Стоимость исключений: zero-cost exception model	30
1.15	Итого: чек-лист по исключениям	30
2	Контейнеры и итераторы	32
2.1	<code>std::vector</code> : реализация изнутри	32
2.1.1	Почему нельзя писать <code>new T[newcap]</code>	32
2.1.2	Placement new	32
2.1.3	Почему нельзя <code>delete[] arr_</code>	32
2.1.4	Реализация <code>reserve</code>	32
2.1.5	Реализация <code>push_back</code>	33
2.2	<code>vector<bool></code> : специализация с битовым хранением	33
2.2.1	Прокси-класс <code>BitReference</code>	34
2.2.2	Трюк Ильи Мещерина: узнать тип выражения	34
3	Библиотека <code><algorithm></code>	36
3.1	Зачем она вообще нужна	36
3.2	<code>std::sort</code>	36
3.3	<code>std::find</code> и <code>std::find_if</code>	36
3.4	<code>std::count</code> и <code>std::count_if</code>	37
3.5	<code>std::for_each</code> и <code>std::transform</code>	37
3.6	<code>std::copy</code> и <code>std::back_inserter</code>	37
3.7	Идиома <code>erase-remove</code>	37
3.8	<code>std::accumulate</code> из <code><numeric></code>	37
4	Дек (<code>std::deque</code>)	39
4.1	Инваляция указателей и ссылок в <code>vector</code>	39
4.2	Два главных отличия дека от вектора	39
4.3	Внутреннее устройство дека	39
5	Адаптеры контейнеров	39
5.1	<code>std::stack</code>	40

6	Итераторы и их категории	41
6.1	Что такое итератор	41
6.2	Категории итераторов	41
6.3	iterator_traits	41
6.4	Реализация std::distance	42
6.5	Реализация итератора вектора	42
7	Потоковые итераторы	44
7.1	std::istream_iterator	44
7.2	std::ostream_iterator	44
7.3	Итераторы файловых потоков	44
8	Аллокаторы и управление памятью	46
8.1	Зачем аллокатор нужен контейнеру	46
8.2	Устройство стандартного аллокатора	46
8.3	Проблема: list аллоцирует не T	47
8.4	allocator_traits	47
8.4.1	Политики распространения аллокатора	48
8.5	Allocator-aware контейнер	48
8.6	Перегрузка operator new и operator delete	49
8.6.1	Глобальная перегрузка	49
8.6.2	Классовая перегрузка	49
8.6.3	Когда использовать что	50
9	Многопоточность в C++	51
9.1	Определение потока исполнения	51
9.1.1	Что такое поток?	51
9.1.2	Аналогия	51
9.1.3	Логическая многопоточность vs Аппаратный параллелизм	51
9.1.4	Типы многозадачности	52
9.2	Работа с потоками в C++	52
9.2.1	Создание потока	52
9.2.2	Передача аргументов в поток	52
9.2.3	Использование лямбд	53
9.2.4	join() vs detach()	53
9.2.5	Опасность: забыть join/detach	54
9.3	Область памяти и состояние гонки (Data Race)	54
9.3.1	Что такое Memory Location?	54
9.3.2	Что такое Data Race?	54
9.3.3	Наглядный пример гонки	55
9.4	Распространённые ошибки и заблуждения	56
9.4.1	Ошибка №1: volatile «чинит» гонки	56
9.4.2	Ошибка №2: Привязка к одному ядру решает гонки	56
9.4.3	Ошибка №3: std::cout безопасен для многопоточности	57
9.5	Синхронизация: Мьютексы	57
9.5.1	Что такое мьютекс?	57
9.5.2	Базовый интерфейс	57
9.5.3	Методы мьютекса	58
9.5.4	Правила и UB при работе с мьютексами	58

9.6	RAII в многопоточности	59
9.6.1	Проблема голых lock/unlock	59
9.6.2	std::lock_guard — простая RAII-обёртка	59
9.6.3	std::unique_lock — гибкая обёртка	60
9.6.4	std::scoped_lock (C++17) — захват нескольких мьютексов	60
9.6.5	Сравнение RAII-обёрток	61
9.7	Проектирование потокобезопасных объектов	61
9.7.1	Уровни потокобезопасности	61
9.7.2	API Race — коварная ловушка	62
9.8	Дедлоки (Deadlocks)	63
9.8.1	Что такое дедлок?	63
9.8.2	Классический пример: перевод денег	63
9.8.3	Задача об обедающих философах	64
9.8.4	Решения в C++	64
9.8.5	4 необходимых условия дедлока (условия Коффмана)	65
9.9	Итого: шпаргалка	65
10	Приведение типов (касты) в C++	66
10.1	«Запрещённые заклинания» — зачем нужны касты?	66
10.1.1	Почему не один каст, а четыре?	66
10.2	C-style cast — «Грубая сила»	66
10.2.1	Синтаксис	66
10.2.2	Как это работает внутри?	66
10.2.3	Почему это плохо?	67
10.2.4	Главная опасность	67
10.3	static_cast — «Безопасное преобразование»	67
10.3.1	Суть	67
10.3.2	Сценарий 1: Числовые преобразования	67
10.3.3	Сценарий 2: Enum ↔ int	68
10.3.4	Сценарий 3: void* → конкретный тип	68
10.3.5	Сценарий 4: Преобразование через конструктор	69
10.3.6	Сценарий 5: Приведение в иерархии наследования (downcast)	69
10.3.7	Чего static_cast НЕ может	69
10.4	reinterpret_cast — «Рентген памяти»	70
10.4.1	Суть	70
10.4.2	Аналогия	70
10.4.3	Пример 1: Заглядываем «внутрь» float	70
10.4.4	Пример 2: Указатель → число и обратно	70
10.4.5	Пример 3: Работа с сетевыми пакетами	71
10.4.6	reinterpret_cast НЕ может снять const	71
10.4.7	Когда reinterpret_cast оправдан?	71
10.5	const_cast — «Снятие защиты»	72
10.5.1	Суть	72
10.5.2	Мягкая форма: добавление const	72
10.5.3	Жёсткая форма: снятие const	72
10.5.4	Главное правило и UB	73
10.5.5	Реальный use case: избежание дублирования кода	73
10.6	dynamic_cast — «Безопасный детектив» (краткий обзор)	74

10.7	Сводная таблица: какой каст когда	74
10.8	Упражнения	75
10.8.1	Упражнение 1: Найди ошибку	75
10.8.2	Упражнение 2: Перепиши без C-style cast	76
10.8.3	Упражнение 3: Что выведет программа?	76
10.8.4	Упражнение 4: Реализуй безопасное приведение	77
10.8.5	Упражнение 5: Детектив по кастам	78
10.8.6	Упражнение 6: Реализуй type-punning безопасно	79
10.9	Итоговая шпаргалка	80
11	Иерархия <code>std::exception</code>	81
11.1	Зачем нужна единая иерархия исключений?	81
11.2	Класс <code>std::exception</code>	81
11.2.1	Интерфейс	81
11.2.2	Пример использования	81
11.3	Дерево стандартных исключений	82
11.4	<code>std::logic_error</code> и <code>std::runtime_error</code>	82
11.4.1	Конструкторы	82
11.4.2	Когда что бросать?	83
11.5	Наследники <code>std::logic_error</code>	83
11.5.1	<code>std::invalid_argument</code>	83
11.5.2	<code>std::out_of_range</code>	83
11.5.3	<code>std::length_error</code>	84
11.5.4	<code>std::domain_error</code>	84
11.6	Наследники <code>std::runtime_error</code>	84
11.6.1	<code>std::overflow_error</code> и <code>std::underflow_error</code>	84
11.6.2	<code>std::system_error</code> (C++11)	85
11.7	Отдельные исключения (не наследуют <code>logic/runtime_error</code>)	85
11.7.1	<code>std::bad_alloc</code>	85
11.7.2	<code>std::bad_cast</code>	86
11.8	Создание собственных исключений	86
12	<code>std::optional</code> (C++17)	88
12.1	Зачем нужен <code>std::optional</code> ?	88
12.2	Создание	88
12.2.1	Конструкторы	88
12.3	Проверка наличия значения	89
12.3.1	<code>has_value()</code> и <code>operator bool()</code>	89
12.4	Доступ к значению	89
12.4.1	<code>operator*()</code> и <code>operator->()</code>	89
12.4.2	<code>value()</code>	89
12.4.3	<code>value_or(default)</code>	90
12.5	Модификация	90
12.5.1	<code>emplace(args...)</code>	90
12.5.2	<code>reset()</code>	90
12.5.3	Присваивание	91
12.6	Сравнение	91
12.7	Монадический интерфейс (C++23)	91

12.7.1	<code>and_then(f)</code>	91
12.7.2	<code>transform(f)</code>	92
12.7.3	<code>or_else(f)</code>	92
12.8	Где хранится значение?	93
12.9	Типичные паттерны использования	93
12.9.1	Функция, которая может не вернуть результат	93
12.9.2	Ленивая инициализация (кэширование)	94
12.9.3	Необязательные параметры структуры	94
13	<code>std::expected</code> (C++23)	96
13.1	Зачем нужен <code>std::expected</code> ?	96
13.2	Создание	96
13.2.1	Создание со значением (успех)	96
13.2.2	Создание с ошибкой (неудача)	97
13.3	Проверка состояния	97
13.3.1	<code>has_value()</code> и <code>operator bool()</code>	97
13.4	Доступ к значению	97
13.4.1	<code>operator*()</code> и <code>operator->()</code>	97
13.4.2	<code>value()</code>	98
13.4.3	<code>value_or(default)</code>	98
13.5	Доступ к ошибке	98
13.5.1	<code>error()</code>	98
13.6	Модификация	98
13.6.1	<code>emplace(args...)</code>	98
13.6.2	Присваивание	99
13.7	Монадический интерфейс (C++23)	99
13.7.1	<code>and_then(f)</code>	99
13.7.2	<code>transform(f)</code>	99
13.7.3	<code>or_else(f)</code>	100
13.7.4	<code>transform_error(f)</code>	100
13.8	Где хранится значение?	100
13.9	<code>std::expected<void, E></code>	101
13.10	Типичные паттерны использования	101
13.10.1	Парсинг с информативной ошибкой	101
13.10.2	Цепочка операций	102
13.11	Сводная таблица: <code>optional</code> vs <code>expected</code> vs исключения	103
14	CRTP — Curiously Recurring Template Pattern	104
14.1	Что такое CRTP?	104
14.1.1	Ключевая идея	104
14.2	Зачем это нужно? Проблема виртуальных функций	105
14.3	Как работает <code>static_cast</code> в CRTP?	105
14.4	Применение 1: Статический полиморфизм	105
14.4.1	С виртуальными функциями (обычный подход)	106
14.4.2	С CRTP (статический полиморфизм)	106
14.5	Применение 2: Добавление функциональности (mixins)	107
14.5.1	Пример: автоматические операторы сравнения	107
14.5.2	Пример: автоматический <code>operator!=</code> и постфиксный <code>++</code>	108

14.6	Применение 3: Подсчёт объектов	109
14.7	Применение 4: Singleton через CRTP	110
14.8	Применение 5: Метод clone() без повторения кода	111
14.9	Применение 6: Статический интерфейс («концепты до концептов»)	112
14.10	Защита от ошибок	113
14.10.1	Проблема: неправильный шаблонный аргумент	113
14.10.2	Решение: проверка в конструкторе	113
14.10.3	Решение C++20: static_assert с концептами	114
14.11	CRTP и множественное наследование	114
14.12	CRTP vs виртуальные функции: когда что?	115
14.13	Пример из реального мира: std::enable_shared_from_this	116
14.14	Deducing this (C++23) — замена CRTP?	116
14.15	Итого: шпаргалка по CRTP	117
15	Указатели на функции, методы, поля. Лямбды. Type erasure. std::function	118
15.1	Указатели на функции	118
15.1.1	Что такое указатель на функцию	118
15.1.2	Использование typedef и using	118
15.1.3	Передача указателя на функцию как параметр	118
15.1.4	Массив указателей на функции	119
15.1.5	Указатели на функции и шаблоны	119
15.1.6	Указатели на noexcept-функции (C++17)	119
15.2	Указатели на методы (member function pointers)	120
15.2.1	Синтаксис	120
15.2.2	Синтаксис с using	120
15.2.3	Указатели на const-методы	120
15.2.4	Указатели на статические методы	121
15.2.5	Использование std::invoke (C++17)	121
15.3	Указатели на поля (pointer to data member)	121
15.3.1	Синтаксис	121
15.3.2	Практическое применение: обобщённый доступ	122
15.3.3	Доступ через std::invoke	122
15.4	Лямбда-выражения	123
15.4.1	Мотивация	123
15.4.2	Базовый синтаксис	123
15.4.3	Что такое лямбда на самом деле?	123
15.4.4	Захват переменных (capture)	124
15.4.5	Захват this	125
15.4.6	Init-capture (обобщённый захват, C++14)	125
15.4.7	Ключевое слово mutable	126
15.4.8	Лямбда без захвата конвертируется в указатель на функцию	126
15.4.9	Generic-лямбды (C++14)	126
15.4.10	Шаблонные лямбды (C++20)	127
15.4.11	Immediately Invoked Lambda Expression (IILE)	128
15.5	Указатели на функции, функторы и лямбды — сравнение	128
15.6	Type erasure (стирание типа)	128
15.6.1	Проблема	128
15.6.2	Идея type erasure	128

15.6.3	Пример: собственный AnyCallable	129
15.6.4	Как это работает (диаграмма)	130
15.6.5	Типе erasure в стандартной библиотеке	130
15.7	std::function	131
15.7.1	Что это	131
15.7.2	Проверка на пустоту	131
15.7.3	Накладные расходы std::function	132
15.7.4	Хранение указателей на методы	132
15.7.5	std::move_only_function (C++23)	133
15.7.6	Пример: система событий (observer pattern)	133
15.8	Итого	135
16	Move-семантика в C++: value categories, rvalue references, perfect forwarding	135
16.1	Value Categories (Категории значений)	135
16.1.1	Lvalue (Long-lived)	135
16.1.2	Rvalue (Short-lived)	136
16.1.3	Тернарный оператор и оператор запятая	136
16.2	RValue Reference (Rvalue-ссылки)	136
16.3	Move Semantics (Семантика перемещения)	136
16.3.1	std::move	137
16.4	Perfect Forwarding (Идеальная передача)	137
16.4.1	Универсальные ссылки (Universal References)	137
16.4.2	Сворачивание ссылок (Reference Collapsing)	137
16.4.3	std::forward	138
16.5	Variadic Templates (Шаблоны с переменным количеством аргументов)	138

1. Исключения в C++

1.1. Ключевое слово throw

1.1.1. Базовый синтаксис

Рассмотрим функцию деления:

```
1 #include <stdexcept>
2
3 int divide(int a, int b) {
4     if (b == 0) {
5         throw std::invalid_argument("division by zero");
6     }
7     return a / b;
8 }
```

При делении на ноль поведение программы неопределено, поэтому используется ключевое слово `throw`. Бросать можно **любое выражение**, однако принято бросать объекты стандартных исключений из `<stdexcept>`.

`throw` — один из способов выйти из функции наряду с `return`.

1.1.2. Что можно бросать?

Любой копируемый или перемещаемый объект:

```
1 throw 42; // int
2 throw std::string("error"); // std::string
3 throw std::runtime_error("fail"); // стандартноеисключение
4 throw 3.14; // double
5 throw std::vector<int>{1, 2, 3}; // дажевекторно (
    нестоит )
6
7 struct MyError { int code; std::string msg; };
8 throw MyError{404, "not found"}; //
```

пользовательскаяструктура

Рекомендация: бросайте только наследников `std::exception`. Это позволяет единообразно ловить все ошибки через `catch(const std::exception&)`.

1.1.3. Что происходит при throw?

1. Объект исключения **копируется** (или перемещается) в специальную область памяти, управляемую рантаймом.
2. Начинается **раскрутка стека** (stack unwinding).
3. Управление передаётся первому подходящему блоку `catch`.
4. Если подходящего `catch` нет — вызывается `std::terminate()`.

1.1.4. throw как выражение

throw — это выражение типа void. Его можно использовать в тернарном операторе и других контекстах:

```
1 int safe_divide(int a, int b) {
2     return b != 0 ? a / b : throw std::invalid_argument("zero
3     divisor");
4 }
5 // Влямбде :
6 auto get_or_throw = [](std::optional<int> opt) -> int {
7     return opt.has_value() ? *opt : throw std::runtime_error("
8     empty");
9 };
```

1.2. Раскрутка стека (Stack Unwinding)

1.2.1. Определение

При выбросе исключения происходит **раскрутка стека** (stack unwinding) — автоматическое уничтожение всех локальных объектов в обратном порядке их создания, с вызовом деструкторов, по цепочке вызовов от точки throw до подходящего блока catch.

1.2.2. Пошаговая механика

```
1 #include <iostream>
2 #include <stdexcept>
3
4 struct Guard {
5     std::string name;
6     Guard(std::string n) : name(std::move(n)) {
7         std::cout << " + " << name << "\n";
8     }
9     ~Guard() {
10         std::cout << " - " << name << "\n";
11     }
12 };
13
14 void f3() {
15     Guard g("f3_local");
16     throw std::runtime_error("boom"); // (1) Бросаем исключение
17     // Код ниже НИКОГДА не выполнится
18     std::cout << "after throw\n";
19 }
20
21 void f2() {
22     Guard g("f2_local");
23     f3(); // (2) Вызываем f3
```

```

24     std::cout << "after f3\n";           // Невыполнится
25 }
26
27 void f1() {
28     Guard g("f1_local");
29     f2();                               // (3) Вызываем f2
30     std::cout << "after f2\n";           // Невыполнится
31 }
32
33 int main() {
34     try {
35         f1();
36     } catch (const std::exception& e) {
37         std::cout << "Caught: " << e.what() << "\n";
38     }
39 }

```

Вывод:

```

+ f1_local
+ f2_local
+ f3_local
- f3_local      <- раскрутка: уничтожается f3_local
- f2_local      <- раскрутка: уничтожается f2_local
- f1_local      <- раскрутка: уничтожается f1_local
Caught: boom

```

Обратите внимание: деструкторы вызываются в **обратном порядке** создания (LIFO), а строки "after ..." не выполняются — управление «перепрыгивает» через них.

1.2.3. Визуализация стека

Стек вызовов в момент

throw:

```

| f3() | <- throw runtime_error("boom")
| f2() | ~Guard("f3_local")
| f1() | ~Guard("f2_local")
| main() | ~Guard("f1_local")
+-----+ catch (const std::exception& e) <- сюда попадаем

```

Рантайм «разматывает» стек снизу вверх: для каждого кадра уничтожает локальные объекты, затем переходит к следующему, пока не найдёт catch.

1.2.4. Что уничтожается, а что нет

Объект	Уничтожается?	Почему
Локальные переменные на стеке	Да	RAII
Объекты, созданные <code>new</code>	Нет	Нет владельца на стеке
<code>std::unique_ptr</code>	Да	Деструктор вызовет <code>delete</code>
<code>std::shared_ptr</code>	Да	Деструктор уменьшит счётчик
Сырые указатели (<code>int*</code>)	Да*	Но память не освобождается!

*Переменная-указатель уничтожается, но `delete` для неё никто не вызывает — это **утечка памяти**.

```

1 void leak_example() {
2     int* raw = new int(42);           // Выделили память
3     std::unique_ptr<int> safe(new int(43)); // RAII-обёртка
4
5     throw std::runtime_error("oops");
6
7     delete raw; // НИКОГДА не выполнится -> утечка!
8     // safe -- деструктор unique_ptr вызовет delete автоматически
9 }

```

Мораль: именно поэтому RAII критически важен. Любой ресурс (память, файл, мьютекс, соединение) должен быть обёрнут в объект с деструктором.

1.2.5. Частично сконструированные объекты

Если исключение вылетает **из конструктора**, деструктор этого объекта **не вызывается** (объект не считается созданным). Но деструкторы уже инициализированных полей **вызываются**:

```

1 struct Resource {
2     Resource(int id) { std::cout << "R(" << id << ")\n"; }
3     ~Resource()      { std::cout << "~R\n"; }
4 };
5
6 struct Widget {
7     Resource a, b, c;
8
9     Widget() : a(1), b(2), c(3) {
10         // Допустим, конструктор c бросил исключение
11         // Тогда:
12         //   a -- уничтожается был ( создан )
13         //   b -- уничтожается был ( создан )
14         //   c -- НЕ уничтожается не ( был до конца создан )
15         //   ~Widget() -- НЕ вызывается (Widget не был создан )
16     }
17 };

```

```

1 struct Dangerous {
2     Resource a{1};
3     Resource b{2};
4 }

```

```

5     Dangerous() {
6         Resource* raw = new Resource(3);
7         throw std::runtime_error("oops");
8         delete raw; // Утечка! ~Dangerous() не вызовется ,
9                     // но ~a и ~b вызовутся
10    }
11 };

```

1.2.6. Раскрутка стека и массивы

При раскрутке массива деструкторы вызываются только для **уже сконструированных** элементов:

```

1  struct Item {
2      static int count;
3      Item() {
4          if (++count == 4) throw std::runtime_error("4th item");
5          std::cout << "Item(" << count << ")\n";
6      }
7      ~Item() { std::cout << "~Item\n"; }
8  };
9  int Item::count = 0;
10
11 void test() {
12     Item arr[5];
13     // Создаётся Item(1), Item(2), Item(3)
14     // Item(4) бросает исключение
15     // Раскрутка: ~Item для arr[2], arr[1], arr[0] в(
16     // обратном порядке )
17     // arr[3] и arr[4] не были созданы -- деструкторы не вызываются
18 }

```

1.2.7. Когда раскрутка НЕ происходит

- **Не пойманное исключение:** если нигде нет подходящего `catch`, стандарт гарантирует только вызов `std::terminate()`. Раскрутка стека *implementation-defined* — может произойти, а может и нет.
- **`std::abort()` / `std::_Exit()`:** немедленное завершение без раскрутки.
- **Нарушение `noexcept`:** если функция помечена `noexcept` и из неё вылетает исключение — `std::terminate()` без гарантированной раскрутки.

1.3. try / catch

1.3.1. Базовый синтаксис

```

1  #include <iostream>
2  #include <stdexcept>
3

```

```

4 int main() {
5     try {
6         divide(1, 0);
7     } catch (const std::exception& e) {
8         std::cout << e.what() << "\n";
9     }
10 }

```

- Метод `what()` определён в `std::exception` и возвращает строку с описанием ошибки — ту самую, что была передана при создании объекта исключения.
- Исключение следует ловить **по константной ссылке**, чтобы избежать лишнего копирования и сохранить полиморфизм.

1.3.2. Множественные блоки `catch`

```

1 try {
2     do_something();
3 } catch (const std::out_of_range& e) {
4     // Обработка выхода за границы
5     std::cerr << "Index error: " << e.what() << "\n";
6 } catch (const std::invalid_argument& e) {
7     // Обработка неверного аргумента
8     std::cerr << "Bad argument: " << e.what() << "\n";
9 } catch (const std::exception& e) {
10    // Все остальные стандартные исключения
11    std::cerr << "Error: " << e.what() << "\n";
12 } catch (...) {
13    // Абсолютно любое исключение включая (int, string итд ..)
14    std::cerr << "Unknown error\n";
15 }

```

Правила:

- Блоки `catch` проверяются **сверху вниз**, побеждает **первый подходящий**.
- `catch(...)` ловит **всё**, поэтому должен быть последним.
- Более специфичные обработчики ставятся **перед** более общими.

1.3.3. Ловля по значению vs по ссылке

```

1 // ПЛОХО: ловля по значению -- срезка (slicing)!
2 try {
3     throw std::runtime_error("specific error");
4 } catch (std::exception e) {
5     // e -- копия, тип СРЕЗАН до std::exception
6     // e.what() может вернуть нето , что ожидали
7 }

```

```

8
9 // ХОРОШО: ловля по константной ссылке
10 try {
11     throw std::runtime_error("specific error");
12 } catch (const std::exception& e) {
13     // e -- ссылка на исходный объект, полиморфизм сохранён
14     // e.what() вернёт "specific error"
15 }

```

При ловле по значению происходит **object slicing**: если бросили наследника, а поймали базовый тип, часть данных наследника **теряется**.

1.3.4. Вложенные try/catch

```

1 void process() {
2     try {
3         try {
4             throw std::runtime_error("inner");
5         } catch (const std::logic_error& e) {
6             std::cout << "logic: " << e.what() << "\n"; //
              Неподходит
7         }
8         // runtime_error не пойман внутренним catch -- летит дальше
9     } catch (const std::exception& e) {
10        std::cout << "outer: " << e.what() << "\n"; // "outer:
              inner"
11    }
12 }

```

1.4. Приведение типов в catch

В catch **не работают** обычные неявные преобразования и правила перегрузки. Доступны только:

- Добавление const / приведение к ссылке
- Приведение типа потомка к типу родителя

```

1 try {
2     throw 1; // int
3 } catch (double d) {
4     std::cout << "double\n"; // не поймает : int -> double
              не работает
5 } catch (long long l) {
6     std::cout << "long long\n"; // не поймает : int -> long long
              не работает
7 } catch (...) {
8     std::cout << "other\n"; // поймает
9 }

```

1.4.1. Наследование в catch

Блоки catch проверяются **сверху вниз**, побеждает первый подходящий:

```
1 struct Mom {};  
2 struct Son : Mom {};  
3  
4 int main() {  
5     try {  
6         throw Son{};  
7     } catch (const Mom& m) {  
8         std::cout << "Mom\n"; // поймает: Son приводится к Mom  
9     } catch (const Son& s) {  
10        std::cout << "Son\n"; // сюда не дойдём  
11    } catch (...) {  
12        std::cout << "other\n";  
13    }  
14 }
```

Вывод: Mom. Именно поэтому обработчики потомков нужно ставить **перед** обработчиками родителей, иначе они никогда не сработают.

1.4.2. Правильный порядок

```
1 try {  
2     throw Son{};  
3 } catch (const Son& s) {  
4     std::cout << "Son\n"; // Теперь поймает !  
5 } catch (const Mom& m) {  
6     std::cout << "Mom\n"; // Ловит остальных наследников Mom  
7 } catch (...) {  
8     std::cout << "other\n";  
9 }
```

1.5. Где встречаются исключения в стандартной библиотеке

Операция	Исключение
<code>vector::at(i)</code>	<code>std::out_of_range</code> при <code>i >= size()</code>
<code>operator new</code>	<code>std::bad_alloc</code> при нехватке памяти
<code>dynamic_cast<T&>(ref)</code>	<code>std::bad_cast</code> при неудачном приведении
<code>std::stoi("abc")</code>	<code>std::invalid_argument</code> (не число)
<code>std::stoi("99999999999")</code>	<code>std::out_of_range</code> (переполнение)
<code>map::at(key)</code>	<code>std::out_of_range</code> при отсутствии ключа
<code>optional::value()</code>	<code>std::bad_optional_access</code> если пуст
<code>variant::get<T>()</code>	<code>std::bad_variant_access</code> при неверном типе
<code>thread::join()</code> на отсоединённом	<code>std::system_error</code>

1.6. Runtime-ошибки vs Исключения C++

Это принципиально разные вещи.

1.6.1. Три вида runtime-ошибок на уровне ОС

Сигнал	Типичные причины
SIGSEGV (Segfault)	Выход за пределы массива, разыменование <code>nullptr</code> , переполнение стека
SIGFPE (Floating Point Exception)	Деление целого на ноль, переполнение целых
SIGABRT (Aborted)	Вызов <code>std::abort()</code> , <code>std::terminate()</code> , падение <code>assert</code>

Эти ошибки **не являются** исключениями C++ и **не ловятся** через `catch`:

```
1 int arr[5];
2 try {
3     arr[10] = 42; // SIGSEGV -- catch не поймает
4 } catch (...) {
5     // сюда мы НЕ попадём
6 }
```

```

7
8 try {
9     int x = 1 / 0; // SIGFPE -- то же исключение C++
10 } catch (...) {
11     // и сюда не попадём
12 }

```

Важно: деление на ноль — это исключение на уровне *процессора*, не языка. Процессор обращается к ОС, которая посылает SIGFPE. Это совпадение терминологии, но разные уровни абстракции.

1.6.2. Пути к `std::terminate()`

- Не пойманное исключение (uncaught exception)
- Явный вызов `std::terminate()` или `std::abort()`
- Упавший `assert` (он напрямую вызывает `abort()`)
- Исключение из `noexcept`-функции
- Исключение из деструктора во время раскрутки стека
- Два одновременных активных исключения (double exception)

С точки зрения ОС программа, завершившаяся из-за не пойманного C++-исключения, получает вердикт SIGABRT.

Также стоит отметить, что `std::runtime_error` — это *частный случай* `std::exception`, то есть обычное C++-исключение, не имеющее отношения к SIGFPE/SIGSEGV/SIGABRT.

1.7. Копирование при броске исключения

```

1 #include <iostream>
2
3 struct A {
4     A()          { std::cout << "A()\n"; }
5     ~A()         { std::cout << "~A()\n"; }
6     A(const A&) { std::cout << "A(copy)\n"; }
7 };
8
9 void f(int x) {
10     A a;
11     if (x == 0) {
12         throw a; // объект копируется в специальную память
13     }
14 }
15
16 int main() {
17     try {
18         f(0);
19     } catch (...) {

```

```

20         std::cout << "caught!\n";
21     }
22 }

```

Порядок вывода:

1. A() — создание a на стеке
2. A(copу) — копирование в специальную память при throw
3. ~A() — уничтожение a при раскрутке стека
4. caught!
5. ~A() — уничтожение копии при выходе из catch

- catch(A a) — ещё одно копирование из дин. памяти на стек.
- catch(A& a) — работаем прямо с объектом исключения, без копирования.
- catch(const A& a) — то же, но без возможности модификации (рекомендуемый вариант).

1.7.1. Где хранится объект исключения?

Стандарт не специфицирует точное место хранения. На практике — в **динамической памяти**. Исключение: `std::bad_alloc` — при нехватке памяти нельзя выделить новую, поэтому компилятор заранее резервирует буфер в **статической памяти** (*emergency buffer*).

Исключения дороги по стоимости: выделение динамической памяти происходит в любом случае. Поэтому их не стоит использовать в горячих путях кода.

1.7.2. Оптимизация: throw временного объекта

```

1 // Лишнее копирование :
2 void bad() {
3     std::runtime_error err("fail");
4     throw err; // Копирование err в специальную память
5 }
6
7 // Без лишнего копирования :
8 void good() {
9     throw std::runtime_error("fail"); //
10         Объект создаётся сразу в спец. памяти
11                                     //
12                                     Компилятор может применить
13                                     copy elision
14 }

```

1.7.3. Повторный проброс

```
1 try {
2     f(0);
3 } catch (const A& a) {
4     throw;          // прокидываем то же исключение дальше без
                     // копирования)
5     // throw a; // создаём новый объект -- возможна срезка типа !
6 }
```

Критическая разница:

- `throw;` — пробрасывает **оригинальный** объект исключения. Тип сохраняется. Нет копирования.
- `throw a;` — создаёт **новый** объект, копируя `a`. Если реальный тип — наследник, происходит **object slicing**!

```
1 struct Base { virtual const char* what() const { return "Base"; }
2   };
3 struct Derived : Base { const char* what() const override { return
4   "Derived"; } };
5
6 void demonstrate_slicing() {
7     try {
8         throw Derived{};
9     } catch (const Base& e) {
10        std::cout << e.what() << "\n"; // "Derived" полиморфизм()
11        // throw;                       // Пробросит Derived --
12        //                             // правильно
13        throw e;                       // Пробросит Base --
14        СРЕЗКА!
15    }
16 }
```

Правило: всегда используйте `throw;` без аргумента для повторного проброса.

Если исключение не поймано нигде — гарантируется только вызов `std::terminate()`. Деструкторы при этом могут не вызваться.

1.8. Спецификатор `noexcept`

1.8.1. Что такое `noexcept`?

`noexcept` — спецификатор, обещающий компилятору, что функция **не бросает исключений**:

```
1 void safe_function() noexcept {
2     // Если отсюда вылетит исключение -- std::terminate()
3 }
4
5 int add(int a, int b) noexcept {
```

```

6     return a + b; // Действительно не бросает
7 }

```

1.8.2. Зачем это нужно?

1. **Оптимизация:** компилятор может генерировать более эффективный код, зная, что исключений не будет (не нужна таблица раскрутки для этой функции).
2. **Семантика `std::vector`:** при реаллокации вектор использует `std::move_if_noexcept`. Если конструктор перемещения помечен `noexcept` — элементы перемещаются. Если нет — **копируются** (для strong exception safety).
3. **Документация:** явное указание контракта функции.

```

1 class MyString {
2 public:
3     // Без noexcept -- vector будет КОПИРОВАТЬ при реаллокации
4     MyString(MyString&& other) { /* ... */ }
5
6     // С noexcept -- vector будет ПЕРЕМЕЩАТЬ быстрее (!)
7     // MyString(MyString&& other) noexcept { /* ... */ }
8 };
9
10 std::vector<MyString> vec;
11 vec.push_back(MyString()); // Реаллокация: move или copy?

```

1.8.3. Условный `noexcept`

`noexcept` может принимать выражение:

```

1 template<typename T>
2 void swap_values(T& a, T& b) noexcept(noexcept(T(std::move(a)))) {
3     T tmp = std::move(a);
4     a = std::move(b);
5     b = std::move(tmp);
6 }
7 // noexcept, если конструктор перемещения T -- noexcept

```

`noexcept(expr)` как оператор возвращает `true/false`:

```

1 static_assert(noexcept(42 + 1)); // true --
    арифметика не бросает
2 static_assert(!noexcept(std::stoi("42"))); // false -- stoi
    может бросить
3
4 void f() noexcept;
5 static_assert(noexcept(f())); // true -- f помечена
    noexcept

```

1.8.4. Что будет, если noexcept-функция бросит?

```
1 void oops() noexcept {
2     throw std::runtime_error("surprise!"); // Компилируется!
3 }
4
5 int main() {
6     try {
7         oops();
8     } catch (...) {
9         // Сюда мы НЕ попадём !
10    }
11    // Программа завершится через std::terminate()
12 }
```

Компилятор **не проверяет** тело noexcept-функции на наличие throw. noexcept — это **обещание** программиста, а не проверка компилятора. Нарушение — std::terminate()

1.8.5. Что помечать noexcept?

Функция	noexcept?
Деструкторы	Да (по умолчанию с C++11)
Move-конструкторы	Да, если возможно
Move-присваивание	Да, если возможно
swap()	Да, если возможно
Функции, работающие с памятью	Зависит
Функции с внешними вызовами	Обычно нет

Важно: деструкторы в C++11 и новее **неявно** noexcept. Исключение из деструктора — почти всегда std::terminate().

1.9. Исключения в деструкторах

1.9.1. Почему нельзя бросать из деструктора?

Деструкторы вызываются при раскрутке стека. Если во время раскрутки (уже есть активное исключение) деструктор бросит **второе** исключение — в C++ не может быть двух активных исключений одновременно, и вызывается std::terminate():

```
1 struct Bad {
2     ~Bad() noexcept(false) {
3         throw std::runtime_error("from destructor");
4     }
5 };
6
7 void example() {
8     try {
9         Bad b;
10        throw std::runtime_error("first");
11        // Начинается раскрутка стека
```

```

12         // Вызывается ~Bad(), которыйбросаетВТОРОЕисключение
13         // -> std::terminate()!
14     } catch (...) {
15         // Сюданедойдем
16     }
17 }

```

1.9.2. Как обрабатывать ошибки в деструкторе?

```

1  class FileWriter {
2      FILE* file_;
3  public:
4      ~FileWriter() {
5          // Вариант 1: проглотитьисключениеобычный ( подход)
6          try {
7              if (file_ && fclose(file_) != 0) {
8                  // Логируемошибку , ноНЕбросаем
9                  std::cerr << "Warning: failed to close file\n";
10             }
11         } catch (...) {
12             // Никогданепозволяемисключениюпокинутьдеструктор
13         }
14     }
15
16     // Вариант 2: явныйметод close() --
17     // пользовательвызоветзаранее
18     void close() {
19         if (file_ && fclose(file_) != 0) {
20             throw std::runtime_error("close failed");
21         }
22         file_ = nullptr;
23     };
24 }

```

1.9.3. Функция std::uncaught_exceptions()

C++17 добавил std::uncaught_exceptions() (с «s»), которая возвращает **количество** активных непойманных исключений. Это позволяет деструктору узнать, вызван ли он в процессе раскрутки:

```

1  class ScopeGuard {
2      int exceptions_on_entry_;
3      std::function<void()> on_success_;
4      std::function<void()> on_failure_;
5  public:
6      ScopeGuard(std::function<void()> success, std::function<void()
7          > failure)
8          : exceptions_on_entry_(std::uncaught_exceptions())
9            , on_success_(std::move(success))
10            , on_failure_(std::move(failure))

```

```

10     {}
11
12     ~ScopeGuard() {
13         if (std::uncaught_exceptions() > exceptions_on_entry_) {
14             on_failure_();    // Деструкторвызванизза - исключения
15         } else {
16             on_success_();    // Нормальныйвыходизобластивидимости
17         }
18     }
19 };
20
21 void test() {
22     ScopeGuard guard(
23         []{ std::cout << "Success!\n"; },
24         []{ std::cout << "Failure!\n"; }
25     );
26     throw std::runtime_error("oops");    // "Failure!"
27 }

```

1.10. Function-try-block

Особый синтаксис try/catch для **всего тела функции**, включая список инициализации конструктора:

```

1  class Connection {
2      DatabaseHandle db_;
3      Logger log_;
4
5  public:
6      Connection(const std::string& url)
7          try : db_(url), log_("connection.log")    // Если db_ или log_
              бросят --
8          {                                         // поймаетниже
9              db_.connect();
10         }
11         catch (const std::exception& e) {
12             std::cerr << "Failed to create Connection: " << e.what()
13                 << "\n";
14             // ВАЖНО:
15             дляконструктораисключениеавтоматическиперевбрасывается
16             !
17             // ОбъектНЕбылсоздан    , вернутьегонельзя
18         }
19 };

```

Особенности function-try-block в конструкторах:

- В catch блоке **нельзя** обратиться к полям объекта — они уже могут быть уничтожены.
- Исключение **автоматически перебрасывается** при выходе из catch. Подавить его невозможно — объект не был создан.

- Можно заменить исключение на другое через `throw other;`.

Для обычных функций `function-try-block` просто оборачивает всё тело:

```
1 int parse(const std::string& s)
2 try {
3     return std::stoi(s);
4 }
5 catch (const std::exception& e) {
6     std::cerr << "Parse failed: " << e.what() << "\n";
7     return -1; // Для обычных функций -- можно вернуть значение
8 }
```

1.11. Гарантии безопасности исключений (Exception Safety)

При проектировании функций и классов важно определить, какие **гарантии** они предоставляют в случае исключения.

1.11.1. Четыре уровня гарантий

Гарантия	Суть	Пример
No-throw	Функция никогда не бросает исключений	Деструкторы, <code>swap()</code> , <code>noexcept</code> -функции
Strong (строгая)	При исключении состояние объекта откатывается к исходному («всё или ничего»)	<code>vector::push_back</code> , <code>std::atomic</code> операции
Basic (базовая)	При исключении объект в <i>валидном</i> , но <i>неопределённом</i> состоянии. Утечек нет	<code>std::string::operator=</code> (если аллокация не удалась)
No guarantee	Никаких гарантий. Возможны утечки и повреждение данных	Код без RAII

1.11.2. Пример: строгая гарантия в `push_back`

```
1 template<typename T>
2 void vector<T>::push_back(const T& value) {
3     if (sz_ == cap_) {
4         // 1. Выделяем новую память
5         T* newarr = allocate(cap_ * 2); // Может бросить
6         bad_alloc
7     }
```

```

6 // Ностарыеданныенетронуты
7 // !
8 // 2. Копируемперемещаем/ элементы
9 try {
10     copy_or_move(arr_, newarr, sz_); // Можетбросить
11 } catch (...) {
12     deallocate(newarr); // Откатываемся:
13     освобождаемновуюпамять
14     throw; // Пробрасываем --
15     старыйвекторцел !
16 }
17 // 3. Успех: заменяемстароеенановое
18 deallocate(arr_);
19 arr_ = newarr;
20 cap_ *= 2;
21 }
22 new (arr_ + sz_) T(value); // Можетбросить --
23     новекторввалидномсостоянии
24 ++sz_;
25 }

```

1.11.3. Copy-and-swap idiom — строгая гарантия для operator=

```

1 class MyString {
2     char* data_;
3     size_t size_;
4 public:
5     // Строгаягарантиячерез copy-and-swap:
6     MyString& operator=(MyString other) { // (1) Копия --
7         можетбросить
8         swap(other); // (2) Swap --
9         noexcept
10        return *this; // (3)
11        Старыеданныеv other
12    } // other
13        уничтожитсявдеструкторе
14
15    void swap(MyString& other) noexcept {
16        std::swap(data_, other.data_);
17        std::swap(size_, other.size_);
18    }
19 };

```

Идея: копирование (шаг 1) может бросить, но исходный объект ещё не тронут. swap (шаг 2) гарантированно не бросает. Если шаг 1 бросил — исходный объект остался в прежнем состоянии (строгая гарантия).

1.12. `std::exception_ptr` — исключения между потоками

`std::exception_ptr` позволяет **захватить** исключение, сохранить его и **перебросить** позже — в том числе в другом потоке.

1.12.1. Основные функции

Функция	Описание
<code>std::current_exception()</code>	Захватывает текущее исключение (внутри <code>catch</code>) и возвращает <code>exception_ptr</code>
<code>std::rethrow_exception(ptr)</code>	Перебрасывает исключение, хранимое в <code>ptr</code>
<code>std::make_exception_ptr(e)</code>	Создаёт <code>exception_ptr</code> из объекта <code>e</code> без входа в <code>catch</code>

1.12.2. Пример: передача исключения между потоками

```
1  #include <exception>
2  #include <thread>
3  #include <iostream>
4
5  void worker(std::exception_ptr& eptr) {
6      try {
7          // Какаято работа, которая может бросить
8          throw std::runtime_error("error in worker thread");
9      } catch (...) {
10         eptr = std::current_exception(); // Захватываем исключение
11     }
12 }
13
14 int main() {
15     std::exception_ptr eptr = nullptr;
16
17     std::thread t(worker, std::ref(eptr));
18     t.join();
19
20     // Теперь в основном потоке :
21     if (eptr) {
22         try {
23             std::rethrow_exception(eptr); // Перебрасываем
24         } catch (const std::exception& e) {
25             std::cout << "From worker: " << e.what() << "\n";
26             // "From worker: error in worker thread"
27         }
28     }
29 }
```

1.12.3. Пример: сбор нескольких исключений

```
1 #include <exception>
2 #include <vector>
3
4 std::vector<std::exception_ptr> errors;
5
6 void process_item(int item) {
7     try {
8         if (item % 3 == 0)
9             throw std::runtime_error("bad item: " + std::to_string
10                                     (item));
11         // Обычная обработка ...
12     } catch (...) {
13         errors.push_back(std::current_exception());
14     }
15 }
16
17 int main() {
18     for (int i = 0; i < 10; ++i) {
19         process_item(i);
20     }
21
22     std::cout << "Errors: " << errors.size() << "\n";
23     for (auto& eptr : errors) {
24         try {
25             std::rethrow_exception(eptr);
26         } catch (const std::exception& e) {
27             std::cout << "  - " << e.what() << "\n";
28         }
29     }
30 }
```

1.13. std::nested_exception — вложенные исключения

Иногда при обработке одного исключения возникает другое. C++11 позволяет **вложить** оригинальное исключение внутрь нового:

```
1 #include <exception>
2 #include <stdexcept>
3 #include <iostream>
4
5 void low_level() {
6     throw std::runtime_error("disk I/O error");
7 }
8
9 void mid_level() {
10     try {
11         low_level();
12     } catch (...) {
```

```

13         // std::throw_with_nested
           обворачивает текущее исключение в новое
14         std::throw_with_nested(
15             std::runtime_error("mid_level failed")
16         );
17     }
18 }
19
20 void high_level() {
21     try {
22         mid_level();
23     } catch (...) {
24         std::throw_with_nested(
25             std::runtime_error("high_level failed")
26         );
27     }
28 }

```

1.13.1. Распаковка вложенных исключений

```

1 void print_exception_chain(const std::exception& e, int depth = 0)
   {
2     std::cout << std::string(depth * 2, ' ') << e.what() << "\n";
3     try {
4         std::rethrow_if_nested(e); // Перебрасывает вложенное,
           если есть
5     } catch (const std::exception& nested) {
6         print_exception_chain(nested, depth + 1);
7     } catch (...) {
8         std::cout << std::string((depth + 1) * 2, ' ') << "unknown
           \n";
9     }
10 }
11
12 int main() {
13     try {
14         high_level();
15     } catch (const std::exception& e) {
16         print_exception_chain(e);
17     }
18 }

```

Вывод:

```

high_level failed
  mid_level failed
    disk I/O error

```

Это аналог «exception chaining» в Java и Python.

1.14. Стоимость исключений: zero-cost exception model

Современные компиляторы (GCC, Clang) используют **table-based** (zero-cost) модель исключений:

	Нормальный путь (без исключений)	При броске исключения
Накладные расходы	Нулевые. Никаких дополнительных ин-струкций	Высокие: поиск по таб-лицам, раскрутка стека
Размер бинарника	Увеличивается (таблицы раскрутки хранятся в секции <code>.eh_frame</code>)	—

Принцип: «платите только когда бросаете». На нормальном пути исполнения нет вообще никаких затрат — ни проверок, ни сохранения контекста. Но когда исключение летит, стоимость **очень высокая** (в 10–100 раз дороже обычного `return`).

Бросок исключения приблизительно

():

1. Выделение памяти для объекта исключения ~100 нс
 2. Поиск обработчика по таблицам (DWARF unwind) ~1–10 мкс
 3. Раскрутка стека вызовов (деструкторов) ~1–10 мкс
- Итого
: ~2–20 мкс Обычный

return:

~1 нс

Вывод: исключения — для **исключительных** ситуаций, а не для управления потоком выполнения. Для ожидаемых ошибок (парсинг, валидация) используйте `std::expected` или коды возврата.

1.15. Итого: чек-лист по исключениям

Правило	Почему
Бросайте наследников <code>std::exception</code>	Единообразная обработка
Ловите по <code>const&</code>	Нет копирования, нет срезки
Пробрасывайте через <code>throw;</code> (без аргумента)	Сохранение типа, нет срезки
Помечайте деструкторы <code>noexcept</code>	Они и так <code>noexcept</code> с C++11, но не бросайте из них
Помечайте <code>move</code> -конструкторы <code>noexcept</code>	Иначе <code>vector</code> будет копировать, а не перемещать
Используйте RAII для ресурсов	Раскрутка стека вызывает только деструкторы
Не используйте исключения в горячих путях	Бросок в $\sim 1000\times$ дороже <code>return</code>
Обработчики потомков — перед родителями	<code>catch</code> проверяется сверху вниз

2. Контейнеры и итераторы

2.1. `std::vector`: реализация изнутри

Начнём с устройства `std::vector` и в первую очередь рассмотрим реализацию метода `push_back`.

Структура вектора состоит из трёх полей: указателя на данные `arr_`, размера `sz_` и ёмкости `cap_`.

2.1.1. Почему нельзя писать `new T[newcap]`

При реаллокации наивная реализация могла бы выглядеть так:

```
1 T* newarr = new T[newcap]; // проблема!
```

Это не работает по двум причинам:

- Мы не знаем, есть ли у `T` конструктор по умолчанию.
- Нам нужно выделить *сырую* память и заполнять её по мере надобности, а не создавать все объекты сразу.

2.1.2. Placement new

Решение — выделить сырую память через `char`, а объекты создавать по месту с помощью **placement new**:

```
1 new (ptr) T(args...); // создаёт объект T по адресу ptr
```

Почему нельзя просто написать `newarr[i] = arr_[i]`? Потому что `newarr` указывает на сырые байты — объекта `T` там ещё нет, и присваивать можно только существующему объекту. `memcpy` тоже не подходит: он не работает корректно, если объект содержит указатель или ссылку на своё собственное поле (например, `std::string` с small-string optimization или декартово дерево).

2.1.3. Почему нельзя `delete[] arr_`

При освобождении старой памяти нельзя писать `delete[] arr_`: это вызовет деструктор для каждого из `cap_` слотов, хотя реальных объектов там только `sz_`. Нужно:

1. вручную вызвать деструктор для каждого существующего объекта;
2. освободить сырую память через `delete[] reinterpret_cast<char*>(arr_)`.

2.1.4. Реализация `reserve`

```
1 template<typename T>
2 class vector {
3     T* arr_;
4     size_t sz_;
5     size_t cap_;
```

```

6
7     void reserve(size_t newcap) {
8         T* newarr = reinterpret_cast<T*>(new char[newcap * sizeof(
9             T)]);
10        size_t i = 0;
11        try {
12            for (; i < sz_; ++i)
13                new (newarr + i) T(arr_[i]); // placement new
14        } catch (...) {
15            for (size_t k = 0; k < i; ++k)
16                (newarr + k)->~T();
17            delete[] reinterpret_cast<char*>(newarr);
18            throw;
19        }
20        for (size_t k = 0; k < sz_; ++k)
21            (arr_ + k)->~T();
22        delete[] reinterpret_cast<char*>(arr_);
23        arr_ = newarr;
24        cap_ = newcap;
25    }

```

Обработка исключений в `reserve`:

- Если `new char[...]` бросает `std::bad_alloc` — мы просто не выделили память, ничего не испорчено, вылетаем вверх.
- Если `placement new` бросает исключение в конструкторе на шаге `i` — нужно уничтожить уже созданные объекты (с 0 по `i-1`), освободить `newarr` и перебросить исключение.

2.1.5. Реализация `push_back`

```

1     void push_back(const T& value) {
2         if (sz_ == cap_)
3             reserve(cap_ > 0 ? cap_ * 2 : 1);
4         new (arr_ + sz_) T(value);
5         ++sz_;
6     }
7 }; // конец class vector

```

2.2. `vector<bool>`: специализация с битовым хранением

`vector<bool>` — это печально известная специализация, которая ломает многие интуитивные ожидания от вектора.

Идея: хранить элементы побитово, упаковывая по 8 штук в `char`. Это экономит память, но делает невозможным взять ссылку `bool&` на отдельный элемент — ссылка не может указывать на отдельный бит.

2.2.1. Прокси-класс BitReference

Чтобы сохранить синтаксис `v[i] = true`, вводится прокси-класс:

```
1  template<>
2  class vector<bool> {
3      char* arr_;
4      size_t sz_;
5      size_t cap_;
6
7      struct BitReference {
8          char* cell;
9          uint8_t idx;
10
11         BitReference(char* cell, uint8_t idx) : cell(cell), idx(
12             idx) {}
13
14         void operator=(bool b) {
15             if (b) *cell |= (1 << idx);
16             else *cell &= ~(1 << idx);
17         }
18
19         operator bool() const {
20             return *cell & (1 << idx);
21         }
22     };
23
24     public:
25     BitReference operator[](size_t idx) {
26         return BitReference(arr_ + idx / 8, idx % 8);
27     };
28 }
```

2.2.2. Трюк Ильи Мещерина: узнать тип выражения

```
1  template<typename T>
2  class Debug {
3      Debug(T) = delete;
4  };
5
6  int main() {
7      std::vector<bool> v(10);
8      v[5] = true;
9      // Debug d(v[5]); -- компилятор скажет тип : std::
10     __bit_reference
11 }
```

Почему это проблема: `vector<bool>` нарушает концепцию контейнера — из него нельзя взять настоящую ссылку на элемент (`bool& r = v[5]` не скомпилируется), и многие шаблонные алгоритмы ломаются при работе с ним. Поэтому для хранения битовых флагов часто предпочитают `std::vector<char>`

или `std::bitset`.

3. Библиотека <algorithm>

3.1. Зачем она вообще нужна

Стандартная библиотека предоставляет **готовые алгоритмы**, которые работают с любыми контейнерами через **итераторы**. Ключевая идея: алгоритм не знает, что за контейнер перед ним — он получает пару итераторов `[begin, end)` и работает с ними.

```
1 #include <algorithm>
2 #include <vector>
3
4 int main() {
5     int arr[] = {5, 3, 1, 4, 2};
6     std::sort(arr, arr + 5);
7
8     std::vector<int> v = {5, 3, 1, 4, 2};
9     std::sort(v.begin(), v.end());
10 }
```

3.2. `std::sort`

Сортирует диапазон **на месте**. Сложность: $O(n \log n)$ (introsort).

```
1 std::vector<int> v = {5, 3, 1, 4, 2};
2 std::sort(v.begin(), v.end()); // повозрастанию
3
4 std::sort(v.begin(), v.end(), [](int a, int b) {
5     return a > b; // поубыванию
6 });
```

Важно: компаратор должен задавать *строгий слабый порядок* (`<`, а не `<=`). Если написать `a >= b` — UB.

3.3. `std::find` и `std::find_if`

Линейный поиск. Возвращает итератор на первый подходящий элемент, или `last` если ничего не нашли.

```
1 std::vector<int> v = {1, 2, 3, 4, 5};
2
3 auto it = std::find(v.begin(), v.end(), 3);
4 if (it != v.end()) std::cout << *it; // 3
5
6 auto it2 = std::find_if(v.begin(), v.end(), [](int x) {
7     return x % 2 == 0; // первый чётный -> 2
8 });
```

`end()` как «не найдено» — универсальное соглашение, потому что алгоритм не знает о типе контейнера и не может вернуть `-1`.

3.4. `std::count` и `std::count_if`

```
1 std::vector<int> v = {1, 2, 2, 3, 2, 4};
2 int cnt = std::count(v.begin(), v.end(), 2);
3           // 3
4 int even = std::count_if(v.begin(), v.end(), [](int x){ return x
5                        %2==0; }); // 4
```

3.5. `std::for_each` и `std::transform`

```
1 std::vector<int> v = {1, 2, 3, 4, 5};
2
3 std::for_each(v.begin(), v.end(), [](int& x) { x *= 2; });
4 // v == {2, 4, 6, 8, 10}
5
6 std::vector<int> out(v.size());
7 std::transform(v.begin(), v.end(), out.begin(), [](int x) {
8     return x * x;
9 });
10 // out == {4, 16, 36, 64, 100}
```

3.6. `std::copy` и `std::back_inserter`

```
1 std::vector<int> src = {1, 2, 3, 4, 5};
2 std::vector<int> evens;
3
4 std::copy_if(src.begin(), src.end(), std::back_inserter(evens),
5             [](int x) { return x % 2 == 0; });
6 // evens == {2, 4}
```

`std::back_inserter` — итератор-адаптер, который при присваивании вызывает `push_back`. Нужен, когда выходной контейнер изначально пуст.

3.7. Идиома `erase-remove`

`std::remove` не удаляет элементы — он лишь сдвигает нужные в начало и возвращает итератор на новый логический конец. Реальное удаление делает `erase`:

```
1 std::vector<int> v = {1, 2, 3, 2, 4, 2, 5};
2 v.erase(std::remove(v.begin(), v.end(), 2), v.end());
3 // v == {1, 3, 4, 5}
```

3.8. `std::accumulate` из `<numeric>`

```
1 #include <numeric>
2 std::vector<int> v = {1, 2, 3, 4, 5};
3 int sum      = std::accumulate(v.begin(), v.end(), 0);
   // 15
4 int product = std::accumulate(v.begin(), v.end(), 1,
5                               [](int acc, int x) { return acc * x; });
   // 120
```

4. Дек (`std::deque`)

4.1. Инвалидация указателей и ссылок в `vector`

```
1 std::vector<int> v(10);
2 int* p = &v[5];
3 int& x = v[5];
4 v.push_back(1);           // реаллокация!
5 std::cout << *p;          // UB -- pointer invalidation
6 std::cout << x;           // UB -- reference invalidation
```

При `push_back` вектор мог реаллоцироваться. Старый кусок памяти нам больше не принадлежит — все указатели и ссылки на элементы, полученные до реаллокации, недействительны.

4.2. Два главных отличия дека от вектора

1. `std::deque` **не инвалидирует** указатели и ссылки на существующие элементы при добавлении новых.
2. Дек умеет `push_front()` и `pop_front()` за $O(1)$.

4.3. Внутреннее устройство дека

Дек хранит массив указателей на бакеты фиксированного размера — `T** arr_`.

- При `push_back`: идём вправо по бакету; когда бакет заполнен — выделяем новый бакет справа.
- При `push_front`: идём влево; когда бакет заполнен — выделяем новый бакет слева и кладём элемент с его конца.
- Дек хранит две пары: (номер бакета начала, индекс внутри него) и (номер бакета конца, индекс внутри него).

Почему не инвалидируются указатели? При переполнении внешнего массива реаллоцируется только он — перепривязываются указатели на бакеты. Сами бакеты с элементами остаются на месте.

operator[] за $O(1)$: зная индекс начала и размер бакета, пересчитываем номер бакета и смещение за константное время.

У дека нет `reserve()` и `shrink_to_fit()` — элементы разбросаны по бакетам, и их нельзя реаллоцировать одним куском.

5. Адаптеры контейнеров

`std::stack`, `std::queue` и `std::priority_queue` — это **не контейнеры**, а **адаптеры**: они оборачивают контейнер и предоставляют ограниченный интерфейс поверх него.

```

1 std::stack<int> s; // по умолчанию на deque
2 std::stack<int, std::vector<int>> s2; // на vector
3 std::stack<int, std::list<int>> s3; // на list

```

5.1. std::stack

```

1 template<typename T, typename Container = std::deque<T>>
2 class stack {
3     Container c_;
4 public:
5     void push(const T& val) { c_.push_back(val); }
6     void pop() { c_.pop_back(); }
7     T& top() { return c_.back(); }
8     bool empty() const { return c_.empty(); }
9     size_t size() const { return c_.size(); }
10 };

```

Почему pop() возвращает void? Если бы pop() возвращал значение, пришлось бы скопировать элемент перед удалением. Если конструктор копирования бросит исключение — элемент уже удалён и значение потеряно. Разделение на top() (посмотреть) и pop() (удалить) даёт exception safety. Вернуть ссылку тоже нельзя: после pop() она стала бы битой.

6. Итераторы и их категории

6.1. Что такое итератор

Итератор — это тип, который позволяет **обходить последовательность**. Это обобщение указателя: его можно разыменовывать (*it), инкрементировать (++it) и сравнивать (it != end). Обычный сырой указатель — полноценный итератор. Range-based for — синтаксический сахар:

```
1 for (int x : s) { ... }
2 // разворачивается в :
3 for (auto it = s.begin(); it != s.end(); ++it) { ... }
```

Достаточно реализовать begin() и end().

6.2. Категории итераторов

Категория	Что умеет	Примеры
InputIterator	*it, ++it, ==, !=	поточные итераторы
ForwardIterator	+ гарантия повторного прохода	unordered_set, forward_list
BidirectionalIterator	+ --it	list, set, map
RandomAccessIterator	+ it+=n, it[n], <, >	deque
ContiguousIterator	+ гарантия непрерывности памяти	vector, array, указатель

Разница между Input и Forward: InputIterator не гарантирует, что при повторном проходе увидим те же данные (поток — прочитанное не вернётся). ForwardIterator эту гарантию даёт.

6.3. iterator_traits

Для получения информации об итераторе внутри шаблонной функции:

```
1 template<typename It>
2 void foo(It begin, It end) {
3     typename std::iterator_traits<It>::value_type x = *begin;
4     using Cat = typename std::iterator_traits<It>::
        iterator_category;
5 }
```

Доступные поля: value_type, reference, pointer, difference_type, iterator_category. Почему не auto x = *begin? Для vector<bool> auto выведет BitReference, а не bool. value_type вернёт правильный тип.

6.4. Реализация `std::distance`

```
1 template<typename It>
2 typename std::iterator_traits<It>::difference_type
3 distance(It first, It last) {
4     if constexpr (std::is_same_v<
5         typename std::iterator_traits<It>::iterator_category,
6         std::random_access_iterator_tag>)
7     {
8         return last - first; // O(1)
9     }
10    typename std::iterator_traits<It>::difference_type i = 0;
11    for (; first != last; ++first, ++i) {}
12    return i; // O(n)
13 }
```

`if constexpr` вычисляется на этапе компиляции: невыбранная ветка не компилируется вообще. Это позволяет писать `last - first` не боясь, что для `non-random-access` итератора это не скомпилируется.

6.5. Реализация итератора вектора

```
1 template<typename T>
2 class Vector {
3 public:
4     class Iterator {
5     public:
6         using iterator_category = std::random_access_iterator_tag;
7         using value_type = T;
8         using difference_type = std::ptrdiff_t;
9         using pointer = T*;
10        using reference = T&;
11
12        explicit Iterator(pointer ptr) : ptr_(ptr) {}
13
14        reference operator*() const { return *ptr_; }
15        pointer operator->() const { return ptr_; }
16
17        Iterator& operator++() { ++ptr_; return *this; }
18        Iterator operator++(int) { Iterator tmp = *this; ++ptr_;
19            return tmp; }
20        Iterator& operator--() { --ptr_; return *this; }
21        Iterator operator--(int) { Iterator tmp = *this; --ptr_;
22            return tmp; }
23
24        Iterator operator+(difference_type n) const { return
25            Iterator(ptr_ + n); }
26        Iterator operator-(difference_type n) const { return
27            Iterator(ptr_ - n); }
28        difference_type operator-(const Iterator& o) const {
29            return ptr_ - o.ptr_; }
30    };
31 }
```

```

25     Iterator& operator+=(difference_type n) { ptr_ += n;
        return *this; }
26     Iterator& operator-=(difference_type n) { ptr_ -= n;
        return *this; }
27     reference operator[](difference_type n) const { return
        ptr_[n]; }
28
29     bool operator==(const Iterator& o) const { return ptr_ ==
        o.ptr_; }
30     bool operator!=(const Iterator& o) const { return ptr_ !=
        o.ptr_; }
31     bool operator< (const Iterator& o) const { return ptr_ <
        o.ptr_; }
32     bool operator> (const Iterator& o) const { return ptr_ >
        o.ptr_; }
33     bool operator<=(const Iterator& o) const { return ptr_ <=
        o.ptr_; }
34     bool operator>=(const Iterator& o) const { return ptr_ >=
        o.ptr_; }
35
36     private:
37         pointer ptr_;
38     };
39
40     Vector() : data_(nullptr), size_(0), capacity_(0) {}
41     ~Vector() { delete[] data_; }
42
43     void push_back(const T& val) {
44         if (size_ == capacity_) reserve(capacity_ == 0 ? 1 :
            capacity_ * 2);
45         data_[size_++] = val;
46     }
47
48     T& operator[](std::size_t i) { return data_[i]; }
49     std::size_t size() const { return size_; }
50     std::size_t capacity() const { return capacity_; }
51
52     Iterator begin() { return Iterator(data_); }
53     Iterator end() { return Iterator(data_ + size_); }
54
55     private:
56     void reserve(std::size_t new_cap) {
57         T* tmp = new T[new_cap];
58         for (std::size_t i = 0; i < size_; ++i) tmp[i] = data_[i];
59         delete[] data_;
60         data_ = tmp; capacity_ = new_cap;
61     }
62
63     T* data_; std::size_t size_; std::size_t capacity_;
64 };

```

Пять type alias-ов в Iterator — контракт с iterator_traits. Без них

`std::sort` и `std::distance` не смогут узнать категорию итератора.

7. Потокковые итераторы

7.1. `std::istream_iterator`

```
1 #include <iterator>
2 #include <vector>
3
4 int main() {
5     std::vector<int> v(
6         std::istream_iterator<int>(std::cin),
7         std::istream_iterator<int>() // итераторконец- (EOF)
8     );
9 }
```

Это **InputIterator** — одноразовый: прочитанное назад не вернуть. `istream_iterator<int>` без аргументов — итератор конца потока.

```
1 std::istream_iterator<int> in(std::cin), eof;
2 int sum = std::accumulate(in, eof, 0); // суммавсехчиселиз
   stdin
```

7.2. `std::ostream_iterator`

```
1 std::vector<int> v = {1, 2, 3, 4, 5};
2
3 std::copy(v.begin(), v.end(),
4           std::ostream_iterator<int>(std::cout, " "));
5 // вывод: 1 2 3 4 5
6
7 std::transform(v.begin(), v.end(),
8                std::ostream_iterator<int>(std::cout, "\n"),
9                [](int x) { return x * x; });
```

`ostream_iterator` — **OutputIterator**: присваивание `*it = val` вызывает `os << val << delim`.

7.3. Итераторы файловых потоков

```
1 #include <fstream>
2 #include <iterator>
3 #include <algorithm>
4
5 int main() {
6     std::ifstream fin("input.txt");
7     std::ofstream fout("output.txt");
8 }
```

```
9     std::vector<int> v(std::istream_iterator<int>(fin),
10                      std::istream_iterator<int>());
11     std::sort(v.begin(), v.end());
12     std::copy(v.begin(), v.end(),
13              std::ostream_iterator<int>(fout, "\n"));
14 }
```

Алгоритмы не знают, откуда данные читаются и куда пишутся — они работают с абстракцией итератора. В этом и есть смысл всей иерархии.

8. Аллокатеры и управление памятью

8.1. Зачем аллокатер нужен контейнеру

У каждого контейнера есть шаблонный параметр аллокатера:

```
1 std::list<int>      // = std::list<int, std::allocator<int>>
2 std::vector<int>    // = std::vector<int, std::allocator<int>>
```

Аллокатер — это промежуточный класс между контейнером и оператором `new`. Стек вызовов:

Container → Allocator → operator new → malloc → OS

8.2. Устройство стандартного аллокатера

У аллокатера четыре метода:

```
1 template<typename T>
2 struct allocator {
3     using value_type = T;
4
5     T* allocate(std::size_t n) {
6         if (n == 0) return nullptr;
7         return static_cast<T*>(::operator new(n * sizeof(T)));
8     }
9
10    void deallocate(T* p, std::size_t) {
11        ::operator delete(p);
12    }
13
14    template<typename... Args>
15    void construct(T* p, Args&&... args) {
16        ::new (static_cast<void*>(p)) T(std::forward<Args>(args)
17        ...);
18    }
19
20    void destroy(T* p) {
21        p->~T();
22    }
23 };
```

- `allocate` — выделяет сырую память, конструкторы не вызываются.
- `deallocate` — освобождает память, деструкторы не вызываются.
- `construct` — вызывает конструктор по адресу (placement new).
- `destroy` — явно вызывает деструктор, память не освобождает.

8.3. Проблема: `list` аллоцирует не `T`

`std::list<int>` хранит не просто `int`, а узлы:

```
1 template<typename T>
2 struct ListNode { T value; ListNode* prev; ListNode* next; };
```

Но аллокатор шаблонизирован по `T = int`. Решение комитета — **rebind**: вложенный шаблон в аллокаторе, который перепривязывает его к другому типу:

```
1 template<typename T>
2 struct allocator {
3     template<typename U>
4     struct rebind { using other = allocator<U>; };
5 };
6
7 // list внутриделает :
8 using NodeAlloc = typename Alloc::template rebind<ListNode<T>>::
9     other;
10 NodeAlloc node_alloc_; // аллокатордляузлов , анедля T
```

8.4. `allocator_traits`

`std::allocator_traits<Alloc>` — адаптер с единым интерфейсом к любому аллокатору и разумными дефолтами для отсутствующих методов.

```
1 template<typename Alloc>
2 struct allocator_traits {
3     using value_type      = typename Alloc::value_type;
4     using pointer         = value_type*;
5     using size_type       = std::size_t;
6     using difference_type = std::ptrdiff_t;
7
8     template<typename U>
9     using rebind_alloc = typename Alloc::template rebind<U>::other
10         ;
11
12     static pointer allocate(Alloc& a, size_type n)
13         { return a.allocate(n); }
14     static void deallocate(Alloc& a, pointer p, size_type n)
15         { a.deallocate(p, n); }
16
17     // если Alloc нет construct -- placement new по умолчанию
18     template<typename T, typename... Args>
19     static void construct(Alloc& a, T* p, Args&&... args)
20         { ::new (static_cast<void*>(p)) T(std::forward<Args>(args)
21             ...); }
22
23     // если Alloc нет destroy -- явный деструктор по умолчанию
24     template<typename T>
25     static void destroy(Alloc& a, T* p) { p->~T(); }
```

```

25 // политики распространения аллокатора
26 using propagate_on_container_copy_assignment = std::false_type
27 ;
27 using propagate_on_container_move_assignment = std::false_type
28 ;
28 using propagate_on_container_swap           = std::false_type
29 ;

```

Главная идея: контейнер должен работать с аллокатором *только через* `allocator_traits`. Тогда кастомный аллокатор может не реализовывать `construct/destroy` и всё равно работать.

8.4.1. Политики распространения аллокатора

Флаг	Что означает
<code>propagate_on_container_copy_assignment</code>	Копировать аллокатор при <code>operator=</code> копирования?
<code>propagate_on_container_move_assignment</code>	Копировать аллокатор при <code>operator=</code> перемещения?
<code>propagate_on_container_swap</code>	Менять аллокаторы местами при <code>swap</code> ?

По умолчанию все три — `false_type`. Это важно для `stateful` аллокаторов: если два вектора используют разные арены, при `swap` нужно либо перетащить элементы, либо поменять аллокаторы.

8.5. Allocator-aware контейнер

```

1  template<typename T, typename Alloc = std::allocator<T>>
2  class vector {
3      using AllocTraits = std::allocator_traits<Alloc>;
4
5      Alloc alloc_;
6      T*    arr_;
7      size_t sz_;
8      size_t cap_;
9
10 public:
11     explicit vector(const Alloc& a = Alloc())
12         : alloc_(a), arr_(nullptr), sz_(0), cap_(0) {}
13
14     void push_back(const T& val) {
15         if (sz_ == cap_) reserve(cap_ ? cap_ * 2 : 1);
16         AllocTraits::construct(alloc_, arr_ + sz_, val);
17         ++sz_;
18     }

```

```

19
20     ~vector() {
21         for (size_t i = 0; i < sz_; ++i)
22             AllocTraits::destroy(alloc_, arr_ + i);
23         AllocTraits::deallocate(alloc_, arr_, cap_);
24     }
25
26 private:
27     void reserve(size_t newcap) {
28         T* newarr = AllocTraits::allocate(alloc_, newcap);
29         size_t i = 0;
30         try {
31             for (; i < sz_; ++i)
32                 AllocTraits::construct(alloc_, newarr + i, arr_[i]);
33         } catch (...) {
34             for (size_t k = 0; k < i; ++k)
35                 AllocTraits::destroy(alloc_, newarr + k);
36             AllocTraits::deallocate(alloc_, newarr, newcap);
37             throw;
38         }
39         for (size_t k = 0; k < sz_; ++k)
40             AllocTraits::destroy(alloc_, arr_ + k);
41         AllocTraits::deallocate(alloc_, arr_, cap_);
42         arr_ = newarr; cap_ = newcap;
43     }
44 };

```

Stateless аллокатор (как `std::allocator`) не занимает места благодаря **EBO** (Empty Base Optimization).

8.6. Перегрузка `operator new` и `operator delete`

8.6.1. Глобальная перегрузка

```

1 void* operator new(std::size_t size) {
2     void* ptr = std::malloc(size);
3     if (!ptr) throw std::bad_alloc{};
4     return ptr;
5 }
6 void operator delete(void* ptr) noexcept { std::free(ptr); }
7
8 // C++14: sized delete
9 void operator delete(void* ptr, std::size_t size) noexcept { std::
    free(ptr); }

```

Затрагивает весь процесс. Используется для профилировщиков памяти и детекторов утечек.

8.6.2. Классовая перегрузка

```

1 struct Node {
2     int value; Node* next;
3
4     void* operator new(std::size_t size) {
5         return ::operator new(size); // делегируем глобальному
6     }
7     void operator delete(void* ptr) noexcept {
8         ::operator delete(ptr);
9     }
10 };

```

При `new Node` компилятор сначала ищет `operator new` в классе, и только если не нашёл — берёт глобальный.

8.6.3. Когда использовать что

Инструмент	Когда применять
Кастомный аллокатор	Своя стратегия памяти для конкретного контейнера
Классовая перегрузка <code>new/delete</code>	Свой пул для конкретного класса (например, узлы дерева)
Глобальная перегрузка <code>new/delete</code>	Профилирование, детектирование утечек для всей программы

9. Многопоточность в C++

Лирическое отступление. Данный раздел **не относится** к текущему курсу и приводится исключительно в ознакомительных целях. Многопоточность будет подробно рассмотрена в последующих курсах.

9.1. Определение потока исполнения

9.1.1. Что такое поток?

Поток исполнения (thread of execution) — это единственная последовательная цепочка команд, выполняемых процессором. Представьте это как «линию чтения» в книге: один палец ведёт по одной строке. Если у вас два пальца — вы читаете два куска книги одновременно (два потока).

Программа считается завершённой **только тогда**, когда завершились **все** её потоки. Даже если `main()` уже отработал, отсоединённые потоки могут ещё работать.

9.1.2. Аналогия

Представьте кухню ресторана:

- **Однопоточная программа** — один повар делает всё по очереди: режет овощи, варит суп, жарит мясо. Пока суп варится, повар просто ждёт.
- **Многопоточная программа** — несколько поваров работают параллельно. Один варит суп, другой жарит мясо, третий режет салат.

9.1.3. Логическая многопоточность vs Аппаратный параллелизм

Это не одно и то же:

Понятие	Суть	Пример
Логическая многопоточность	ОС быстро переключается между потоками на одном ядре, создавая иллюзию параллельности	1 ядро, 4 потока — ОС по очереди даёт каждому ~10 мс
Аппаратный параллелизм	Потоки реально выполняются одновременно на разных ядрах	4 ядра × 1 поток — каждый работает физически одновременно

Многопоточность появилась **раньше** многоядерных процессоров. Зачем? Чтобы решить проблему **блокирующего ввода-вывода**. Классический пример: в ранних ОС форматирование дискеты «вешало» всю систему, потому что процессор просто ждал завершения I/O-операции. С появлением потоков ОС могла переключиться на другую задачу, пока диск форматируется.

9.1.4. Типы многозадачности

Кооперативная многозадачность — поток *сам* решает, когда отдать управление другим. Если поток «зажадничал» и не отдаёт CPU — все остальные замирают.

```
Поток
A: работаю... работаю... ладно, передаю управление -> yield()
Поток
B: о, моя очередь ! работаю... yield()
Поток
A: снова !
```

Пример в современном C++ — **корутины** (`co_await`, `co_yield` в C++20).

Вытесняющая многозадачность — ОС *сама* прерывает потоки по таймеру (обычно каждые ~10 мс). Поток не может «зависнуть» и заблокировать всю систему.

```
Поток
A: работаю... работ--ОС
[]: Время вышло ! Переключаю на B. Поток
B: работаю...
```

Это стандарт для всех современных ОС (Windows, Linux, macOS).

9.2. Работа с потоками в C++

9.2.1. Создание потока

Потоки создаются через `std::thread` (доступен с C++11). Важная деталь: **поток запускается немедленно** в момент создания объекта — нет отдельного метода `start()`.

```
1 #include <iostream>
2 #include <thread>
3
4 void say_hello() {
5     std::cout << "Привет из потока !\n";
6 }
7
8 int main() {
9     std::thread t(say_hello); // Поток УЖЕ работает !
10    t.join();                 // Ждём завершения
11    return 0;
12 }
```

9.2.2. Передача аргументов в поток

```
1 void print_sum(int a, int b) {
2     std::cout << a + b << "\n";
3 }
4
```

```

5 int main() {
6     std::thread t(print_sum, 3, 7); //
        Аргументы передаются после функции
7     t.join();
8 }

```

Внимание: аргументы копируются в поток. Если нужна ссылка, используйте `std::ref()`:

```

1 void increment(int& x) { x++; }
2
3 int main() {
4     int value = 0;
5     std::thread t(increment, std::ref(value)); // Передаём ссылку
6     t.join();
7     std::cout << value; // 1
8 }

```

9.2.3. Использование лямбд

```

1 int main() {
2     int data = 42;
3     std::thread t([&data]() {
4         data *= 2;
5         std::cout << "В потоке: " << data << "\n";
6     });
7     t.join();
8     std::cout << "В main: " << data << "\n"; // 84
9 }

```

9.2.4. `join()` vs `detach()`

Метод	Что делает	Когда использовать
<code>join()</code>	Блокирует текущий поток до завершения вызванного	Когда нужен результат работы потока
<code>detach()</code>	Отсоединяет поток, он живёт самостоятельно	Фоновые задачи (логирование, мониторинг)

```

1 // Пример join() -- ждём результат
2 std::thread t(heavy_computation);
3 // ... тут main делает своё дело ...
4 t.join(); // Теперь точно результат готов
5
6 // Пример detach() -- fire and forget
7 std::thread logger(write_logs);
8 logger.detach(); // Пусть пишет лог в фоне

```

9.2.5. Опасность: забыть join/detach

Если объект `std::thread` уничтожается без вызова `join()` или `detach()`, программа **аварийно завершается** через `std::terminate()`:

```
1 void dangerous() {
2     std::thread t(some_work);
3     // Упс! Функция завершилась, t уничтожается
4     // без join/detach -> std::terminate()!
5 }
```

Безопасный паттерн — RAII-обёртка (или `std::jthread` из C++20, который сам делает `join` в деструкторе):

```
1 // C++20
2 void safe() {
3     std::jthread t(some_work);
4     // Деструктор jthread сам вызовет join()
5 }
```

9.3. Область памяти и состояние гонки (Data Race)

9.3.1. Что такое Memory Location?

Memory Location (область памяти) по стандарту — это:

- Объект **скалярного типа** (`int`, `char`, `float`, указатель и т. д.)
- Или **максимальная последовательность битовых полей ненулевой длины**

Важный нюанс: два соседних поля структуры — это **разные** области памяти, к ним можно обращаться параллельно без гонки. Но соседние битовые поля — это **одна** область памяти!

```
1 struct S {
2     int x;           // Отдельная memory location
3     int y;           // Отдельная memory location --
                       // можно менять параллельно с x
4 };
5
6 struct Bits {
7     unsigned a : 4;  // a и b -- ОДНА memory location!
8     unsigned b : 4;  // Параллельное изменение a и b = Data Race
9     unsigned   : 0;  // Разделитель
10    unsigned c : 4;  // c -- отдельная memory location
11 };
```

9.3.2. Что такое Data Race?

Data Race (гонка данных) — это ситуация, когда:

1. Два потока обращаются к **одной области памяти**

2. Хотя бы один из них записывает
3. Между ними нет синхронизации (нет happens-before)
4. Хотя бы одно действие не атомарное

Data Race = Undefined Behavior (UB). Это не просто «неправильный результат». Компилятор имеет право на *что угодно*: удалить код, выдать мусор, отформатировать диск (теоретически).

9.3.3. Наглядный пример гонки

```

1  #include <thread>
2
3  int counter = 0;  // Общая переменная
4
5  void increment_1000() {
6      for (int i = 0; i < 1000; ++i) {
7          counter++;  // DATA RACE!
8      }
9  }
10
11 int main() {
12     std::thread t1(increment_1000);
13     std::thread t2(increment_1000);
14     t1.join();
15     t2.join();
16
17     // Ожидаем 2000, но результат непредсказуем !
18     // Может быть 1500, 1837, 2000, или что угодно
19     std::cout << counter << "\n";
20 }
```

Почему не 2000? Операция counter++ не атомарна. На уровне процессора это три шага:

- | | | |
|--|------------|---------|
| 1. Прочитать counter из памяти | -> регистр | (LOAD) |
| 2. Увеличить значение в регистре на 1 | | (ADD) |
| 3. Записать результат обратно в память | | (STORE) |

Если два потока выполняют это одновременно:

Поток		
1: LOAD counter (=0)	-> регистр = 0	Поток
2: LOAD counter (=0)	-> регистр = 0	Поток
1: ADD	-> регистр = 1	Поток
2: ADD	-> регистр = 1	Поток
1: STORE	-> counter = 1	Поток
2: STORE	-> counter = 1	<- Инкремент потерян !

9.4. Распространённые ошибки и заблуждения

9.4.1. Ошибка №1: `volatile` «чинит» гонки

Нет! `volatile` — это указание компилятору:

- Не оптимизировать чтение/запись этой переменной
- Каждый раз читать значение из памяти, а не из регистра

Но `volatile` **НЕ обеспечивает**:

- Атомарность операций
- Барьеры памяти (memory barriers)
- Когерентность кэшей между ядрами

```
1 volatile int counter = 0; // НЕ решает проблему !
2
3 // counter++ попрежнему = LOAD + ADD + STORE
4 // volatile лишь гарантирует, что компилятор не закэширует
5 // значение в регистре, но гонка всё равно есть
6
7 // Правильное решение:
8 std::atomic<int> counter = 0; // Атомарная переменная
```

`volatile` полезен для:

- Регистров аппаратуры (MMIO)
- Переменных, изменяемых обработчиками сигналов
- Переменных, изменяемых через `setjmp/longjmp`

9.4.2. Ошибка №2: Привязка к одному ядру решает гонки

Можно использовать системные вызовы для привязки потоков к одному ядру:

```
1 // Linux-специфичный код
2 cpu_set_t cpuset;
3 CPU_ZERO(&cpuset);
4 CPU_SET(0, &cpuset); // Привязать к ядру 0
5 sched_setaffinity(0, sizeof(cpuset), &cpuset);
```

Визуально симптомы гонки **могут исчезнуть** (потоки не работают *физически* параллельно). Но по стандарту C++ это **всё равно UB**, потому что:

- Язык C++ «не знает» про ядра процессора
- Стандарт определяет поведение абстрактной машины, а не конкретного железа
- Компилятор может оптимизировать код, опираясь на правила стандарта

9.4.3. Ошибка №3: `std::cout` безопасен для многопоточности

Стандарт **гарантирует**, что в `std::cout` не будет Data Race (внутренний буфер защищён). Но **порядок вывода** может перемешиваться:

```
1 // Поток 1:
2 std::cout << "Hello " << "from " << "thread 1\n";
3
4 // Поток 2:
5 std::cout << "Hello " << "from " << "thread 2\n";
6
7 // Возможный вывод :
8 // Hello Hello from from thread 1thread 2
9 //
10 // Каждаяотдельнаяоперация << атомарна,
11 // номеждунимидругойпотокможетвклиниться
```

Решение — вывод одной строкой:

```
1 // Вариант 1: формируемстрокузаранее
2 std::string msg = "Hello from thread 1\n";
3 std::cout << msg; // Одинвызов << = одиннепрерывныйвывод
4
5 // Вариант 2: std::osyncstream (C++20)
6 std::osyncstream(std::cout) << "Hello " << "from " << "thread 1\n"
7 ;
8 // Весьвыводбуферизуетсяиотправляетсяатомарно
```

9.5. Синхронизация: Мьютексы

9.5.1. Что такое мьютекс?

Мьютекс (mutex = mutual exclusion) — это объект синхронизации, который гарантирует, что только **один поток** в каждый момент времени выполняет защищённый участок кода (критическую секцию).

Аналогия: туалетная кабинка с замком. Зашёл — закрыл (lock). Вышел — открыл (unlock). Если занято — жди.

9.5.2. Базовый интерфейс

```
1 #include <mutex>
2
3 std::mutex mtx;
4 int counter = 0;
5
6 void safe_increment() {
7     for (int i = 0; i < 1000; ++i) {
8         mtx.lock();           // Захватитьмьютексили (ждать)
9         counter++;           // Критическаясекция -- толькоодинпоток
10        mtx.unlock();        // Освободитьмьютекс
11    }
```

```

12 }
13
14 int main() {
15     std::thread t1(safe_increment);
16     std::thread t2(safe_increment);
17     t1.join();
18     t2.join();
19     std::cout << counter << "\n"; // Всегда 2000!
20 }

```

9.5.3. Методы мьютекса

Метод	Поведение
lock()	Захватить. Если занят — заблокировать текущий поток до освобождения
unlock()	Освободить мьютекс
try_lock()	Попробовать захватить. Если занят — не ждать , вернуть false

```

1 if (mtx.try_lock()) {
2     // Захватили! Делаем работу ...
3     mtx.unlock();
4 } else {
5     // Мьютекс занят , делаем что-то - другое
6 }

```

9.5.4. Правила и UB при работе с мьютексами

UB №1: Повторный захват в том же потоке (рекурсивный lock)

```

1 std::mutex mtx;
2
3 void foo() {
4     mtx.lock();
5     // ... делаем что-то - ...
6     mtx.lock(); // UB! Этот поток уже владеет мьютексом !
7                 // Deadlock или неопределённое поведение
8     mtx.unlock();
9     mtx.unlock();
10 }

```

Решение — `std::recursive_mutex` (если рекурсия действительно нужна):

```

1 std::recursive_mutex rmtx;
2
3 void foo() {
4     rmtx.lock();
5     rmtx.lock(); // Рекурсивный мьютекс допускает это
6     rmtx.unlock();

```

```

7     mtx.unlock(); // Нужно столько же unlock, сколько lock
8 }

```

UB №2: Разблокировка чужого мьютекса

```

1 std::mutex mtx;
2
3 // Поток 1:
4 mtx.lock();
5 // работает...
6
7 // Поток 2:
8 mtx.unlock(); // UB! Поток 2 не захватывал этот мьютекс !

```

Почему нет метода `is_locked()`?

Потому что он бесполезен: к моменту, когда вы получите ответ, состояние уже могло измениться:

```

1 // Бессмысленный гипотетический код :
2 if (!mtx.is_locked()) { // В этот момент мьютекс свободен
3     mtx.lock(); //
4     // Здесь другой поток мог его захватить !
5 }
6 // Именно поэтому существует try_lock() --
7 // атомарная проверка захвата +

```

9.6. RAII в многопоточности

9.6.1. Проблема голых lock/unlock

```

1 void dangerous_function() {
2     mtx.lock();
3
4     do_work(); // Что если тут исключение ?
5
6     if (error) {
7         return; // Забыли unlock!
8         // Мьютекс навсегда заблокирован !
9     }
10    mtx.unlock(); // Досюда можем не дойти
11 }

```

9.6.2. `std::lock_guard` — простая RAII-обёртка

Принцип RAII: конструктор захватывает ресурс, деструктор освобождает. Гарантирует освобождение при **любом** выходе из области видимости (return, исключение, break...).

```

1 void safe_function() {

```

```

2      std::lock_guard<std::mutex> lock(mtx); // lock()
        вконструкторе
3
4      do_work();           // Исключение? Нестрашно !
5
6      if (error) {
7          return;          // Деструктор lock_guard вызовет unlock()
8      }
9
10     // Деструктор lock_guard вызовет unlock()
11 }

```

С C++17 можно не указывать тип (STAD):

```

1 std::lock_guard lock(mtx); // Компиляторвыведеттипсам

```

9.6.3. `std::unique_lock` — гибкая обёртка

В отличие от `lock_guard`, позволяет:

- Откладывать захват (`std::defer_lock`)
- Досрочно освободить (`unlock()`)
- Перемещать между областями видимости

```

1 void flexible_function() {
2     std::unique_lock<std::mutex> lock(mtx, std::defer_lock);
3     // МьютексещёНЕзахвачен
4
5     prepare_data();
6
7     lock.lock();           // Захватываем, когданужно
8     modify_shared_data();
9     lock.unlock();        // Освобождаемдосрочно
10
11     do_unrelated_work();  // Работаембезмьютекса
12 }

```

9.6.4. `std::scoped_lock` (C++17) — захват нескольких мьютексов

Главное преимущество: захватывает **несколько** мьютексов **без риска дедло-ка**:

```

1 std::mutex mtx1, mtx2;
2
3 void transfer(Account& from, Account& to, int amount) {
4     std::scoped_lock lock(mtx1, mtx2); //
        Оба мьютекса захвачены безопасно
5     from.balance -= amount;
6     to.balance += amount;

```

```

7 // Деструктор освободитоба мьютекса
8 }

```

9.6.5. Сравнение RAII-обёрток

Свойство	lock_guard	unique_lock	scoped_lock
Несколько мьютексов	нет	нет	да
Отложенный захват	нет	да	нет
Досрочное освобождение	нет	да	нет
Перемещение	нет	да	нет
Стандарт	C++11	C++11	C++17

9.7. Проектирование потокобезопасных объектов

9.7.1. Уровни потокобезопасности

Уровень «Нейтральный» — объект сам по себе не защищён, но безопасен при внешней синхронизации.

```

1 int x = 0; // Нейтральный тип
2
3 // Безопасно, если обернуть в мьютекс :
4 std::mutex mtx;
5 {
6     std::lock_guard lock(mtx);
7     x++;
8 }

```

Уровень «Ниже нейтрального» — указатели. Даже если синхронизировать сам указатель, данные по нему могут быть не защищены:

```

1 std::mutex mtx;
2 int* ptr = new int(42);
3
4 // Поток 1:
5 {
6     std::lock_guard lock(mtx);
7     ptr = new int(100); // Защитили ptr
8 }
9
10 // Поток 2:
11 int local = *ptr; // А данные по указателю --
    незащищены !
12 // ptr
    может указывать на удалённую память

```

Уровень «Потокобезопасный» — все операции внутренне защищены. Пример: `std::atomic<int>`.

```

1 std::atomic<int> counter = 0;
2

```

```

3 // Поток 1:
4 counter++; // Атомарноибезопасно
5
6 // Поток 2:
7 counter++; // Атомарноибезопасно

```

9.7.2. API Race — коварная ловушка

Это ситуация, когда **каждый отдельный метод** защищён мьютексом (Data Race нет!), но **последовательность вызовов** создаёт логическую ошибку.

Классический пример: потокобезопасный стек

```

1 // Допустим, у нас есть потокобезопасный "" стек:
2 class ThreadSafeStack {
3     std::mutex mtx;
4     std::stack<int> data;
5 public:
6     bool empty() {
7         std::lock_guard lock(mtx);
8         return data.empty();
9     }
10    int top() {
11        std::lock_guard lock(mtx);
12        return data.top();
13    }
14    void pop() {
15        std::lock_guard lock(mtx);
16        data.pop();
17    }
18 };

```

Каждый метод по отдельности безопасен. Но посмотрите:

```

1 ThreadSafeStack stack;
2
3 // Поток 1:
4 if (!stack.empty()) { // 1. Проверили -- непуст
5                       // Между empty() и top() поток 2
5                       // вызвал pop()!
6         int val = stack.top(); // 2. Стеку непуст -> UB!
7         stack.pop();
8     }
9
10 // Поток 2 выполняется( между шагами 1 и 2 потока 1):
11 if (!stack.empty()) {
12     stack.pop(); // Забрал последний элемент
13 }

```

Решение: объединение операций

```

1 class SafeStack {
2     std::mutex mtx;
3     std::stack<int> data;

```

```

4 public:
5     // Одна атомарная операция вместо трёх
6     std::optional<int> try_pop() {
7         std::lock_guard lock(mtx);
8         if (data.empty()) {
9             return std::nullopt; // Пусто -- верну ничего ""
10        }
11        int val = data.top();
12        data.pop();
13        return val; // Проверка + чтение + удаление
14        // = одна операция
15    }
16    void push(int val) {
17        std::lock_guard lock(mtx);
18        data.push(val);
19    }
20 };
21
22 // Использование:
23 if (auto val = stack.try_pop()) {
24     std::cout << "Получили: " << *val << "\n";
25 } else {
26     std::cout << "Стек пуст\n";
27 }

```

9.8. Дедлоки (Deadlocks)

9.8.1. Что такое дедлок?

Дедлок (взаимная блокировка) — ситуация, когда два (или более) потока бесконечно ждут друг друга, потому что каждый держит ресурс, нужный другому.

9.8.2. Классический пример: перевод денег

```

1 std::mutex mtx_A, mtx_B; // Мьютексы для счетов А и В
2
3 // Поток 1: перевод А -> В
4 void transfer_A_to_B() {
5     mtx_A.lock(); // 1. Захватил А
6     // ... тут переключение контекста ...
7     mtx_B.lock(); // 3. Ждёт В... но В занят потоком 2
8     // -> ДЕДЛОК
9     // перевод...
10    mtx_B.unlock();
11    mtx_A.unlock();
12 }
13
14 // Поток 2: перевод В -> А
15 void transfer_B_to_A() {

```

```

15     mtx_B.lock();           // 2. Захватил B
16     // ... тут переключение контекста ...
17     mtx_A.lock();           // 4. Ждёт A... но A занят потоком 1
    -> ДЕДЛОК
18     // перевод...
19     mtx_A.unlock();
20     mtx_B.unlock();
21 }

```

Визуально:

Поток	
1: держит [A], хочет [B] -----+	
	+--- Обаждутьвечно !Поток
2: держит [B], хочет [A] -----+	

9.8.3. Задача об обедающих философах

5 философов сидят за круглым столом. Между каждыми двумя лежит одна вилка. Чтобы есть, нужны две вилки. Если каждый одновременно возьмёт левую вилку — все будут ждать правую вечно = дедлок.

9.8.4. Решения в C++

Решение 1: `std::lock()` — алгоритм безопасного захвата

```

1  std::mutex mtx_A, mtx_B;
2
3  void safe_transfer() {
4      std::lock(mtx_A, mtx_B); // Захватываетобабездедлока !
5      // std::lock используетвнутреннийалгоритм :
6      // try_lock наодном -> try_lock надругом ->
        еслинеполучилось ,
7      // отпуститьвсёипопробоватьзаново
8
9      // Оборачиваемв lock_guard с adopt_lock мьютексы(
        ужезахвачены )
10     std::lock_guard lockA(mtx_A, std::adopt_lock);
11     std::lock_guard lockB(mtx_B, std::adopt_lock);
12
13     // Делаемперевод ...
14     // Деструкторы lock_guard освободятмьютексы
15 }

```

Решение 2: `std::scoped_lock` (C++17) — всё в одну строку

```

1  void safe_transfer_modern() {
2      std::scoped_lock lock(mtx_A, mtx_B); // Всё! Безопаснои RAII
3      // Делаемперевод ...
4  }

```

9.8.5. 4 необходимых условия дедлока (условия Коффмана)

Дедлок возможен **только** если выполнены **все четыре** условия одновременно:

1. **Взаимное исключение** — ресурс может быть захвачен только одним потоком
2. **Удержание и ожидание** — поток держит один ресурс и ждёт другой
3. **Невытесняемость** — ресурс нельзя отобрать у потока силой
4. **Циклическое ожидание** — замкнутая цепочка потоков, каждый ждёт следующего

Разрушение **любого** из условий предотвращает дедлок. `std::lock` разрушает условие №2 (захватывает всё или ничего).

9.9. Итого: шпаргалка

Проблема	Решение
Забыл <code>join/detach</code>	<code>std::jthread</code> (C++20)
Data Race	<code>std::mutex</code> + RAII-обёртки или <code>std::atomic</code>
Забыл <code>unlock</code>	<code>std::lock_guard</code> / <code>std::scoped_lock</code>
API Race	Объединение операций (<code>try_pop</code> и т. п.)
Дедлок	<code>std::scoped_lock(mtx1, mtx2, ...)</code>
<code>volatile</code> вместо <code>atomic</code>	Заменить на <code>std::atomic<T></code>
Перемешанный вывод	<code>std::osyncstream</code> (C++20)

10. Приведение типов (касты) в C++

10.1. «Запрещённые заклинания» — зачем нужны касты?

В C++ система типов — это ваш главный защитник. Она ловит ошибки на этапе компиляции: нельзя случайно сложить строку с указателем или передать `float` туда, где ожидается `std::vector`. Но иногда программисту **нужно** сказать компилятору: «Я знаю, что делаю — преобразуй этот тип в другой».

Для этого существуют **четыре заклинания** (каста), каждое со своей силой и опасностью. Лектор не зря называет их «запрещёнными» — неправильное использование ведёт к Undefined Behavior.

10.1.1. Почему не один каст, а четыре?

В языке C был всего один способ: (тип)выражение. Он делал *всё подряд* — и безопасные, и опасные преобразования — и программист не мог отличить одно от другого. C++ разделил эту грубую силу на четыре точных инструмента, чтобы:

- Код **явно показывал намерение** программиста
- Опасные операции **бросались в глаза** при чтении кода
- Можно было **искать** конкретные касты `grep`’ом (попробуйте найти `(int)` в миллионе строк — удачи)

10.2. C-style cast — «Грубая сила»

10.2.1. Синтаксис

```
1 double pi = 3.14159;  
2 int truncated = (int)pi;           // C-style cast  
3 int also_works = int(pi);          // Функциональный стиль то же самое )
```

10.2.2. Как это работает внутри?

C-style cast — это **не отдельная операция**. Компилятор перебирает касты в строгом порядке и применяет **первый подходящий**:

1. `const_cast` — попробовать снять/добавить `const`
2. `static_cast` — попробовать стандартное преобразование
3. `static_cast + const_cast` — комбинация
4. `reinterpret_cast` — попробовать переинтерпретировать байты
5. `reinterpret_cast + const_cast` — самая опасная комбинация

10.2.3. Почему это плохо?

```
1 class Base { virtual ~Base() = default; };
2 class Derived : public Base { public: int x = 42; };
3
4 void example() {
5     const Base* base_ptr = new Derived();
6
7     // C-style cast: что именно здесь происходит ?
8     Derived* d = (Derived*)base_ptr;
9     // Ответ: reinterpret_cast + const_cast снял( const
10    // И переинтерпретировал )
11    // Вы это поняли , просто глядя на код ? Вряд ли .
12
13    // Явный C++ каст -- намерение не прозрачно :
14    const Derived* d2 = static_cast<const Derived*>(base_ptr); //
    const сохранён
}
```

10.2.4. Главная опасность

C-style cast **никогда не выдаёт ошибку компиляции** для указателей. Он всегда найдёт способ — пусть даже через `reinterpret_cast`:

```
1 std::string str = "hello";
2 int* p = (int*)&str; // Компилируется! Но это полная чушь .
3                      // reinterpret_cast<int*>(&str) --
4                      // хотя бы видно , что опасно
5
6 // С C++ кастами:
7 // int* p = static_cast<int*>(&str); // Ошибка компиляции !
8 // Несовместимы.
9 // int* p = reinterpret_cast<int*>(&str); // Компилируется,
10 // но ЯВНО опасно
```

Правило: в C++ коде C-style cast использовать нельзя. Точка.

10.3. static_cast — «Безопасное преобразование»

10.3.1. Суть

`static_cast` создаёт **новый объект** другого типа, используя определённые правила преобразования. Это самый частый и самый безопасный каст.

Ключевое слово: **создаёт новый объект**. Старый остаётся нетронутым.

Если вы не знаете, какой каст нужен — скорее всего, нужен **static_cast**.

10.3.2. Сценарий 1: Числовые преобразования

```
1 double pi = 3.14159;
```

```

2  int truncated = static_cast<int>(pi);  // 3 дробная(
    часть отброшена )
3
4  int big = 1'000'000;
5  short small = static_cast<short>(big);  //
    Возможен переполнение  ( )
6
7  // Безкасткомпилятор выдаст предупреждение :
8  int x = 3.14;  // Warning: implicit conversion from 'double' to '
    int'
9  int y = static_cast<int>(3.14);  // Предупреждения нет --
    явно указали  " знаю"

```

Как `static_cast<int>(3.14)` работает на уровне процессора?

Это не просто отбрасывание битов. Числа `int` и `double` хранятся совершенно по-разному:

```

double 3.14 в памяти (IEEE 754):
01000000 00001001 00011110 10111000 01010001 11101100 00000000
    00000000 знак
|   | экспонента | мантисса
|
int 3 в памяти :
00000000 00000000 00000000 00000011

```

`static_cast` **вычисляет** новое значение: извлекает мантиссу и экспоненту, считает целую часть и записывает в формате `int`. Это реальная работа процессора (инструкция `cvttsd2si` на x86).

10.3.3. Сценарий 2: Enum ↔ int

```

1  enum class Color { Red = 0, Green = 1, Blue = 2 };
2
3  Color c = Color::Green;
4  int num = static_cast<int>(c);           // 1
5  Color back = static_cast<Color>(2);      // Color::Blue
6
7  // Без static_cast:
8  // int num = c;  // Ошибка! enum class не преобразуется неявно

```

10.3.4. Сценарий 3: void* → конкретный тип

```

1  // Классический паттерн из C API (pthread, qsort итд ..)
2  void callback(void* data) {
3      // Мы ЗНАЕМ, что передали int*
4      int* ptr = static_cast<int*>(data);
5      std::cout << *ptr << "\n";
6  }
7

```

```

8 int main() {
9     int value = 42;
10    callback(&value);    // int* неявно -> void*
11 }

```

10.3.5. Сценарий 4: Преобразование через конструктор

```

1 class Celsius {
2 public:
3     double temp;
4     explicit Celsius(double t) : temp(t) {}
5 };
6
7 double raw = 36.6;
8 // Celsius c = raw;    // Ошибка! Конструктор explicit
9 Celsius c = static_cast<Celsius>(raw);    // Явный вызов конструктора

```

10.3.6. Сценарий 5: Приведение в иерархии наследования (downcast)

```

1 class Animal {
2 public:
3     virtual ~Animal() = default;
4     virtual void speak() = 0;
5 };
6
7 class Dog : public Animal {
8 public:
9     void speak() override { std::cout << "Гав!\n"; }
10    void fetch() { std::cout << "Принёс палку!\n"; }
11 };
12
13 void process(Animal* animal) {
14     // Мы УБЕДИЛИСЬ, что это Dog например(, из контекста )
15     Dog* dog = static_cast<Dog*>(animal);
16     dog->fetch();    // Работает, если animal действительно Dog
17                     // UB, если animal -- не Dog!
18 }

```

Важно: `static_cast` не проверяет тип в рантайме. Если вы ошиблись — UB. Для безопасной проверки используйте `dynamic_cast`.

10.3.7. Что `static_cast` НЕ может

```

1 int x = 42;
2 // static_cast<double*>(&x);    // Ошибка! int* и double*
   // несовместимы
3 // static_cast<std::string>(x);    // Ошибка! Нет конструктора
   // string(int)

```

```

4
5 const int cx = 10;
6 // static_cast<int&>(cx);           // Ошибка! Нemoжетснять const

```

10.4. reinterpret_cast — «Рентген памяти»

10.4.1. Суть

`reinterpret_cast` не создаёт новый объект. Он говорит: «Возьми эти байты в памяти и считай, что это другой тип». Никаких вычислений, никаких преобразований — чистая переинтерпретация.

10.4.2. Аналогия

Представьте книгу на русском языке. `static_cast` — это перевод на английский (создаётся новый текст с тем же смыслом). `reinterpret_cast` — это попытка прочитать русские буквы как английские (ерунда, но технически возможно).

10.4.3. Пример 1: Заглядываем «внутри» float

```

1 float f = 1.0f;
2
3 // Как float хранится в памяти ?
4 // IEEE 754: 0 01111111 000000000000000000000000 = 0x3F800000
5
6 // НЕПРАВИЛЬНО -- UB по стандарту нарушение (strict aliasing):
7 int bits_bad = *reinterpret_cast<int*>(&f);
8
9 // ПРАВИЛЬНО -- через memcpy стандарт( разрешает):
10 int bits;
11 std::memcpy(&bits, &f, sizeof(float));
12 std::cout << std::hex << bits << "\n"; // 3f800000

```

Strict aliasing rule: обращение к объекту через указатель на «чужой» тип — UB (за исключением `char*`, `unsigned char*`, `std::byte*`).

10.4.4. Пример 2: Указатель → число и обратно

```

1 int x = 42;
2 int* ptr = &x;
3
4 // Сохраняем адрес как число
5 uintptr_t addr = reinterpret_cast<uintptr_t>(ptr);
6 std::cout << "Адрес: 0x" << std::hex << addr << "\n";
7
8 // Восстанавливаем указатель из числа
9 int* restored = reinterpret_cast<int*>(addr);
10 std::cout << *restored << "\n"; // 42

```

```

11
12 // Это один из НЕМНОГИХ гарантированных сценариев для
    reinterpret_cast

```

10.4.5. Пример 3: Работа с сетевыми пакетами

```

1 // Типичный паттерн при работе с сырыми данными из сети
2 struct PacketHeader {
3     uint32_t magic;
4     uint16_t version;
5     uint16_t length;
6 };
7
8 void process_raw_data(char* buffer, size_t size) {
9     // Технически UB из-за strict aliasing и alignment
10    // Но широко используется на практике
11    auto* header = reinterpret_cast<PacketHeader*>(buffer);
12
13    // Безопасная альтернатива :
14    PacketHeader header_safe;
15    std::memcpy(&header_safe, buffer, sizeof(PacketHeader));
16
17    if (header_safe.magic == 0xDEADBEEF) {
18        // Обработка пакета ...
19    }
20 }

```

10.4.6. reinterpret_cast НЕ может снять const

```

1 const int x = 42;
2 // int* p = reinterpret_cast<int*>(&x); // Ошибка компиляции !
3 // Для снятия const нужен const_cast

```

10.4.7. Когда reinterpret_cast оправдан?

Практически **никогда** в обычном коде. Легитимные случаи:

- Преобразование указатель ↔ uintptr_t
- Преобразование между указателями на функции (с осторожностью)
- Низкоуровневый код (драйверы, аллокаторы, сериализация)
- Работа с C API, которое требует void*

10.5. const_cast — «Снятие защиты»

10.5.1. Суть

const_cast — единственный каст, который может **добавлять или убирать** модификаторы const и volatile. Никакой другой каст этого не умеет.

10.5.2. Мягкая форма: добавление const

Происходит **постоянно и неявно** — вы даже не замечаете:

```
1 std::vector<int> vec = {1, 2, 3};
2
3 // Когда вызываете size(), неявно происходит :
4 // vec неконстантный() -> const vec& константная( ссылка внутри
   size())
5 size_t s = vec.size(); // size() объявлен как : size_t size()
   const;
6
7 // Это эквивалентно :
8 const std::vector<int>& const_ref = vec; // Неявное добавление
   const
9 size_t s2 = const_ref.size();
10
11 // Явный const_cast не( нужен, но для понимания ):
12 const std::vector<int>& ref = const_cast<const> std::vector<int>&>(
   vec);
```

Добавление const — **бесплатная** операция. Процессор не выполняет никаких инструкций — это чисто компиляторная проверка.

10.5.3. Жёсткая форма: снятие const

Вот здесь начинается опасность:

```
1 void legacy_api(char* str) {
2     // Старая функция, которая НЕ модифицирует строку,
3     // но автор забыл написать const
4     std::cout << str << "\n";
5 }
6
7 void modern_code(const std::string& s) {
8     // Нужно вызвать legacy_api, но у нас const
9     // legacy_api(s.c_str()); // Ошибка! c_str() возвращает const
   char*
10
11     legacy_api(const_cast<char*>(s.c_str())); // Снимаем const
12     // Безопасно ТОЛЬКО если legacy_api реально не изменяет данные
   !
13 }
```

10.5.4. Главное правило и UB

```
1 // БЕЗОПАСНО: объектнозначальноНЕ const, мыпросто снимаемлишний  
   "" const  
2 int x = 42;  
3 const int& ref = x; // Добавили const  
4 int& mutable_ref = const_cast<int&>(ref); // Сняли const  
5 mutable_ref = 100; // ОК -- x изначальноне const  
6 std::cout << x << "\n"; // 100  
7  
8 // UB: объектнозначальноОБЪЯВЛЕНкак const  
9 const int y = 42;  
10 int& bad_ref = const_cast<int&>(y); // Компилируется...  
11 bad_ref = 100; // UNDEFINED BEHAVIOR!  
12 // Компилятормогпоместить y в read-only память  
13 // Илизаменитьвсеобращенияк y налитерал 42
```

Почему UB? Компилятор имеет право:

- Поместить const объект в **read-only секцию** памяти (запись = segfault)
- **Заинлайнить** значение: заменить все y на литерал 42 при компиляции
- Оптимизировать код, **предполагая**, что значение никогда не меняется

10.5.5. Реальный use case: избежание дублирования кода

Классический паттерн для operator[]:

```
1 class MyArray {  
2     int data[100];  
3 public:  
4     // Константнаяверсия -- основнаялогика  
5     const int& operator[](size_t idx) const {  
6         // Допустим, тутсложнаяпроверкаграниц , логированиеитд  
7         ..  
8         if (idx >= 100) throw std::out_of_range("Index out of  
9         bounds");  
10        return data[idx];  
11    }  
12  
13    // Неконстантнаяверсия -- делегируеткконстантной  
14    int& operator[](size_t idx) {  
15        // Шаг 1: добавляем const к *this -> вызываем constверсию-  
16        const auto& result = static_cast<const MyArray&>(*this)[  
17        idx];  
18        // Шаг 2: снимаем const срезультата  
19        return const_cast<int&>(result);  
20        // Безопасно! Мызнаем , что *this не const мы(  
21        внеконстантномметоде )  
22    }  
23 };
```

Этот паттерн рекомендован Скоттом Мейерсом в «Effective C++».

10.6. `dynamic_cast` — «Безопасный детектив» (краткий обзор)

`dynamic_cast` проверяет тип в **рантайме** через таблицу виртуальных функций (RTTI). Подробно изучается в теме ООП, но вот основная идея:

```
1  class Base {
2  public:
3      virtual ~Base() = default;    // Виртуальный деструктор ОБЯЗАТЕЛЕН
4  };
5
6  class Derived : public Base {
7  public:
8      void special_method() { std::cout << "Я Derived!\n"; }
9  };
10
11 void safe_process(Base* base) {
12     // dynamic_cast проверяет реальный тип в рантайме
13     Derived* d = dynamic_cast<Derived*>(base);
14
15     if (d != nullptr) {
16         d->special_method();    // Гарантированно безопасно
17     } else {
18         std::cout << "Это не Derived\n";
19     }
20 }
21
22 // Для сылок : бросает std::bad_cast при неудаче
23 void safe_process_ref(Base& base) {
24     try {
25         Derived& d = dynamic_cast<Derived&>(base);
26         d.special_method();
27     } catch (const std::bad_cast& e) {
28         std::cout << "He Derived: " << e.what() << "\n";
29     }
30 }
```

Свойство	static_cast	dynamic_cast
Проверка в рантайме	нет	да
Нужен virtual	нет	да
Скорость	быстрый (0 затрат)	медленнее (RTTI)
При ошибке	UB	nullptr или исключение

10.7. Сводная таблица: какой каст когда

Задача	Каст	Пример
<code>int → double</code>	<code>static_cast</code>	<code>static_cast<double>(42)</code>
<code>enum → int</code>	<code>static_cast</code>	<code>static_cast<int>(Color::Red)</code>
<code>Base* → Derived*</code> (уверен)	<code>static_cast</code>	<code>static_cast<Derived*>(base)</code>
<code>Base* → Derived*</code> (не уверен)	<code>dynamic_cast</code>	<code>dynamic_cast<Derived*>(base)</code>
Посмотреть байты объекта	<code>memcpy</code>	Через <code>char*</code> или <code>memcpy</code>
Указатель ↔ число	<code>reinterpret_cast</code>	<code>reinterpret_cast<uintptr_t></code>
Снять <code>const</code> для legacy API	<code>const_cast</code>	<code>const_cast<char*>(c_str)</code>
<code>void* → конкретный тип</code>	<code>static_cast</code>	<code>static_cast<int*>(void_ptr)</code>
«Хочу чтобы скомпилировалось»	Ничего	Подумайте ещё раз о дизайне

10.8. Упражнения

10.8.1. Упражнение 1: Найди ошибку

Что не так в каждом фрагменте? Какой результат?

```

1 // Фрагмент А
2 float f = 3.14f;
3 int* p = (int*)&f;
4 std::cout << *p << "\n";
5
6 // Фрагмент В
7 const std::string name = "Alice";
8 std::string& ref = const_cast<std::string&>(name);
9 ref = "Bob";
10
11 // Фрагмент С
12 int x = 65;
13 char* c = static_cast<char*>(&x);

```

Решение:

- **Фрагмент А:** C-style cast выполняет `reinterpret_cast<int*>(&f)`. Чтение `*p` — UB (нарушение strict aliasing: `int*` указывает на объект типа `float`). На практике выведет 1078523331 (0x4048F5C3 — IEEE 754 представление 3.14f), но полагаться на это нельзя.
- **Фрагмент В:** UB! Объект `name` объявлен как `const`. Снятие `const` и модификация — Undefined Behavior. Компилятор мог разместить "Alice" в read-only памяти (segfault). Или заоптимизировать и продолжать «видеть» "Alice".
- **Фрагмент С:** Ошибка компиляции! `static_cast` не может преобразовать `int* → char*` (несовместимые типы указателей). Для этого нужен `reinterpret_cast<` — и это один из немногих случаев, где такой каст легален (через `char*` можно читать байты любого объекта).

10.8.2. Упражнение 2: Перепиши без C-style cast

Замените C-style касты на правильные C++ касты:

```
1 double d = 3.14;
2 int i = (int)d;
3
4 const char* s = "hello";
5 char* p = (char*)s;
6
7 Base* b = new Derived();
8 Derived* d = (Derived*)b;
9
10 void* raw = malloc(100);
11 int* arr = (int*)raw;
```

Решение:

```
1 double d = 3.14;
2 int i = static_cast<int>(d); // Числовое преобразование
3
4 const char* s = "hello";
5 char* p = const_cast<char*>(s); // Снятие const модификация( = UB
6     !)
7
8 Base* b = new Derived();
9 Derived* d = dynamic_cast<Derived*>(b); // Безопасный downcast
10     проверкой
11 // Или static_cast<Derived*>(b) если вы на 100% уверены в типе
12
13 void* raw = malloc(100);
14 int* arr = static_cast<int*>(raw); // void* -> конкретный тип
```

10.8.3. Упражнение 3: Что выведет программа?

```
1 #include <iostream>
2
3 int main() {
4     int x = 42;
5     const int& cref = x;
6
7     int& ref = const_cast<int&>(cref);
8     ref = 100;
9
10     std::cout << "x = " << x << "\n";
11     std::cout << "cref = " << cref << "\n";
12     std::cout << "ref = " << ref << "\n";
13
14     return 0;
15 }
```

Решение:

Вывод:

```
x = 100
cref = 100
ref = 100
```

Это **не UB**, потому что `x` изначально объявлен как обычный (не `const`) `int`. Мы лишь добавили `const` через ссылку, а потом сняли его обратно. Все три имени (`x`, `cref`, `ref`) ссылаются на одну и ту же область памяти, поэтому изменение через `ref` видно через все имена.

Было бы UB, если бы `x` был объявлен как `const int x = 42;`.

10.8.4. Упражнение 4: Реализуй безопасное приведение

Напишите шаблонную функцию `safe_narrow`, которая преобразует числовой тип с проверкой потери данных:

```
1 template<typename To, typename From>
2 To safe_narrow(From value) {
3     // Вашкодздесь
4 }
5
6 // Использование:
7 int big = 1'000'000;
8 short s = safe_narrow<short>(big); // Должноброситьисключение !
9
10 double pi = 3.14;
11 int i = safe_narrow<int>(pi); // Должноброситьисключение !
    потеря( .14)
```

Решение:

```
1 #include <stdexcept>
2 #include <sstream>
3
4 template<typename To, typename From>
5 To safe_narrow(From value) {
6     To converted = static_cast<To>(value);
7
8     // Проверяем: еслипреобразоватьобратно ,
9     // получимлиисходноезначение ?
10    if (static_cast<From>(converted) != value) {
11        std::ostringstream oss;
12        oss << "Narrowing conversion lost data: " << value
13            << " became " << converted;
14        throw std::runtime_error(oss.str());
15    }
16    return converted;
17 }
18
19 // Тесты:
20 int main() {
```

```

21 // Безопасные преобразования
22 short s1 = safe_narrow<short>(42);           // OK: 42
           помещается в short
23 int i1 = safe_narrow<int>(3.0);              // OK: 3.0 -> 3
           без потерь
24
25 // Опасные -- бросают исключение
26 try {
27     short s2 = safe_narrow<short>(1'000'000); //
           Переполнение!
28 } catch (const std::runtime_error& e) {
29     std::cout << e.what() << "\n";
30     // "Narrowing conversion lost data: 1000000 became 16960"
31 }
32
33 try {
34     int i2 = safe_narrow<int>(3.14); // Потеря дробной части !
35 } catch (const std::runtime_error& e) {
36     std::cout << e.what() << "\n";
37     // "Narrowing conversion lost data: 3.14 became 3"
38 }
39 }

```

Идея: `static_cast` в обе стороны. Если после «туда-обратно» значение изменилось — данные потеряны.

10.8.5. Упражнение 5: Детектив по кастам

Определите, какой именно каст скрывается за каждым C-style cast. Помните порядок: `const_cast` → `static_cast` → `static_cast` + `const_cast` → `reinterpret_cast` → `reinterpret_cast` + `const_cast`.

```

1 // 1.
2 double d = 3.14;
3 int i = (int)d;
4
5 // 2.
6 const int x = 42;
7 int* p = (int*)&x;
8
9 // 3.
10 int* ptr = (int*)0xDEADBEEF;
11
12 // 4.
13 class A {};
14 class B : public A {};
15 A* a = new B();
16 B* b = (B*)a;
17
18 // 5.
19 const double cd = 2.71;
20 int ci = (int)cd;

```

Решение:

1. `static_cast<int>(d)` — стандартное числовое преобразование `double` → `int`. `const_cast` не подходит (типы разные), `static_cast` подходит первым.
2. `const_cast<int*>(&x)` — `&x` имеет тип `const int*`, а цель `int*`. Разница только в `const`, поэтому это `const_cast`. Первый в очереди, и он подходит.
3. `reinterpret_cast<int*>(0xDEADBEEF)` — целое число → указатель. `const_cast` не подходит (разные типы), `static_cast` не умеет `int` → указатель. Поэтому `reinterpret_cast`.
4. `static_cast<B*>(a)` — приведение вниз по иерархии (downcast). `const_cast` не подходит, `static_cast` подходит (есть отношение наследования).
5. `static_cast<int>(cd)` — `cd` это значение (rvalue), `const` на значениях не мешает. Это просто `static_cast<int>(cd)` — числовое преобразование.

10.8.6. Упражнение 6: Реализуй type-running безопасно

Напишите функцию, которая показывает побитовое представление любого типа:

```
1 template<typename T>
2 std::string bit_representation(const T& value);
3
4 // Использование:
5 std::cout << bit_representation(3.14f) << "\n";
6 // "01000000 01001000 11110101 11000011"
7
8 std::cout << bit_representation(42) << "\n";
9 // "00000000 00000000 00000000 00101010"
```

Решение:

```
1 #include <cstring>
2 #include <string>
3 #include <bitset>
4 #include <sstream>
5
6 template<typename T>
7 std::string bit_representation(const T& value) {
8     // Читаем байты через memcpy -- это БЕЗОПАСНО по стандарту
9     unsigned char bytes[sizeof(T)];
10    std::memcpy(bytes, &value, sizeof(T));
11
12    std::ostringstream oss;
13    for (size_t i = 0; i < sizeof(T); ++i) {
14        if (i > 0) oss << " ";
15        oss << std::bitset<8>(bytes[i]);
16    }
```

```

17     return oss.str();
18 }
19
20 int main() {
21     std::cout << "float 3.14f:  " << bit_representation(3.14f) <<
        "\n";
22     std::cout << "int 42:      " << bit_representation(42) << "\n";
23     std::cout << "double 3.14:  " << bit_representation(3.14) << "\n";
24     std::cout << "char 'A':    " << bit_representation('A') << "\n";
25 }

```

Ключевой момент: мы используем `std::memcpu`, а **не** `reinterpret_cast`. Это единственный стандартно-безопасный способ «посмотреть на байты» объекта. Чтение через `unsigned char*` / `std::byte*` тоже допустимо по стандарту.

10.9. Итоговая шпаргалка

Какойкаствыбрать	?
Нужнопреобразоватьзначение	?
(<code>int</code> -> <code>double</code> , <code>enum</code> -> <code>int</code> , <code>void*</code> -> <code>T*</code>)	
->	<code>static_cast</code>
Нужноснятьдобавить	/ <code>const</code> ?
->	<code>const_cast</code>
Нужнопосмотретьнабайты	/ указательчисло<->?
->	<code>reinterpret_cast</code> а(лучше <code>memcpu</code>)
Нужнобезопаснопривеститипвиерархии	?
->	<code>dynamic_cast</code>
Незнаетекакой	?
->	<code>static_cast</code> и(подумайте, правильнолиспроектирован код , еслиприходитсякастовать)
C-style cast?	
->	НИКОГДАв C++ коде

11. Иерархия `std::exception`

11.1. Зачем нужна единая иерархия исключений?

В разделе про `throw/catch` мы видели, что бросать можно **любой** объект. Но если каждый программист будет бросать свои типы — наступит хаос: как ловить исключения из чужих библиотек?

Стандартная библиотека определяет **базовый класс** `std::exception` и целое дерево наследников. Это позволяет:

- Ловить **все** стандартные исключения одним `catch(const std::exception&)`
- Ловить **конкретные** типы более специализированными блоками `catch`
- Создавать **свои** исключения, наследуясь от стандартных

11.2. Класс `std::exception`

Определён в заголовке `<exception>`.

11.2.1. Интерфейс

```
1 class exception {
2 public:
3     exception() noexcept;
4     exception(const exception&) noexcept;
5     exception& operator=(const exception&) noexcept;
6     virtual ~exception() noexcept;
7
8     virtual const char* what() const noexcept;
9 };
```

- Все специальные методы помечены `noexcept` — исключение **не должно** бросать исключений при своём создании/копировании.
- Единственный содержательный метод — `what()`: возвращает C-строку с описанием ошибки.
- `what()` объявлен `virtual` — наследники переопределяют его.

11.2.2. Пример использования

```
1 #include <iostream>
2 #include <exception>
3
4 int main() {
5     try {
6         throw std::exception(); // Базовоеисключениередко (
7         бросаютнапрямую )
8     } catch (const std::exception& e) {
```

```

8         std::cout << e.what() << "\n"; // "std::exception"
           зависит( отреализации )
9     }
10 }

```

11.3. Дерево стандартных исключений

```

std::exception
|
+-- std::logic_error ошибки ( логики программы )
|   +-- std::invalid_argument неверный ( аргумент )
|   +-- std::domain_error значение ( вне области определения )
|   +-- std::length_error превышение ( максимального размера )
|   +-- std::out_of_range индекс ( за пределами )
|
+-- std::runtime_error ошибки ( времени выполнения )
|   +-- std::range_error результат ( вне диапазона )
|   +-- std::overflow_error арифметическое ( переполнение )
|   +-- std::underflow_error арифметическое ( антипереполнение )
|   +-- std::system_error ошибки ( ОС, C++11 )
|
+-- std::bad_alloc нехватка ( памяти при new )
+-- std::bad_cast неудачный ( dynamic_cast )
+-- std::bad_typeid (typeid от nullptr)
+-- std::bad_exception нарушение ( спецификации исключений )

```

11.4. std::logic_error и std::runtime_error

Оба определены в <stdexcept> и хранят строку сообщения.

11.4.1. Конструкторы

```

1  #include <stdexcept>
2
3  // Принимают std::string или const char*
4  std::logic_error err1("ошибка логики");
5  std::runtime_error err2("ошибка выполнения");
6
7  // what() вернёт переданную строку
8  std::cout << err1.what() << "\n"; // ошибка" логики"
9  std::cout << err2.what() << "\n"; // ошибка" выполнения"

```

11.4.2. Когда что бросать?

Тип	Смысл	Пример
logic_error	Ошибка, которую <i>можно</i> было предотвратить проверкой в коде	Передали отрицательный размер массива
runtime_error	Ошибка, которую <i>нельзя</i> предсказать заранее	Файл не найден, сеть недоступна

11.5. Наследники `std::logic_error`

11.5.1. `std::invalid_argument`

Аргумент функции имеет недопустимое значение.

```
1 #include <stdexcept>
2 #include <string>
3
4 int parse_positive(const std::string& s) {
5     int val = std::stoi(s); // сам stoi может бросить
6     // invalid_argument
7     if (val <= 0) {
8         throw std::invalid_argument("expected positive number, got
9         : " + s);
10    }
11    return val;
12 }
13
14 int main() {
15     try {
16         parse_positive("-5");
17     } catch (const std::invalid_argument& e) {
18         std::cout << "Ошибка: " << e.what() << "\n";
19         // Ошибка: expected positive number, got: -5"
20     }
21 }
```

11.5.2. `std::out_of_range`

Индекс или ключ вне допустимого диапазона.

```
1 #include <vector>
2 #include <stdexcept>
3
4 int main() {
5     std::vector<int> v = {10, 20, 30};
6
7     try {
8         int val = v.at(100); // бросает std::out_of_range
9     } catch (const std::out_of_range& e) {
10    }
```

```

10         std::cout << e.what() << "\n"; //
           сообщениезависитотреализации
11     }
12
13     // Длясравнения : v[100] -- UB, никакогоисключения
14 }

```

11.5.3. `std::length_error`

Попытка создать объект, превышающий максимально допустимый размер.

```

1  #include <vector>
2  #include <stdexcept>
3
4  int main() {
5      try {
6          std::vector<int> v;
7          v.reserve(v.max_size() + 1); // бросает std::length_error
8      } catch (const std::length_error& e) {
9          std::cout << e.what() << "\n";
10     }
11 }

```

11.5.4. `std::domain_error`

Математическая функция получила значение вне области определения.

```

1  #include <cmath>
2  #include <stdexcept>
3
4  double safe_sqrt(double x) {
5      if (x < 0) {
6          throw std::domain_error("sqrt of negative number");
7      }
8      return std::sqrt(x);
9  }

```

11.6. Наследники `std::runtime_error`

11.6.1. `std::overflow_error` и `std::underflow_error`

```

1  #include <stdexcept>
2  #include <limits>
3
4  unsigned int safe_multiply(unsigned int a, unsigned int b) {
5      if (a != 0 && b > std::numeric_limits<unsigned int>::max() / a
6          ) {
7          throw std::overflow_error("unsigned multiplication
8              overflow");
9      }
10 }

```

```

7     }
8     return a * b;
9 }
10
11 int main() {
12     try {
13         auto result = safe_multiply(1'000'000'000u, 5u);
14     } catch (const std::overflow_error& e) {
15         std::cout << e.what() << "\n";
16     }
17 }

```

11.6.2. `std::system_error` (C++11)

Обёртка над ошибками операционной системы. Хранит `std::error_code`.

```

1 #include <system_error>
2 #include <fstream>
3
4 void open_file(const std::string& path) {
5     std::ifstream f(path);
6     if (!f.is_open()) {
7         throw std::system_error(
8             std::make_error_code(std::errc::
9                 no_such_file_or_directory),
10            "cannot open: " + path
11        );
12    }
13 }
14
15 int main() {
16     try {
17         open_file("/nonexistent/file.txt");
18     } catch (const std::system_error& e) {
19         std::cout << "Код: " << e.code() << "\n";
20         std::cout << "Что: " << e.what() << "\n";
21     }
22 }

```

11.7. Отдельные исключения (не наследуют `logic/runtime_error`)

11.7.1. `std::bad_alloc`

Бросается оператором `new` при нехватке памяти.

```

1 #include <new>
2 #include <iostream>
3
4 int main() {
5     try {
6         // Попытка выделить абсурдно много памяти

```

```

7      int* p = new int[1'000'000'000'000LL];
8  } catch (const std::bad_alloc& e) {
9      std::cout << "Нехватка памяти: " << e.what() << "\n";
10 }
11
12 // Альтернатива: new(std::nothrow) возвращает nullptr
    вместоисключения
13 int* p = new(std::nothrow) int[1'000'000'000'000LL];
14 if (p == nullptr) {
15     std::cout << "Не удалось выделить память \n";
16 }
17 }

```

11.7.2. std::bad_cast

Бросается при неудачном dynamic_cast к ссылке.

```

1 #include <typeinfo>
2
3 class Base { public: virtual ~Base() = default; };
4 class Derived : public Base {};
5 class Other : public Base {};
6
7 int main() {
8     Derived d;
9     Base& ref = d;
10
11     try {
12         Other& o = dynamic_cast<Other&>(ref); // Неудача!
13     } catch (const std::bad_cast& e) {
14         std::cout << e.what() << "\n"; // "std::bad_cast"
15     }
16 }

```

11.8. Создание собственных исключений

```

1 #include <stdexcept>
2 #include <string>
3
4 class HttpError : public std::runtime_error {
5     int status_code_;
6 public:
7     HttpError(int code, const std::string& msg)
8         : std::runtime_error(msg), status_code_(code) {}
9
10    int status_code() const noexcept { return status_code_; }
11 };
12
13 class NotFoundError : public HttpError {
14 public:

```

```

15     explicit NotFoundError(const std::string& url)
16         : HttpError(404, "Not found: " + url) {}
17 };
18
19 void fetch(const std::string& url) {
20     throw NotFoundError(url);
21 }
22
23 int main() {
24     try {
25         fetch("/api/users/999");
26     } catch (const HttpError& e) {
27         std::cout << "HTTP " << e.status_code() << ": " << e.what
28             () << "\n";
29         // "HTTP 404: Not found: /api/users/999"
30     }
31     // Также можно поймать как std::runtime_error или std::
    exception

```

Рекомендации:

- Наследуйтесь от `std::runtime_error` или `std::logic_error`, а не от того же `std::exception` — так вы получаете хранение строки бесплатно.
- Все методы исключений делайте по-эксперт (кроме конструктора, которому нужно скопировать строку).

12. std::optional (C++17)

12.1. Зачем нужен std::optional?

Как обозначить «значения нет»? Исторически использовались костыли:

Способ	Пример	Проблема
Магическое значение	<code>return -1;</code>	А если -1 — допустимый результат?
Указатель	<code>return nullptr;</code>	Динамическая память, ownership неясен
Пара <code>bool + value</code>	<code>std::pair<bool, T></code>	Можно прочитать <code>value</code> при <code>false</code>
Исключение	<code>throw NotFound{}</code>	Дорого, если «нет значения» — обычная ситуация

`std::optional<T>` — это контейнер, который хранит **либо значение типа T, либо ничего**. Определён в `<optional>`.

12.2. Создание

12.2.1. Конструкторы

```
1 #include <optional>
2 #include <string>
3
4 // 1. Пустой optional
5 std::optional<int> empty; // нетзначения
6 std::optional<int> also_empty = std::nullopt; // тожесамое , явно
7
8 // 2. Сзначением
9 std::optional<int> opt1 = 42; // содержит 42
10 std::optional<int> opt2(42); // тожесамое
11 std::optional<std::string> opt3("hello"); // содержит "hello"
12
13 // 3. in-place конструированиебез ( лишнихкопий )
14 std::optional<std::string> opt4(std::in_place, 5, 'x'); //
    содержит "xxxxx"
15 // Эквивалентно: std::string(5, 'x')
16
17 // 4. make_optional выводит( типавтоматически )
18 auto opt5 = std::make_optional(3.14); // std::optional<
    double>
19 auto opt6 = std::make_optional<std::string>(5, 'x'); // "xxxxx"
```

12.3. Проверка наличия значения

12.3.1. `has_value()` и `operator bool()`

```
1 std::optional<int> opt = 42;
2 std::optional<int> empty;
3
4 // Способ 1: has_value()
5 if (opt.has_value()) {
6     std::cout << "Есть значение\n";
7 }
8
9 // Способ 2: неявное преобразование в bool
10 if (opt) {
11     std::cout << "Есть значение\n"; // то же самое
12 }
13
14 if (!empty) {
15     std::cout << "Пусто\n";
16 }
```

Оба способа эквивалентны. `operator bool()` — просто сокращение для `has_value()`.

12.4. Доступ к значению

12.4.1. `operator*()` и `operator->()`

Доступ без проверки. Если значения нет — UB.

```
1 std::optional<std::string> opt = "hello";
2
3 // operator* -- доступ к значению
4 std::string& s = *opt;
5 std::cout << s << "\n"; // "hello"
6
7 // operator-> -- доступ к членам
8 std::cout << opt->size() << "\n"; // 5
9
10 // UB! Нет значения, но обращаемся:
11 std::optional<int> empty;
12 // int x = *empty; // UB -- поведение не определено
```

12.4.2. `value()`

Доступ с проверкой. Если значения нет — бросает `std::bad_optional_access`.

```
1 #include <optional>
2 #include <iostream>
3
4 int main() {
5     std::optional<int> opt = 42;
6     std::cout << opt.value() << "\n"; // 42
7 }
```

```

7
8     std::optional<int> empty;
9     try {
10         int x = empty.value(); // Бросает исключение !
11     } catch (const std::bad_optional_access& e) {
12         std::cout << e.what() << "\n"; // "bad optional access"
13     }
14 }

```

12.4.3. value_or(default)

Возвращает значение, если есть, иначе — переданное значение по умолчанию.

```

1 std::optional<int> opt = 42;
2 std::optional<int> empty;
3
4 std::cout << opt.value_or(0) << "\n"; // 42 есть (значение)
5 std::cout << empty.value_or(0) << "\n"; // 0 пусто (, вернул
    default)
6
7 // Удобно для конфигурации :
8 std::optional<std::string> user_name; // пользователь не указали
9 std::string name = user_name.value_or("Anonymous");

```

12.5. Модификация

12.5.1. emplace(args...)

Создаёт значение «на месте», разрушая предыдущее (если было).

```

1 std::optional<std::string> opt;
2
3 opt.emplace(5, 'a'); // Создаёт "aaaaa" внутри optional
4 std::cout << *opt << "\n"; // "aaaaa"
5
6 opt.emplace("hello"); // Старое разрушено, создано "hello"
7 std::cout << *opt << "\n"; // "hello"

```

12.5.2. reset()

Разрушает значение (если было), optional становится пустым.

```

1 std::optional<std::string> opt = "hello";
2 std::cout << opt.has_value() << "\n"; // 1 (true)
3
4 opt.reset();
5 std::cout << opt.has_value() << "\n"; // 0 (false)
6
7 // Эквивалентно:
8 opt = std::nullopt;

```

12.5.3. Присваивание

```
1 std::optional<int> opt;
2
3 opt = 42;           // Теперь содержит 42
4 opt = 100;          // Теперь содержит 100
5 opt = std::nullopt; // Теперь пустой
6 opt = {};           // Тоже пустой
```

12.6. Сравнение

`std::optional` поддерживает все операторы сравнения:

```
1 std::optional<int> a = 5;
2 std::optional<int> b = 10;
3 std::optional<int> c;           // пустой
4 std::optional<int> d;           // пустой
5
6 // optional с optional
7 std::cout << (a < b) << "\n";   // 1 (true): 5 < 10
8 std::cout << (a == b) << "\n";  // 0 (false)
9 std::cout << (c == d) << "\n";  // 1 (true): оба пустые
10 std::cout << (c < a) << "\n";   // 1 (true): пустой <
    любое значение
11
12 // optional с значением
13 std::cout << (a == 5) << "\n";  // 1 (true)
14 std::cout << (c == 5) << "\n";  // 0 (false): пустой != значение
15
16 // optional с nullopt
17 std::cout << (c == std::nullopt) << "\n"; // 1 (true)
18 std::cout << (a == std::nullopt) << "\n"; // 0 (false)
```

Правила:

- Два пустых `optional` **равны**
- Пустой `optional` **меньше** любого непустого
- Два непустых сравниваются по значениям внутри

12.7. Монадический интерфейс (C++23)

C++23 добавил три метода для удобной цепочки операций.

12.7.1. `and_then(f)`

Если есть значение — применяет `f` (которая сама возвращает `optional`). Если пусто — возвращает пустой `optional`.

```

1 // C++23
2 std::optional<std::string> get_env(const char* name); //
   может вернуть nullopt
3 std::optional<int> parse_int(const std::string& s); //
   может вернуть nullopt
4
5 // Цепочка: получить переменную среды -> распарсить как int
6 std::optional<int> port = get_env("PORT").and_then(parse_int);
7
8 // Без and_then (C++17):
9 std::optional<int> port2;
10 if (auto env = get_env("PORT")) {
11     port2 = parse_int(*env);
12 }

```

12.7.2. transform(f)

Если есть значение — применяет `f` и оборачивает результат в `optional`. Если пусто — возвращает пустой.

```

1 // C++23
2 std::optional<std::string> name = "hello";
3 std::optional<size_t> len = name.transform([](const std::string& s
   ) {
4     return s.size();
5 });
6 // len содержит 5
7
8 std::optional<std::string> empty;
9 std::optional<size_t> no_len = empty.transform([](const std::
   string& s) {
10     return s.size();
11 });
12 // no_len пуст

```

12.7.3. or_else(f)

Если есть значение — возвращает его. Если пусто — вызывает `f` (которая возвращает `optional`).

```

1 // C++23
2 std::optional<std::string> primary_config(); // может быть
   nullopt
3 std::optional<std::string> fallback_config(); // может быть
   nullopt
4
5 // Попробовать primary, если нет -- fallback
6 auto config = primary_config().or_else(fallback_config);

```

12.8. Где хранится значение?

`std::optional<T>` хранит объект **внутри себя** (на стеке), без динамической памяти. Упрощённая реализация:

```
1  template<typename T>
2  class optional {
3      alignas(T) unsigned char storage_[sizeof(T)]; // Сырая память
4      bool has_value_;
5
6      T& get() { return *reinterpret_cast<T*>(storage_); }
7      const T& get() const { return *reinterpret_cast<const T*>(
8          storage_); }
9
10     public:
11         optional() : has_value_(false) {}
12
13         optional(const T& value) : has_value_(true) {
14             new (storage_) T(value); // placement new
15         }
16
17         ~optional() {
18             if (has_value_) get().~T(); // явный вызов деструктора
19         }
20
21         bool has_value() const { return has_value_; }
22         T& operator*() { return get(); }
23         T& value() {
24             if (!has_value_) throw std::bad_optional_access();
25             return get();
26         }
27
28         void reset() {
29             if (has_value_) {
30                 get().~T();
31                 has_value_ = false;
32             }
33     };
```

$\text{sizeof}(\text{std::optional}<\text{T}>) \approx \text{sizeof}(\text{T}) + \text{sizeof}(\text{bool}) + \text{выравнивание}$.

12.9. Типичные паттерны использования

12.9.1. Функция, которая может не вернуть результат

```
1  #include <optional>
2  #include <vector>
3  #include <algorithm>
4
5  // Найти элемент в векторе
```

```

6  template<typename T>
7  std::optional<size_t> find_index(const std::vector<T>& v, const T&
   target) {
8      for (size_t i = 0; i < v.size(); ++i) {
9          if (v[i] == target) return i;
10     }
11     return std::nullopt; // Ненашли
12 }
13
14 int main() {
15     std::vector<int> v = {10, 20, 30, 40, 50};
16
17     if (auto idx = find_index(v, 30)) {
18         std::cout << "Найден на позиции " << *idx << "\n"; // 2
19     } else {
20         std::cout << "Не найден\n";
21     }
22 }

```

12.9.2. Ленивая инициализация (кэширование)

```

1  class Config {
2      mutable std::optional<std::string> cached_name_;
3
4  public:
5      const std::string& get_name() const {
6          if (!cached_name_) {
7              cached_name_ = expensive_computation(); //
              Считаю один раз
8          }
9          return *cached_name_;
10     }
11
12     void invalidate() { cached_name_.reset(); } // Сбросить кэш
13 };

```

12.9.3. Необязательные параметры структуры

```

1  struct UserProfile {
2      std::string name;
3      std::string email;
4      std::optional<std::string> phone; // Может отсутствовать
5      std::optional<int> age;           // Может отсутствовать
6  };
7
8  void print_profile(const UserProfile& u) {
9      std::cout << "Name: " << u.name << "\n";
10     std::cout << "Email: " << u.email << "\n";
11 }

```

```
12     if (u.phone) {
13         std::cout << "Phone: " << *u.phone << "\n";
14     }
15
16     std::cout << "Age: " << u.age.value_or(-1) << "\n";
17 }
```

13. `std::expected` (C++23)

13.1. Зачем нужен `std::expected`?

`std::optional` говорит: «значение есть или его нет». Но почему его нет? `optional` не даёт ответа.

`std::expected<T, E>` — это контейнер, который хранит **либо значение типа `T` (успех), либо ошибку типа `E` (неудача)**. Определён в `<expected>` (C++23).

Тип	Хранит	Когда использовать
<code>optional<T></code>	<code>T</code> или ничего	«Может не быть значения»
<code>expected<T, E></code>	<code>T</code> или <code>E</code>	«Может быть ошибка, и важно знать какая»

Это альтернатива исключениям для случаев, когда ошибка — **ожидаемая** часть логики (не исключительная ситуация):

	Исключения	<code>std::expected</code>
Когда использовать	Действительно исключительные ситуации	Ожидаемые ошибки (парсинг, I/O, валидация)
Стоимость ошибки	Дорого (раскрутка стека)	Дёшево (просто union)
Видимость в API	Скрыта (надо читать документацию)	Явна в типе возвращаемого значения

13.2. Создание

13.2.1. Создание со значением (успех)

```
1 #include <expected>
2 #include <string>
3
4 // Неявноиззначения
5 std::expected<int, std::string> result = 42; //
   Содержит значение 42
6
7 // Явно
8 std::expected<int, std::string> result2(42);
9
10 // in-place конструирование
11 std::expected<std::string, int> result3(std::in_place, 5, 'x');
   // "xxxxx"
```

13.2.2. Создание с ошибкой (неудача)

```
1 // Через std::unexpected
2 std::expected<int, std::string> err = std::unexpected("file not
   found");
3
4 // Явное создание
5 std::expected<int, std::string> err2(std::unexpected("parse error"
   ));
6
7 // in-place конструирование ошибки
8 std::expected<int, std::string> err3(std::unexpected, "timeout");
```

Важно: нельзя просто присвоить ошибку напрямую — нужна обёртка `std::unexpected`, чтобы компилятор отличил значение от ошибки:

```
1 // Если T и E -- один тип, без unexpected была бы неоднозначность :
2 std::expected<std::string, std::string> result = "hello"; //
   Это значение !
3 std::expected<std::string, std::string> error =
4     std::unexpected("oops"); //
   Это ошибка !
```

13.3. Проверка состояния

13.3.1. `has_value()` и `operator bool()`

```
1 std::expected<int, std::string> ok = 42;
2 std::expected<int, std::string> err = std::unexpected("fail");
3
4 if (ok.has_value()) { /* true */ }
5 if (ok) { /* true */ }
6
7 if (err.has_value()) { /* false */ }
8 if (!err) { /* true */ }
```

13.4. Доступ к значению

13.4.1. `operator*()` и `operator->()`

Доступ без проверки. Если содержит ошибку — UB.

```
1 std::expected<std::string, int> result = "hello";
2
3 std::string& s = *result;
4 std::cout << s << "\n"; // "hello"
5 std::cout << result->size() << "\n"; // 5
6
7 // UB:
8 std::expected<int, std::string> err = std::unexpected("fail");
9 // int x = *err; // UB!
```

13.4.2. value()

Доступ с проверкой. Если ошибка — бросает `std::bad_expected_access<E>`.

```
1 std::expected<int, std::string> ok = 42;
2 std::cout << ok.value() << "\n"; // 42
3
4 std::expected<int, std::string> err = std::unexpected("file not
   found");
5 try {
6     int x = err.value(); // Бросаетисключение !
7 } catch (const std::bad_expected_access<std::string>& e) {
8     // e.error() возвращаетхранимуюошибку
9     std::cout << "Ошибка: " << e.error() << "\n"; // "file not
   found"
10 }
```

13.4.3. value_or(default)

```
1 std::expected<int, std::string> ok = 42;
2 std::expected<int, std::string> err = std::unexpected("fail");
3
4 std::cout << ok.value_or(0) << "\n"; // 42
5 std::cout << err.value_or(0) << "\n"; // 0
```

13.5. Доступ к ошибке

13.5.1. error()

Возвращает хранимую ошибку. Если содержит значение (а не ошибку) — UB.

```
1 std::expected<int, std::string> err = std::unexpected("timeout");
2
3 std::cout << err.error() << "\n"; // "timeout"
4
5 // UB:
6 std::expected<int, std::string> ok = 42;
7 // auto e = ok.error(); // UB! Естьзначение , анеошибка
```

13.6. Модификация

13.6.1. emplace(args...)

```
1 std::expected<std::string, int> result = std::unexpected(404);
2
3 // Заменяемошибкуназначение
4 result.emplace(5, 'x'); // Теперьсодержит "xxxxx"
5 std::cout << *result << "\n"; // "xxxxx"
```

13.6.2. Присваивание

```
1 std::expected<int, std::string> result = 42;
2
3 result = 100; // Значение 100
4 result = std::unexpected("new error"); // Теперь ошибка
5 result = 200; // Снова значение
```

13.7. Монадический интерфейс (C++23)

Те же три метода, что и у `optional`, плюс `or_else` работает с ошибкой.

13.7.1. `and_then(f)`

Если значение — применяет `f` (возвращающую `expected`). Если ошибка — пробрасывает ошибку.

```
1 using Result = std::expected<int, std::string>;
2
3 Result parse_int(const std::string& s) {
4     try {
5         return std::stoi(s);
6     } catch (...) {
7         return std::unexpected("parse error: " + s);
8     }
9 }
10
11 Result validate_positive(int x) {
12     if (x > 0) return x;
13     return std::unexpected("must be positive");
14 }
15
16 // Цепочка: парсим -> проверяем
17 Result result = parse_int("42").and_then(validate_positive);
18 // result содержит 42
19
20 Result fail = parse_int("abc").and_then(validate_positive);
21 // fail содержит ошибку "parse error: abc"
22 // validate_positive даже не вызывалась !
```

13.7.2. `transform(f)`

Если значение — применяет `f`, оборачивает результат в `expected`. Если ошибка — пробрасывает.

```
1 std::expected<int, std::string> result = 42;
2
3 auto doubled = result.transform([](int x) { return x * 2; });
4 // doubled содержит 84
5
```

```

6 std::expected<int, std::string> err = std::unexpected("fail");
7 auto still_err = err.transform([](int x) { return x * 2; });
8 // still_err содержит ошибку "fail"

```

13.7.3. or_else(f)

Если значение — возвращает его. Если ошибка — вызывает `f` с ошибкой.

```

1 using Result = std::expected<int, std::string>;
2
3 Result primary_source();
4 Result fallback_source(const std::string& error);
5
6 // Если primary не удался -- пробуем fallback,
   передавая ему текст ошибки
7 auto result = primary_source().or_else(fallback_source);

```

13.7.4. transform_error(f)

Преобразует ошибку, не трогая значение. Полезно для конвертации типов ошибок.

```

1 std::expected<int, int> result = std::unexpected(404);
2
3 // Преобразуем int-ошибку в строку
4 auto with_msg = result.transform_error([](int code) {
5     return "HTTP error " + std::to_string(code);
6 });
7 // with_msg имеет тип std::expected<int, std::string>
8 // Содержит ошибку "HTTP error 404"

```

13.8. Где хранится значение?

Как и `optional`, хранит данные **внутри себя** без динамической памяти. Внутри — `union` из `T` и `E`:

```

1 // Упрощённая реализация
2 template<typename T, typename E>
3 class expected {
4     union {
5         T value_;
6         E error_;
7     };
8     bool has_value_;
9
10 public:
11     expected(const T& val) : value_(val), has_value_(true) {}
12
13     expected(std::unexpected<E> err)
14         : error_(std::move(err).error()), has_value_(false) {}

```

```

15
16     ~expected() {
17         if (has_value_) value_.~T();
18         else            error_.~E();
19     }
20
21     bool has_value() const { return has_value_; }
22     T& operator*() { return value_; }
23     E& error() { return error_; }
24
25     T& value() {
26         if (!has_value_) throw std::bad_expected_access(error_);
27         return value_;
28     }
29 };

```

`sizeof(std::expected<T, E>) ≈ max(sizeof(T), sizeof(E)) + sizeof(bool)`
+ выравнивание.

13.9. `std::expected<void, E>`

Специализация для функций, которые не возвращают значение, но могут вернуть ошибку.

```

1  std::expected<void, std::string> save_file(const std::string& path
2      ) {
3      std::ofstream f(path);
4      if (!f.is_open()) {
5          return std::unexpected("cannot open: " + path);
6      }
7      f << "data";
8      if (f.fail()) {
9          return std::unexpected("write failed");
10     }
11     return {}; // Успех (void)
12 }
13
14 int main() {
15     auto result = save_file("/tmp/test.txt");
16     if (!result) {
17         std::cerr << "Ошибка: " << result.error() << "\n";
18     }
19 }

```

13.10. Типичные паттерны использования

13.10.1. Парсинг с информативной ошибкой

```

1  #include <expected>
2  #include <string>

```

```

3  #include <charconv>
4
5  enum class ParseError {
6      empty_input,
7      invalid_format,
8      out_of_range,
9  };
10
11  std::expected<int, ParseError> parse_int(std::string_view sv) {
12      if (sv.empty()) {
13          return std::unexpected(ParseError::empty_input);
14      }
15
16      int result;
17      auto [ptr, ec] = std::from_chars(sv.data(), sv.data() + sv.
18          size(), result);
19
20      if (ec == std::errc::invalid_argument) {
21          return std::unexpected(ParseError::invalid_format);
22      }
23      if (ec == std::errc::result_out_of_range) {
24          return std::unexpected(ParseError::out_of_range);
25      }
26
27      return result;
28  }
29
30  int main() {
31      auto r1 = parse_int("42");
32      if (r1) std::cout << "OK: " << *r1 << "\n";
33
34      auto r2 = parse_int("abc");
35      if (!r2) {
36          switch (r2.error()) {
37              case ParseError::empty_input:    std::cout << "Пусто\n"; break;
38              case ParseError::invalid_format: std::cout << "Формат\n"; break;
39              case ParseError::out_of_range:   std::cout << "
40                  Диапазон\n"; break;
41          }
42      }
43  }

```

13.10.2. Цепочка операций

```

1  using Result = std::expected<int, std::string>;
2
3  Result read_config_value(const std::string& key);
4  Result validate_range(int value);

```

```

5 Result apply_multiplier(int value);
6
7 // Элегантная цепочка без вложенных if:
8 auto final_result = read_config_value("timeout")
9     .and_then(validate_range)
10    .and_then(apply_multiplier)
11    .transform([](int x) { return x + 1; });
12
13 // Если любой шаг вернул ошибку -- она пробрасывается до конца
14 if (final_result) {
15     std::cout << "Результат: " << *final_result << "\n";
16 } else {
17     std::cerr << "Ошибка: " << final_result.error() << "\n";
18 }

```

13.11. Сводная таблица: optional vs expected vs исключения

	optional<T>	expected<T,E>	Исключения
Информация об ошибке	Нет	Да (тип E)	Да
Стоимость ошибки	Нулевая	Нулевая	Высокая
Стоимость успеха	Нулевая	Нулевая	Нулевая
Можно забыть проверить	Да	Да	Нет
Видимость в API	Явная	Явная	Скрытая
Стандарт	C++17	C++23	C++98

14. CRTP — Curiously Recurring Template Pattern

14.1. Что такое CRTP?

CRTP (Curiously Recurring Template Pattern) — паттерн, при котором класс наследуется от шаблонного базового класса, **передавая себя** в качестве шаблонного параметра:

```
1 template<typename Derived>
2 class Base {
3     // Base знает"" типсвоегонаследника
4 };
5
6 class MyClass : public Base<MyClass> {
7     // MyClass наследуетсяот Base<MyClass>
8 };
```

Выглядит странно: класс наследуется от шаблона, параметризованного *самим собой*. Отсюда слово «curiously» (любопытно) в названии.

14.1.1. Ключевая идея

Базовый класс Base<Derived> может **вызывать методы наследника** на этапе компиляции, без виртуальных функций:

```
1 template<typename Derived>
2 class Base {
3 public:
4     void interface() {
5         // Приводим this к типунаследникаивызываемегометод
6         static_cast<Derived*>(this)->implementation();
7     }
8 };
9
10 class Concrete : public Base<Concrete> {
11 public:
12     void implementation() {
13         std::cout << "Concrete::implementation()\n";
14     }
15 };
16
17 int main() {
18     Concrete c;
19     c.interface(); // "Concrete::implementation()"
20 }
```

Base::interface() вызывает Derived::implementation() через static_cast. Никаких виртуальных таблиц, никаких косвенных вызовов — всё разрешается **на этапе компиляции**.

14.2. Зачем это нужно? Проблема виртуальных функций

Виртуальные функции — стандартный механизм полиморфизма в C++. Но у них есть цена:

	Виртуальные функции	CRTP
Полиморфизм	Динамический (runtime)	Статический (compile-time)
Накладные расходы	vptr + косвенный вызов через vtable	Нулевые (всё инлайнится)
Объект содержит vptr	Да (+8 байт на x86-64)	Нет
Можно хранить в одном контейнере	Да (vector<Base*>)	Нет (разные типы)
Определяется в	Runtime	Compile-time

CRTP — это **статический полиморфизм**: поведение определяется на этапе компиляции, а не в рантайме. Код получается таким же гибким, но **быстрее**, потому что компилятор видит конкретные типы и может инлайнить вызовы.

14.3. Как работает static_cast в CRTP?

Центральный приём CRTP — приведение this к типу наследника:

```
1 template<typename Derived>
2 class Base {
3 public:
4     Derived& self() {
5         return static_cast<Derived&>(*this);
6     }
7     const Derived& self() const {
8         return static_cast<const Derived&>(*this);
9     }
10 };
```

Почему это безопасно? Потому что Base<Derived> используется **только** как базовый класс для Derived. Объект всегда имеет тип Derived (или его наследника), поэтому static_cast вниз по иерархии здесь корректен.

Когда это опасно? Если кто-то нарушит контракт:

```
1 class A : public Base<A> { /* ... */ };
2 class B : public Base<A> { /* ... */ }; // BUG! B наследует Base<
    A>, а не Base<B>
3                                     // static_cast<A*>(this)
                                     // в объекте B = UB
```

Защита от этой ошибки рассмотрена в разделе [14.10](#).

14.4. Применение 1: Статический полиморфизм

Классический пример — интерфейс фигур без виртуальных функций.

14.4.1. С виртуальными функциями (обычный подход)

```
1 class Shape {
2 public:
3     virtual ~Shape() = default;
4     virtual double area() const = 0;
5     virtual void draw() const = 0;
6 };
7
8 class Circle : public Shape {
9     double radius_;
10 public:
11     explicit Circle(double r) : radius_(r) {}
12     double area() const override { return 3.14159 * radius_ *
        radius_; }
13     void draw() const override { std::cout << "Circle r=" <<
        radius_ << "\n"; }
14 };
```

14.4.2. С CRTP (статический полиморфизм)

```
1 template<typename Derived>
2 class Shape {
3 public:
4     double area() const {
5         return static_cast<const Derived*>(this)->area_impl();
6     }
7     void draw() const {
8         static_cast<const Derived*>(this)->draw_impl();
9     }
10 };
11
12 class Circle : public Shape<Circle> {
13     double radius_;
14 public:
15     explicit Circle(double r) : radius_(r) {}
16     double area_impl() const { return 3.14159 * radius_ * radius_;
        }
17     void draw_impl() const { std::cout << "Circle r=" << radius_
        << "\n"; }
18 };
19
20 class Square : public Shape<Square> {
21     double side_;
22 public:
23     explicit Square(double s) : side_(s) {}
24     double area_impl() const { return side_ * side_; }
25     void draw_impl() const { std::cout << "Square s=" << side_ <<
        "\n"; }
26 };
```

Использование с шаблонами вместо указателей на базовый класс:

```
1 // Шаблонная функция -- принимает любую фигуру
2 template<typename ShapeT>
3 void print_area(const Shape<ShapeT>& shape) {
4     std::cout << "Area: " << shape.area() << "\n";
5 }
6
7 int main() {
8     Circle c(5.0);
9     Square s(3.0);
10
11     print_area(c); // "Area: 78.5398"
12     print_area(s); // "Area: 9"
13
14     c.draw();      // "Circle r=5"
15     s.draw();      // "Square s=3"
16 }
```

Что происходит при компиляции:

- `Shape<Circle>::area()` инлайнится: компилятор подставляет `Circle::area_impl()` напрямую.
- Никакого `vptr`, никакого косвенного вызова через таблицу.
- `print_area` инстанцируется для каждого типа фигуры — получаются две отдельные, полностью оптимизированные функции.

14.5. Применение 2: Добавление функциональности (mixins)

CRTP позволяет базовому классу «добавлять» методы, которые используют интерфейс наследника. Это называется **mixin**.

14.5.1. Пример: автоматические операторы сравнения

До C++20 (до `operator<=>`) приходилось писать все 6 операторов вручную. CRTP позволяет вывести их из одного:

```
1 // Mixin: требует от наследника только operator< и operator==
2 template<typename Derived>
3 class Comparable {
4 public:
5     friend bool operator>(const Derived& a, const Derived& b) {
6         return b < a;
7     }
8     friend bool operator<=(const Derived& a, const Derived& b) {
9         return !(b < a);
10    }
11    friend bool operator>=(const Derived& a, const Derived& b) {
12        return !(a < b);
13    }
14    friend bool operator!=(const Derived& a, const Derived& b) {
```

```

15         return !(a == b);
16     }
17 };
18
19 class Temperature : public Comparable<Temperature> {
20     double value_;
21 public:
22     explicit Temperature(double v) : value_(v) {}
23
24     // Нужно определить только два оператора :
25     friend bool operator<(const Temperature& a, const Temperature&
26         b) {
27         return a.value_ < b.value_;
28     }
29     friend bool operator==(const Temperature& a, const Temperature
30         & b) {
31         return a.value_ == b.value_;
32     }
33 };
34
35 int main() {
36     Temperature t1(36.6), t2(37.0);
37
38     std::cout << (t1 < t2) << "\n";    // 1 -- определён вручную
39     std::cout << (t1 > t2) << "\n";    // 0 -- сгенерирован CRTP
40     std::cout << (t1 <= t2) << "\n";   // 1 -- сгенерирован CRTP
41     std::cout << (t1 != t2) << "\n";   // 1 -- сгенерирован CRTP
42 }

```

Примечание: в C++20 для этого есть оператор `<=>` (spaceship operator), который решает ту же задачу средствами языка. Но паттерн `mixin` через CRTP применим и к другим задачам.

14.5.2. Пример: автоматический оператор `!=` и постфиксный `++`

```

1 // Mixin для итераторов : из prefix ++ и == генерирует != и postfix
2 ++
3 template<typename Derived>
4 class IteratorMixin {
5 public:
6     // Постфиксный ++ через префиксный
7     Derived operator++(int) {
8         Derived old = static_cast<Derived&>(*this);
9         ++static_cast<Derived&>(*this); // Вызывает prefix ++
10        return old;
11    }
12
13    friend bool operator!=(const Derived& a, const Derived& b) {
14        return !(a == b);
15    }
16 };

```

```

16
17 class Counter : public IteratorMixin<Counter> {
18     int value_;
19 public:
20     explicit Counter(int v) : value_(v) {}
21
22     // Определяем только prefix ++ и ==
23     Counter& operator++() { ++value_; return *this; }
24
25     friend bool operator==(const Counter& a, const Counter& b) {
26         return a.value_ == b.value_;
27     }
28
29     int get() const { return value_; }
30 };
31
32 int main() {
33     Counter c(0);
34     Counter end(5);
35
36     while (c != end) {          // != сгенерирован CRTP
37         std::cout << c.get() << " ";
38         c++;                    // postfix ++ сгенерирован CRTP
39     }
40     // 0 1 2 3 4
41 }

```

14.6. Применение 3: Подсчёт объектов

CRTP позволяет создать счётчик экземпляров, **индивидуальный для каждого класса**:

```

1 template<typename Derived>
2 class Counter {
3     static inline int count_ = 0; // inline static (C++17)
4 public:
5     Counter() { ++count_; }
6     Counter(const Counter&) { ++count_; }
7     ~Counter() { --count_; }
8
9     static int alive() { return count_; }
10 };
11
12 class Dog : public Counter<Dog> {
13     std::string name_;
14 public:
15     explicit Dog(std::string n) : name_(std::move(n)) {}
16 };
17
18 class Cat : public Counter<Cat> {
19     std::string name_;

```

```

20 public:
21     explicit Cat(std::string n) : name_(std::move(n)) {}
22 };
23
24 int main() {
25     Dog d1("Шарик"), d2("Бобик");
26     Cat c1("Мурка");
27
28     std::cout << "Собак: " << Dog::alive() << "\n"; // 2
29     std::cout << "Кошек: " << Cat::alive() << "\n"; // 1
30
31     {
32         Dog d3("Пекс");
33         std::cout << "Собак: " << Dog::alive() << "\n"; // 3
34     }
35     std::cout << "Собак: " << Dog::alive() << "\n"; // 2
36 }

```

Почему без CRTP это не работает?

```

1 // Без CRTP: один счётчик на ВСЕ классы
2 class CounterBad {
3     static inline int count_ = 0;
4 public:
5     CounterBad() { ++count_; }
6     ~CounterBad() { --count_; }
7     static int alive() { return count_; }
8 };
9
10 class Dog : public CounterBad {};
11 class Cat : public CounterBad {};
12
13 // Dog::alive() и Cat::alive() -- это ОДИН И ТОТ ЖЕ счётчик !
14 // Потому что Dog и Cat наследуют один и тот же CounterBad

```

С CRTP Counter<Dog> и Counter<Cat> — это **разные классы**, каждый со своей статической переменной count_.

14.7. Применение 4: Singleton через CRTP

```

1 template<typename Derived>
2 class Singleton {
3 protected:
4     Singleton() = default;
5     ~Singleton() = default;
6
7     Singleton(const Singleton&) = delete;
8     Singleton& operator=(const Singleton&) = delete;
9
10 public:
11     static Derived& instance() {
12         static Derived inst; // Meyer's singleton

```

```

13         return inst;
14     }
15 };
16
17 class Logger : public Singleton<Logger> {
18     friend class Singleton<Logger>; // Даём доступ к private
19     конструктору
20     Logger() = default;
21
22 public:
23     void log(const std::string& msg) {
24         std::cout << "[LOG] " << msg << "\n";
25     }
26 };
27
28 class Config : public Singleton<Config> {
29     friend class Singleton<Config>;
30     Config() = default;
31
32     int port_ = 8080;
33 public:
34     int port() const { return port_; }
35 };
36
37 int main() {
38     Logger::instance().log("started"); // "[LOG] started"
39     std::cout << Config::instance().port() << "\n"; // 8080
40
41     // Logger l; // Ошибка компиляции ! Конструктор protected
42 }

```

14.8. Применение 5: Метод clone() без повторения кода

При глубоком наследовании каждый класс должен переопределять clone() с правильным типом. CRTP автоматизирует это:

```

1 #include <memory>
2
3 class Animal {
4 public:
5     virtual ~Animal() = default;
6     virtual std::unique_ptr<Animal> clone() const = 0;
7     virtual void speak() const = 0;
8 };
9
10 // CRTP прослойка-, которая реализует clone() за наследника
11 template<typename Derived>
12 class Cloneable : public Animal {
13 public:
14     std::unique_ptr<Animal> clone() const override {
15         // Derived ОБЯЗАНА иметь копирующий конструктор
16     }
17 };

```

```

16         return std::make_unique<Derived>(
17             static_cast<const Derived*>(*this)
18         );
19     }
20 };
21
22 // Теперь наследникам не нужно писать clone() вручную:
23 class Dog : public Cloneable<Dog> {
24     std::string name_;
25 public:
26     explicit Dog(std::string n) : name_(std::move(n)) {}
27     void speak() const override {
28         std::cout << name_ << " говорит: Гав!\n";
29     }
30 };
31
32 class Cat : public Cloneable<Cat> {
33     std::string name_;
34 public:
35     explicit Cat(std::string n) : name_(std::move(n)) {}
36     void speak() const override {
37         std::cout << name_ << " говорит: Мяу!\n";
38     }
39 };
40
41 int main() {
42     std::unique_ptr<Animal> dog = std::make_unique<Dog>("Шарик");
43     dog->speak(); // Шарик" говорит: Гав!"
44
45     auto cloned = dog->clone();
46     cloned->speak(); // Шарик" говорит: Гав!" -- это КОПИЯ
47 }

```

Без CRTP пришлось бы копировать реализацию `clone()` в каждый класс.

14.9. Применение 6: Статический интерфейс («концепты до концептов»)

До C++20 CRTP использовали для проверки интерфейса на этапе компиляции:

```

1 template<typename Derived>
2 class Printable {
3 public:
4     // Derived обязана иметь метод to_string()
5     friend std::ostream& operator<<(std::ostream& os, const
6         Derived& obj) {
7         os << obj.to_string();
8         return os;
9     }
10 };

```

```

11 class Point : public Printable<Point> {
12     double x_, y_;
13 public:
14     Point(double x, double y) : x_(x), y_(y) {}
15
16     std::string to_string() const {
17         return "(" + std::to_string(x_) + ", " + std::to_string(y_)
18             + ")";
19     };
20
21 class Color : public Printable<Color> {
22     int r_, g_, b_;
23 public:
24     Color(int r, int g, int b) : r_(r), g_(g), b_(b) {}
25
26     std::string to_string() const {
27         return "rgb(" + std::to_string(r_) + "," +
28             std::to_string(g_) + "," + std::to_string(b_) + ")"
29             ;
30     };
31
32 int main() {
33     Point p(1.5, 2.3);
34     Color c(255, 128, 0);
35
36     std::cout << p << "\n"; // "(1.500000, 2.300000)"
37     std::cout << c << "\n"; // "rgb(255,128,0)"
38 }

```

Наследник определяет `to_string()`, а CRTP-база автоматически генерирует `operator<<`. Если наследник забудет определить `to_string()` — ошибка компиляции.

14.10. Защита от ошибок

14.10.1. Проблема: неправильный шаблонный аргумент

```

1 class A : public Base<A> {};
2 class B : public Base<A> {}; // Ошибка! B передал A вместо B
3                               // static_cast<A*>(this)
                               // наобъекте B = UB

```

14.10.2. Решение: проверка в конструкторе

```

1 template<typename Derived>
2 class Base {
3     // Конструктор private, только Derived может вызывать через (
4     // наследование)

```

```

4     Base() = default;
5     friend Derived; // Только Derived имеет доступ к конструктору
6 public:
7     void interface() {
8         static_cast<Derived*>(this)->implementation();
9     }
10 };
11
12 class A : public Base<A> {}; // OK: A -- friend Base<A>
13 // class B : public Base<A> {}; // Ошибка компиляции !
14 // B не friend Base<A>,
// не может вызывать конструктор

```

14.10.3. Решение C++20: static_assert с концептами

```

1 template<typename Derived>
2 class Base {
3 public:
4     Base() {
5         // Проверяем на этапе компиляции, что Derived
// действительно наследник
6         static_assert(std::is_base_of_v<Base<Derived>, Derived>,
7             "CRTP: Derived must inherit from Base<
// Derived>");
8     }
9 };

```

14.11. CRTP и множественное наследование

Класс может наследовать **несколько** CRTP-баз — каждая добавляет свою функциональность:

```

1 template<typename Derived>
2 class Serializable {
3 public:
4     std::string serialize() const {
5         return static_cast<const Derived*>(this)->serialize_impl()
6         ;
7     }
8 };
9
10 template<typename Derived>
11 class Loggable {
12 public:
13     void log(const std::string& msg) const {
14         std::cout << "[" << static_cast<const Derived*>(this)->
15             class_name()
16             << "]" << msg << "\n";
17     }
18 };

```

```

16 };
17
18 // User наследуетоба mixina'
19 class User : public Serializable<User>, public Loggable<User> {
20     std::string name_;
21     int age_;
22 public:
23     User(std::string name, int age) : name_(std::move(name)), age_
        (age) {}
24
25     // Для Serializable
26     std::string serialize_impl() const {
27         return "{\\"name\\":\\" + name_ + "\",\\"age\\":\\" + std::
            to_string(age_) + "\\"}";
28     }
29
30     // Для Loggable
31     const char* class_name() const { return "User"; }
32 };
33
34 int main() {
35     User u("Alice", 25);
36     std::cout << u.serialize() << "\n"; // {"name":"Alice","age
        ":25}
37     u.log("created"); // [User] created
38 }

```

14.12. CRTP vs виртуальные функции: когда что?

Критерий	Виртуальные функции	CRTP
Тип известен на этапе компиляции	Не обязательно	Да, всегда
Нужен <code>vector<Base*></code>	Да, это единственный способ	Нет (разные разные типы)
Критична производительность	Косвенный вызов, нет инлайнинга	Нулевые затраты инлайнинга
Количество типов растёт в рантайме (плагины)	Подходит	Не подходит
Добавление общей функциональности (mixin)	Неудобно	Идеально
Простота понимания	Привычно, интуитивно	Требуется знание

Правило:

- Если набор типов известен **на этапе компиляции** и важна производительность — CRTP.
- Если нужен **runtime-полиморфизм** (контейнер разнородных объектов, плагины, фабрики) — виртуальные функции.

- Если нужно **добавить поведение** к разным классам (mixin) — CRTP.

14.13. Пример из реального мира: `std::enable_shared_from_this`

Стандартная библиотека использует CRTP в `std::enable_shared_from_this<T>`:

```

1 #include <memory>
2
3 class Widget : public std::enable_shared_from_this<Widget> {
4 public:
5     std::shared_ptr<Widget> get_ptr() {
6         return shared_from_this(); // Метод из CRTP-базы
7     }
8
9     void do_work() {
10        // Безопасно создаём shared_ptr на себя
11        auto self = shared_from_this();
12        // Можно передать self в асинхронную операцию,
13        // гарантируя, что объект не будет уничтожен
14    }
15 };
16
17 int main() {
18     auto w = std::make_shared<Widget>();
19     auto w2 = w->get_ptr(); // w и w2 разделяют владение
20
21     std::cout << w.use_count() << "\n"; // 2
22 }

```

`enable_shared_from_this<Widget>` — это CRTP-база. Она хранит внутри `weak_ptr<Widget>`, который инициализируется при создании `shared_ptr<Widget>`. Метод `shared_from_this()` возвращает `shared_ptr`, созданный из этого `weak_ptr`.

14.14. Deducing this (C++23) — замена CRTP?

C++23 ввёл **deducing this** — явный объектный параметр, который упрощает многие сценарии, ранее требовавшие CRTP:

```

1 // CRTP до( C++23):
2 template<typename Derived>
3 class Comparable {
4     friend bool operator!=(const Derived& a, const Derived& b) {
5         return !(a == b);
6     }
7 };
8
9 class Int : public Comparable<Int> {
10     int val_;
11 public:
12     explicit Int(int v) : val_(v) {}
13     friend bool operator==(const Int& a, const Int& b) {
14         return a.val_ == b.val_;
15     }
16 };

```

```

15     }
16 };

```

```

1 // Deducing this (C++23):
2 class Comparable23 {
3     // this передаётся явно, тип выводится автоматически
4     bool operator!=(this const auto& self, const auto& other) {
5         return !(self == other);
6     }
7 };
8
9 class Int23 : public Comparable23 {
10     int val_;
11 public:
12     explicit Int23(int v) : val_(v) {}
13     friend bool operator==(const Int23& a, const Int23& b) {
14         return a.val_ == b.val_;
15     }
16 };

```

С deducing this не нужен шаблонный параметр — компилятор сам выводит конкретный тип `self`. Код проще и безопаснее (нет риска передать неправильный тип в шаблон базы).

Однако CRTP не устарел:

- Deducing this доступен только с C++23
- CRTP работает и для статических членов (подсчёт объектов, singleton)
- Многие библиотеки и кодовые базы используют CRTP
- Понимание CRTP необходимо для чтения существующего кода

14.15. Итого: шпаргалка по CRTP

Применение	Суть
Статический полиморфизм	Вызов методов наследника без <code>virtual</code>
Mixin (добавление функциональности)	Генерация операторов, <code>operator<<</code> , <code>clone()</code> и т. д.
Подсчёт объектов	Отдельный <code>static</code> счётчик для каждого класса
Singleton	Шаблонный <code>instance()</code> для любого класса
<code>enable_shared_from_this</code>	Получение <code>shared_ptr</code> на <code>this</code>
Избежание дублирования	Один <code>clone()</code> , один <code>serialize()</code> в базе

15. Указатели на функции, методы, поля. Лямбды. Type erasure. `std::function`

15.1. Указатели на функции

15.1.1. Что такое указатель на функцию

Функция в C++ занимает определённое место в памяти (сегмент кода). Указатель на функцию хранит **адрес точки входа** функции и позволяет вызывать её косвенно — через указатель.

```
1 #include <iostream>
2
3 int add(int a, int b) { return a + b; }
4 int mul(int a, int b) { return a * b; }
5
6 int main() {
7     // Объявление указателя на функцию      int(int,int)
8     int (*op)(int, int) = nullptr;
9
10    op = add;           // имя функции неявно приводится к указателю
11    std::cout << op(2, 3) << "\n";    // 5
12
13    op = &mul;          // явное взятие адреса    -- тоже допустимо
14    std::cout << op(2, 3) << "\n";    // 6
15 }
```

Синтаксис объявления:

возвращаемый_тип (*имя)(список_параметров)

Скобки вокруг *имя обязательны: без них получится объявление функции, возвращающей указатель.

15.1.2. Использование `typedef` и `using`

Синтаксис указателей на функции громоздкий. Упрощаем:

```
1 // C-style typedef
2 typedef int (*BinaryOp)(int, int);
3
4 // C++11 using предпочтительно()
5 using BinaryOp = int(*)(int, int);
6
7 BinaryOp op = add;
8 std::cout << op(4, 5) << "\n";    // 9
```

15.1.3. Передача указателя на функцию как параметр

```

1 void apply_and_print(int a, int b, int (*func)(int, int)) {
2     std::cout << func(a, b) << "\n";
3 }
4
5 // Илис using:
6 using BinaryOp = int (*)(int, int);
7 void apply_and_print(int a, int b, BinaryOp func) {
8     std::cout << func(a, b) << "\n";
9 }
10
11 int main() {
12     apply_and_print(10, 20, add); // 30
13     apply_and_print(10, 20, mul); // 200
14 }

```

Это классический приём в С — callback-функции. Именно так работают `qsort`, `signal` и другие С-функции.

15.1.4. Массив указателей на функции

```

1 int sub(int a, int b) { return a - b; }
2
3 int main() {
4     BinaryOp ops[] = {add, mul, sub};
5     for (auto op : ops) {
6         std::cout << op(10, 3) << " "; // 13 30 7
7     }
8 }

```

15.1.5. Указатели на функции и шаблоны

Шаблонная функция не является одной функцией — это семейство. Нужно указать конкретную инстанциацию:

```

1 template<typename T>
2 T maximum(T a, T b) { return a > b ? a : b; }
3
4 int main() {
5     int (*p)(int, int) = maximum<int>; // ОК:
6     // конкретная инстанция
7     // int (*q)(int, int) = maximum; // Ошибка: какая именно?
8     std::cout << p(3, 7) << "\n"; // 7
9 }

```

15.1.6. Указатели на лямбда-функции (C++17)

С C++17 лямбда является частью типа функции:

```

1 void foo() noexcept {}
2 void bar() {}
3
4 void (*p1)() noexcept = foo;    // OK
5 void (*p2)()           = foo;    // OK: noexcept -> обычный неявное (
    приведение)
6 // void (*p3)() noexcept = bar; // Ошибка: обычный -> noexcept
    нельзя

```

15.2. Указатели на методы (member function pointers)

15.2.1. Синтаксис

Методы класса — это **не обычные функции**. Они принимают скрытый параметр `this`. Поэтому обычный указатель на функцию для них **не подходит**:

```

1 struct Calculator {
2     int add(int a, int b) { return a + b; }
3     int mul(int a, int b) { return a * b; }
4 };
5
6 int main() {
7     // Тип: указатель на метод   Calculator, принимающий (int,int),
7     // возвращающий int
8     int (Calculator::*method)(int, int) = &Calculator::add;
9
10    Calculator calc;
11    // Вызов через объект :
12    std::cout << (calc.*method)(2, 3) << "\n";    // 5
13
14    // Вызов через указатель на объект :
15    Calculator* ptr = &calc;
16    std::cout << (ptr->*method)(2, 3) << "\n";    // 5
17
18    method = &Calculator::mul;
19    std::cout << (calc.*method)(2, 3) << "\n";    // 6
20 }

```

Важно: оператор `&` при взятии адреса метода **обязателен** (в отличие от обычных функций, где имя неявно конвертируется в указатель).

15.2.2. Синтаксис с `using`

```

1 using CalcMethod = int (Calculator::*)(int, int);
2 CalcMethod m = &Calculator::add;

```

15.2.3. Указатели на `const`-методы

```

1 struct Foo {
2     int value() const { return 42; }
3 };
4
5 // Обратителен внимание на const:
6 int (Foo::*p)() const = &Foo::value;
7
8 const Foo f;
9 std::cout << (f.*p)() << "\n"; // 42

```

15.2.4. Указатели на статические методы

Статические методы **не имеют** `this`, поэтому для них используется **обычный** указатель на функцию:

```

1 struct Utils {
2     static int square(int x) { return x * x; }
3 };
4
5 int (*p)(int) = &Utils::square; // обычный указатель на функцию !
6 std::cout << p(5) << "\n"; // 25

```

15.2.5. Использование `std::invoke` (C++17)

Операторы `.*` и `->*` неудобны. `std::invoke` унифицирует вызов любого callable-объекта:

```

1 #include <functional>
2
3 Calculator calc;
4 auto method = &Calculator::add;
5
6 // Вместо (calc.*method)(2, 3):
7 std::cout << std::invoke(method, calc, 2, 3) << "\n"; // 5
8
9 // Работает и с указателем на объект:
10 std::cout << std::invoke(method, &calc, 2, 3) << "\n"; // 5

```

15.3. Указатели на поля (pointer to data member)

15.3.1. Синтаксис

Можно взять «адрес» поля класса. На самом деле это не настоящий адрес, а **сместе-ние** поля относительно начала объекта:

```

1 struct Point {
2     double x;
3     double y;
4     double z;

```

```

5 };
6
7 int main() {
8     // Указатель на поле типа double класса Point
9     double Point::*coord = &Point::x;
10
11     Point p{1.0, 2.0, 3.0};
12
13     std::cout << p.*coord << "\n";    // 1.0 (x)
14
15     coord = &Point::y;
16     std::cout << p.*coord << "\n";    // 2.0 (y)
17
18     coord = &Point::z;
19     std::cout << p.*coord << "\n";    // 3.0 (z)
20 }

```

15.3.2. Практическое применение: обобщённый доступ

```

1 struct Student {
2     std::string name;
3     int age;
4     double gpa;
5 };
6
7 // Функция, сортирующая по полю
8 template<typename T>
9 void sort_by(std::vector<Student>& v, T Student::*field) {
10     std::sort(v.begin(), v.end(),
11         [field](const Student& a, const Student& b) {
12             return a.*field < b.*field;
13         });
14 }
15
16 int main() {
17     std::vector<Student> students = {
18         {"Alice", 20, 3.8},
19         {"Bob", 22, 3.5},
20         {"Carol", 19, 3.9}
21     };
22
23     sort_by(students, &Student::age);    // по возрасту
24     sort_by(students, &Student::gpa);    // по GPA
25     sort_by(students, &Student::name);    // по имени
26 }

```

15.3.3. Доступ через std::invoke

std::invoke работает и с указателями на поля:

```

1 Student s{"Alice", 20, 3.8};
2 auto ptr = &Student::name;
3 std::cout << std::invoke(ptr, s) << "\n"; // "Alice"

```

15.4. Лямбда-выражения

15.4.1. Мотивация

До C++11 для передачи «кусочка логики» в алгоритм нужно было писать отдельную функцию или функтор (класс с `operator()`):

```

1 // Функтор -- классс operator()
2 struct IsEven {
3     bool operator()(int x) const { return x % 2 == 0; }
4 };
5
6 std::vector<int> v = {1, 2, 3, 4, 5, 6};
7 auto it = std::find_if(v.begin(), v.end(), IsEven{});

```

Это слишком многословно для простой проверки. Лямбды решают эту проблему.

15.4.2. Базовый синтаксис

```

1 auto it = std::find_if(v.begin(), v.end(),
2     [](int x) { return x % 2 == 0; });

```

Общий вид лямбда-выражения:

[захват](параметры) спецификаторы -> тип_возврата { тело }

- [захват] — какие переменные из окружающего контекста доступны внутри лямбды;
- (параметры) — как у обычной функции;
- спецификаторы — `mutable`, `noexcept`, `constexpr` и др.;
- -> тип_возврата — можно опустить, если компилятор выведет сам;
- { тело } — код лямбды.

15.4.3. Что такое лямбда на самом деле?

Лямбда — это **синтаксический сахар** для создания анонимного класса с `operator()`. Компилятор генерирует примерно следующее:

```

1 // Лямбда:
2 auto f = [](int x) { return x * 2; };
3
4 // Компиляторгенерируетчтото - вроде:

```

```

5 class __lambda_1 {
6 public:
7     int operator()(int x) const { // const по умолчанию !
8         return x * 2;
9     }
10 };
11 auto f = __lambda_1{};

```

Каждая лямбда имеет **уникальный тип**, даже если тело идентично:

```

1 auto f1 = [](int x) { return x; };
2 auto f2 = [](int x) { return x; };
3 // decltype(f1) != decltype(f2) -- это разные типы !

```

15.4.4. Захват переменных (capture)

[] — без захвата. Лямбда не видит локальные переменные:

```

1 int threshold = 10;
2 // auto f = [](int x) { return x > threshold; }; // Ошибка!

```

[=] --- захват всех по значению (копия).

```

1 int threshold = 10;
2 auto f = [=](int x) { return x > threshold; };
3 // threshold скопирован внутри лямбды
4 threshold = 100; // не повлияет на f
5 std::cout << f(50); // true сравнивается с 10

```

[&] --- захват всех по ссылке.

```

1 int count = 0;
2 auto increment = [&]() { ++count; };
3 increment();
4 increment();
5 std::cout << count; // 2

```

Осторожно: захваченная ссылка может стать висячей!

```

1 std::function<int()> make_counter() {
2     int n = 0;
3     return [&n]() { return ++n; }; // ОШИБКА: n
        уничтожен при выходе !
4 }

```

Поимённый захват

```

1 int a = 1, b = 2, c = 3;
2 auto f = [a, &b](int x) {
3     // a -- копия (read-only, тк.. operator() const)
4     // b -- ссылка (можно менять)
5     // c -- не захвачен, недоступен
6     b += a + x;
7 };

```

Смешанный захват

```
1 auto f = [=, &b]() { ... }; // всепозначению , b посылке
2 auto g = [&, a]() { ... }; // всепосылке , a позначению
```

15.4.5. Захват this

Внутри метода класса лямбда может захватить this:

```
1 class Timer {
2     int interval_ = 100;
3 public:
4     auto make_callback() {
5         // [this] -- захват указателя this
6         return [this]() {
7             std::cout << "interval = " << interval_ << "\n";
8         };
9     }
10
11     auto make_safe_callback() {
12         // [*this] (C++17) -- захват копии всего объекта
13         return [*this]() {
14             std::cout << "interval = " << interval_ << "\n";
15         };
16     }
17 };
```

[this] — захватывает **указатель**. Если объект уничтожен, будет dangling pointer.

[*this] (C++17) — захватывает **копию** объекта. Безопасно, но дорого для больших объектов.

15.4.6. Init-capture (обобщённый захват, C++14)

Позволяет создать новую переменную внутри захвата:

```
1 // Переместить вектор в лямбду (не копировать):
2 std::vector<int> data = {1, 2, 3, 4, 5};
3 auto f = [data = std::move(data)]() {
4     for (int x : data) std::cout << x << " ";
5 };
6 // data здесь пуст -- он перемещён в лямбду
7
8 // Захват с переименованием:
9 int x = 42;
10 auto g = [value = x * 2]() { return value; }; // value == 84
11
12 // Захват unique_ptr не копируемый (тип):
13 auto ptr = std::make_unique<int>(42);
14 auto h = [p = std::move(ptr)]() { return *p; };
```

15.4.7. Ключевое слово mutable

По умолчанию `operator()` лямбды — `const`. Чтобы изменять захваченные по значению переменные, нужен `mutable`:

```
1 int counter = 0;
2 auto inc = [counter]() mutable {
3     return ++counter; // OK: mutable позволяет менять копию
4 };
5 std::cout << inc() << "\n"; // 1
6 std::cout << inc() << "\n"; // 2
7 std::cout << counter << "\n"; // 0 -- оригинал не изменён !
```

Без `mutable`:

```
1 auto bad = [counter]() {
2     // return ++counter; // Ошибка: operator() const
3 };
```

15.4.8. Лямбда без захвата конвертируется в указатель на функцию

Лямбда **без захвата** может быть неявно преобразована в указатель на функцию с подходящей сигнатурой:

```
1 int (*fp)(int, int) = [](int a, int b) { return a + b; };
2 std::cout << fp(3, 4) << "\n"; // 7
3
4 // Это позволяет передавать лямбды в С API:
5 void c_api(void (*callback)(int)) { callback(42); }
6 c_api([](int x) { std::cout << x << "\n"; }); // OK
```

Лямбда **с захватом** в указатель на функцию **не конвертируется**:

```
1 int offset = 10;
2 // int (*fp)(int) = [offset](int x) { return x + offset; }; //
   Ошибка!
```

15.4.9. Generic-лямбды (C++14)

Параметры лямбды можно объявить как `auto` — компилятор сделает шаблонный `operator()`:

```
1 auto print = [](const auto& x) {
2     std::cout << x << "\n";
3 };
4 print(42); // int
5 print(3.14); // double
6 print("hello"s); // std::string
```

Что генерирует компилятор:

```
1 class __lambda {
2 public:
3     template<typename T>
```

```

4     void operator()(const T& x) const {
5         std::cout << x << "\n";
6     }
7 };

```

15.4.10. Шаблонные лямбды (C++20)

C++20 позволяет явно указать шаблонные параметры:

```

1 // Ограничение: только целочисленные типы
2 auto sum = []<typename T>(const std::vector<T>& v) requires std::
   integral<T> {
3     T result = 0;
4     for (auto x : v) result += x;
5     return result;
6 };
7
8 std::vector<int> vi = {1, 2, 3};
9 std::cout << sum(vi) << "\n"; // 6
10
11 // std::vector<double> vd = {1.0, 2.0};
12 // sum(vd); // Ошибка: double не integral
13 \end{lstlisting}
14
15 \subsection{Рекурсивные лямбды}
16     Лямбда не может вызывать сама себя напрямую
17     , потому что \texttt{auto} ещё не выведен в момент
18     ~ определения тела . Решения:
19
20 \begin{lstlisting}
21 // 1. Через std::function с накладными расходами ):
22 std::function<int(int)> factorial = [&factorial](int n) -> int {
23     return n <= 1 ? 1 : n * factorial(n - 1);
24 };
25
26 // 2. Передача самой себя как параметра без ( накладных расходов ):
27 auto factorial2 = [](auto self, int n) -> int {
28     return n <= 1 ? 1 : n * self(self, n - 1);
29 };
30 std::cout << factorial2(factorial2, 5) << "\n"; // 120
31
32 // 3. C++23: deducing this
33 auto factorial3 = [](this auto self, int n) -> int {
34     return n <= 1 ? 1 : n * self(n - 1);
35 };

```

15.4.11. Immediately Invoked Lambda Expression (IILE)

Лямбду можно вызвать сразу на месте — удобно для инициализации сложных const-переменных:

```
1 const auto config = [&]() {  
2     Config c;  
3     c.width = parse_int(argv[1]);  
4     c.height = parse_int(argv[2]);  
5     c.fullscreen = (argc > 3);  
6     return c;  
7 }(); // <-- вызовнаместе !  
8 // config -- const, инициализированодинраз
```

15.5. Указатели на функции, функторы и лямбды — сравнение

Свойство	Указатель	Функтор	Лямбда
Состояние (захват)	Нет	Да	Да
inline-оптимизация	Нет ¹	Да	Да
Передача в C API	Да	Нет	Только без захвата
Тип известен	Да	Да	Нет ²

15.6. Type erasure (стирание типа)

15.6.1. Проблема

Мы хотим хранить callable-объекты разных типов в одном контейнере или передавать их через единый интерфейс. Но:

- Указатели на функции не хранят состояние.
- Каждая лямбда — уникальный тип, нельзя положить в один `std::vector`.
- Шаблоны — compile-time решение, а нам нужен runtime полиморфизм.

```
1 auto f = [x=1](int a) { return a + x; };  
2 auto g = [y=2](int a) { return a * y; };  
3 // decltype(f) != decltype(g)  
4  
5 // std::vector<???
```

15.6.2. Идея type erasure

Type erasure — техника, позволяющая «стереть» конкретный тип объекта, сохранив возможность вызывать его операции через единый интерфейс. Это runtime-полиморфизм **без** наследования пользователем от базового класса.

Концептуально type erasure состоит из трёх компонентов:

1. **Внешний класс** (wrapper) — то, с чем работает пользователь. Имеет value-семантику (копируется, перемещается).
2. **Внутренний абстрактный интерфейс** — определяет набор операций.
3. **Шаблонная реализация** — конкретная обёртка вокруг пользовательского типа, реализующая интерфейс.

15.6.3. Пример: собственный AnyCallable

Реализуем упрощённый аналог `std::function<int(int)>`:

```

1  class AnyCallable {
2      // Шаг 1: абстрактный интерфейс
3      struct Concept {
4          virtual int call(int) = 0;
5          virtual ~Concept() = default;
6          virtual std::unique_ptr<Concept> clone() const = 0;
7      };
8
9      // Шаг 2: шаблонная реализация для любого callable
10     template<typename F>
11     struct Model : Concept {
12         F func_;
13         Model(F f) : func_(std::move(f)) {}
14         int call(int x) override { return func_(x); }
15         std::unique_ptr<Concept> clone() const override {
16             return std::make_unique<Model>(func_);
17         }
18     };
19
20     // Шаг 3: хранение через указатель на базу
21     std::unique_ptr<Concept> ptr_;
22
23 public:
24     // Шаблонный конструктор -- принимает что угодно callable
25     template<typename F>
26     AnyCallable(F f) : ptr_(std::make_unique<Model<F>>(std::move(f))) {}
27
28     // Копирование через ( clone )
29     AnyCallable(const AnyCallable& other) : ptr_(other.ptr_->clone()) {}
30     AnyCallable& operator=(const AnyCallable& other) {
31         ptr_ = other.ptr_->clone();
32         return *this;
33     }
34
35     // Перемещение
36     AnyCallable(AnyCallable&&) = default;
37     AnyCallable& operator=(AnyCallable&&) = default;
38

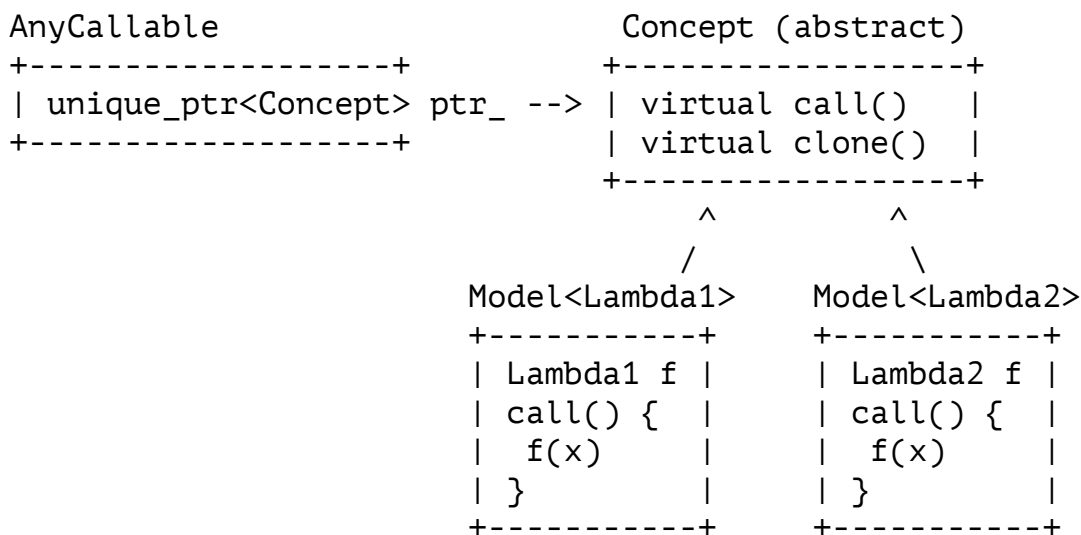
```

```

39 // Вызов
40 int operator()(int x) { return ptr_->call(x); }
41 };
42
43 int main() {
44     AnyCallable f = [x=1](int a) { return a + x; };
45     AnyCallable g = [y=2](int a) { return a * y; };
46
47     std::vector<AnyCallable> funcs = {f, g};
48     for (auto& func : funcs) {
49         std::cout << func(10) << " "; // 11 20
50     }
51 }

```

15.6.4. Как это работает (диаграмма)



Ключевая идея: шаблонный конструктор AnyCallable(F f) «стирает» тип F, оборачивая его в Model<F>, а хранит указатель на нешаблонный Concept.

15.6.5. Type erasure в стандартной библиотеке

Type erasure — не экзотика. Стандартная библиотека активно его использует:

Класс	Что стирает
std::function<R(Args...)>	Любой callable с подходящей сигнатурой
std::any (C++17)	Любой копируемый тип
std::move_only_function (C++23)	Любой move-only callable
std::format_args	Аргументы форматирования

15.7. std::function

15.7.1. Что это

`std::function<R(Args...)>` — обёртка из `<functional>`, которая может хранить **любой** вызываемый объект с подходящей сигнатурой:

- обычную функцию;
- лямбду (в том числе с захватом);
- функтор (объект с `operator()`);
- указатель на метод (через `std::bind` или лямбду);
- результат `std::bind`.

```
1  #include <functional>
2
3  int add(int a, int b) { return a + b; }
4
5  struct Multiplier {
6      int factor;
7      int operator()(int x) const { return x * factor; }
8  };
9
10 int main() {
11     // Храни тобывчную функцию
12     std::function<int(int, int)> f1 = add;
13     std::cout << f1(2, 3) << "\n";          // 5
14
15     // Храни тлямбду с захватом
16     int offset = 10;
17     std::function<int(int)> f2 = [offset](int x) { return x +
18         offset; };
19     std::cout << f2(5) << "\n";              // 15
20
21     // Храни тфунктор
22     std::function<int(int)> f3 = Multiplier{3};
23     std::cout << f3(7) << "\n";              // 21
24 }
```

15.7.2. Проверка на пустоту

`std::function` может быть пустой (не содержать callable):

```
1  std::function<void()> f;          // пустая
2  if (f) {                          // проверка на непустоту
3      f();
4  } else {
5      std::cout << "empty!\n";      // сюда попадём
6  }
```

```

7
8 f = []() { std::cout << "hello\n"; };
9 if (f) f(); // "hello"
10
11 f = nullptr; // сновапустая
12 // f(); // выбросит std::bad_function_call!

```

15.7.3. Накладные расходы `std::function`

`std::function` использует type erasure внутри, что влечёт расходы:

1. **Динамическая аллокация.** Если callable не помещается в SBO (Small Buffer Optimization, обычно 16–32 байт), выделяется память в куче.
2. **Виртуальный вызов.** Каждый `operator()` — это косвенный вызов через vtable (не инлайнится).
3. **Копирование.** При копировании `std::function` копируется и хранимый callable.

Когда это важно:

```

1 // Плохо: std::function в hot loop
2 void process(const std::vector<int>& data,
3             std::function<int(int)> transform) { // type erasure
4     for (int x : data)
5         result.push_back(transform(x)); //
6         косвенныйвызовнакаждойитерации
7 }
8 // Лучше: шаблонныйпараметр (zero-cost)
9 template<typename F>
10 void process(const std::vector<int>& data, F transform) {
11     for (int x : data)
12         result.push_back(transform(x)); // прямойвызов ,
13         инлайнится
14 }

```

Правило: используйте `std::function`, когда вам **нужно хранить** callable (в поле класса, в контейнере, в callback). Для **передачи** callable в функцию предпочитайте шаблоны.

15.7.4. Хранение указателей на методы

```

1 struct Greeter {
2     std::string name;
3     void hello() const {
4         std::cout << "Hello, " << name << "!\n";
5     }
6     void hello_n(int n) const {
7         for (int i = 0; i < n; ++i)

```

```

8         std::cout << "Hello, " << name << "!\n";
9     }
10 };
11
12 // Метод без аргументов : первый аргумент -- объект
13 std::function<void(const Greeter&)> f = &Greeter::hello;
14 Greeter g{"Alice"};
15 f(g); // "Hello, Alice!"
16
17 // Метод с аргументами :
18 std::function<void(const Greeter&, int)> f2 = &Greeter::hello_n;
19 f2(g, 3); // три раза "Hello, Alice!"
20
21 // Привязка объекта через лямбду :
22 Greeter bob{"Bob"};
23 std::function<void()> f3 = [&bob]() { bob.hello(); };
24 f3(); // "Hello, Bob!"

```

15.7.5. std::move_only_function (C++23)

std::function требует, чтобы хранимый callable был **копируемым**. Это не работает с unique_ptr в захвате:

```

1 auto ptr = std::make_unique<int>(42);
2 // std::function<int()> f = [p = std::move(ptr)]() { return *p; };
3 // Ошибка: лямбда не копируемая !

```

C++23 вводит std::move_only_function — аналог, который перемещается, но не копируется:

```

1 #include <functional>
2
3 auto ptr = std::make_unique<int>(42);
4 std::move_only_function<int()> f = [p = std::move(ptr)]() { return
    *p; };
5 std::cout << f() << "\n"; // 42
6 // auto g = f; // Ошибка: нельзя копировать
7 auto g = std::move(f); // OK: перемещение

```

15.7.6. Пример: система событий (observer pattern)

```

1 #include <functional>
2 #include <vector>
3 #include <string>
4
5 class EventBus {
6     std::unordered_map<std::string,
7         std::vector<std::function<void(const std::string&>>>
8         handlers_;
9 public:

```

```

9     void subscribe(const std::string& event,
10                   std::function<void(const std::string&)> handler
11                   ) {
12         handlers_[event].push_back(std::move(handler));
13     }
14
15     void emit(const std::string& event, const std::string& data) {
16         if (auto it = handlers_.find(event); it != handlers_.end()) {
17             for (auto& handler : it->second) {
18                 handler(data);
19             }
20         }
21     };
22
23     int main() {
24         EventBus bus;
25
26         bus.subscribe("click", [](const std::string& data) {
27             std::cout << "Button clicked: " << data << "\n";
28         });
29
30         bus.subscribe("click", [](const std::string& data) {
31             std::cout << "Logging click: " << data << "\n";
32         });
33
34         bus.subscribe("keypress", [](const std::string& data) {
35             std::cout << "Key pressed: " << data << "\n";
36         });
37
38         bus.emit("click", "OK");
39         // Button clicked: OK
40         // Logging click: OK
41
42         bus.emit("keypress", "Enter");
43         // Key pressed: Enter
44     }

```

15.8. Итого

Механизм	Когда использовать
Указатель на функцию	Взаимодействие с C API; compile-time dispatch таблицы
Указатель на метод	Обобщённый вызов методов; reflection-подобные паттерны
Указатель на поле	Обобщённый доступ к полям; сортировка по полю
Лямбда + auto/шаблон	Передача логики в алгоритмы; zero-cost абстракция
<code>std::function</code>	Хранение callback'ов; runtime полиморфизм callable
Type erasure (свой)	Когда нужен runtime полиморфизм без навязывания базового класса

16. Move-семантика в C++: value categories, rvalue references, perfect forwarding

16.1. Value Categories (Категории значений)

Главное заблуждение относительно move-семантики заключается в том, что категории **lvalue** и **rvalue** — это свойства объектов. На самом деле — это **синтаксические свойства выражений** (expressions). Один и тот же объект в разных строчках кода может быть представлен выражениями разных категорий.

Иерархия категорий выглядит следующим образом:

Группа	Категории
expression	glvalue, rvalue
glvalue	lvalue, xvalue
rvalue	xvalue, prvalue

16.1.1. Lvalue (Long-lived)

Выражения, имеющие имя, адрес в памяти и устойчивую сущность:

- Любой идентификатор (имя переменной) — всегда **lvalue**.
- Результаты присваивания (`=`, `+=`), префиксный инкремент/декремент (`++x`), результат разыменования (`*p`).
- Результат функции, возвращающей **lvalue**-ссылку (T&).

16.1.2. Rvalue (Short-lived)

Временные значения, которые будут уничтожены сразу после использования:

- Литералы (кроме строковых), результат арифметических операций ($a + b$), постфиксный инкремент/декремент ($x++$).
- Результат функции, возвращающей тип по значению (T) или `rvalue`-ссылку ($T\&\&$).
- Результат приведения типа (**Cast**) к типу, не являющемуся `lvalue`-ссылкой.

16.1.3. Тернарный оператор и оператор запятая

Результат тернарного оператора является `lvalue` только если оба его подвыражения — `lvalue`. Оператор запятая принимает категорию своего **правого** операнда.

16.2. RValue Reference (Rvalue-ссылки)

Это новый вид ссылок ($T\&\&$), введённый для реализации семантики перемещения.

- **Инициализация:** `rvalue`-ссылку можно инициализировать только `rvalue`-выражениями. Попытка привязать её к `lvalue` вызовет ошибку компиляции.
- **Свойство имени:** если `rvalue`-ссылке дано имя (например, `int&& x = 5;`), то само выражение `x` является `lvalue`, так как это идентификатор.
- **Константность:** нельзя привязать неконстантную ссылку к константному объекту — обычные правила константности сохраняются.

```
1 int&& r = 42;      // OK: привязываемк rvalue
2 // int&& r2 = r; // Ошибка: r — lvalue y( негоестьимя )
3
4 const int&& cr = 10; // OK
5 // int&& r3 = cr;   // Ошибка: нельзяотбросить const
```

16.3. Move Semantics (Семантика перемещения)

Семантика перемещения позволяет «забирать» ресурсы у временных объектов вместо их копирования, что критически важно для производительности.

16.3.1. std::move

`std::move` **ничего не перемещает физически** и не генерирует процессорных инструкций. Это инструмент для компилятора, преобразующий `lvalue` в `rvalue`.

Реализация `std::move`:

```
1 template<typename T>
2 constexpr std::remove_reference_t<T>&& move(T&& t) noexcept {
3     return static_cast<std::remove_reference_t<T>&&>(t);
4 }
```

- Принимает аргумент по **универсальной ссылке** (`T&&`), чтобы работать с любыми категориями значений.
- Использует `std::remove_reference_t<T>`, чтобы получить чистый тип без ссылок.
- Выполняет `static_cast` к типу `чистый_тип&&`.

Результат: выражение `std::move(x)` всегда является `rvalue`, что заставляет компилятор выбирать **перемещающий конструктор** или **оператор присваивания перемещением**.

```
1 std::vector<int> a = {1, 2, 3};
2 std::vector<int> b = std::move(a); // a теперь в "valid but
   unspecified" состоянии
```

16.4. Perfect Forwarding (Идеальная передача)

Проблема идеальной передачи возникает в функциях-обёртках (например, `emplace_back`), которые должны передать аргументы дальше, сохранив их оригинальную категорию значения.

16.4.1. Универсальные ссылки (Universal References)

Если параметр шаблона функции имеет вид `T&&`, он может принимать и `lvalue`, и `rvalue`:

```
1 template<typename T>
2 void wrapper(T&& arg) {
3     // arg может быть lvalue ссылкой- или rvalue ссылкой-
4     // в зависимости от T
5 }
```

16.4.2. Сворачивание ссылок (Reference Collapsing)

При выводе типов шаблонов компилятор «схлопывает» ссылки по следующим правилам:

Т выводится как	Параметр	Результат
X&	T&&	X& & & → X&
X&	T&&	X& && → X&
X&&	T&&	X&& & → X&
X&&	T&&	X&& && → X&&

Правило одной строкой: **только два rvalue дают rvalue**, любое & «заражает» результат.

16.4.3. std::forward

std::forward необходим, чтобы внутри функции превратить аргумент обратно в rvalue, если он изначально был таковым (поскольку внутри функции у него есть имя и он стал lvalue):

```
1 template<typename T>
2 void wrapper(T&& arg) {
3     target(std::forward<T>(arg)); // сохраняем категорию значения
4 }
```

```
1 // Реализация std::forward упрощённо():
2 template<typename T>
3 T&& forward(std::remove_reference_t<T>& t) noexcept {
4     return static_cast<T&&>(t);
5 }
```

16.5. Variadic Templates (Шаблоны с переменным количеством аргументов)

В контексте perfect forwarding шаблоны с переменным числом аргументов позволяют принять **произвольное число аргументов любых категорий** и передать их дальше без потери информации о категориях:

```
1 template<typename... Args>
2 void emplace_back_impl(Args&&... args) {
3     // каждый аргумент в пакете может иметь свою категорию значения
4     construct(std::forward<Args>(args)...);
5 }
```

- Args&&... args — пакет универсальных ссылок; каждый элемент независимо выводит свою категорию.
- std::forward<Args>(args)... — раскрывает пакет, применяя forward к каждому элементу отдельно.
- Такой подход используется в std::vector::emplace_back, std::make_unique, std::make_shared и других фабричных функциях стандартной библиотеки.

```
1 // Пример: своя фабрика объектов      perfect forwarding
2 template<typename T, typename... Args>
3 T* create(Args&&... args) {
4     return new T(std::forward<Args>(args)...);
5 }
6
7 struct Point { int x, y; };
8 auto* p = create<Point>(1, 2); // конструируется напрямую ,
    без лишних копий
```